构建指南

发行版本 *10.10*

Sage 开发团队

2026 年 10 月 28 日

Contents

1 微积分 **3**
- 1.1 微分 . . . 3
- 1.2 积分 . . . 6
- 1.3 常微分方程 . . . 9
- 1.4 周期函数的傅里叶级数 . . . 11

2 绘图 **13**
- 2.1 二维函数绘图 . . . 13
- 2.2 绘制曲线 . . . 14
- 2.3 openmath . . . 17
- 2.4 Tachyon 3D 绘图 . . . 18
- 2.5 gnuplot . . . 18
- 2.6 绘制曲面 . . . 19

3 群 **21**
- 3.1 置换群 . . . 21
- 3.2 共轭类 . . . 23
- 3.3 正规子群 . . . 24
- 3.4 中心 . . . 25
- 3.5 群 id 数据库 . . . 26
- 3.6 小于 32 阶的群的构建指令 . . . 27
- 3.7 小于等于 15 阶的有限呈示群的构建说明 . . . 38

4 线性代数 **43**
- 4.1 向量空间 . . . 43
- 4.2 矩阵幂 . . . 43
- 4.3 核 . . . 44
- 4.4 特征向量和特征值 . . . 47
- 4.5 行化简 . . . 50
- 4.6 特征多项式 . . . 51
- 4.7 求解线性方程组 . . . 52

5 线性码与密码 **55**
- 5.1 码 . . . 55
- 5.2 密码 . . . 58

6 表示论 **63**

6.1 普通特征标 (Ordinary characters) . . . 63
6.2 布劳尔特征标 (Brauer characters) . . . 67

7 环 **71**
7.1 矩阵环 . . . 71
7.2 多项式环 . . . 72
7.3 p-进数 . . . 73
7.4 多项式的商环 . . . 74

8 多项式 **77**
8.1 多项式幂 . . . 77
8.2 因式分解 . . . 79
8.3 多项式最大公约数 . . . 79
8.4 多项式的根 . . . 81
8.5 多元函数求值 . . . 82
8.6 多元多项式的根 . . . 83
8.7 Gröbner 基 . . . 83

9 初等数论 **85**
9.1 计算模幂 . . . 85
9.2 离散对数 . . . 85
9.3 质数 . . . 86
9.4 因子 . . . 87
9.5 二次剩余 . . . 87

10 模形式 **89**
10.1 尖点形式 . . . 89
10.2 陪集表示 . . . 90
10.3 模符号和 Hecke 算子 . . . 90
10.4 亏格公式 (Genus formulas) . . . 91

11 椭圆曲线 **93**
11.1 导子 . . . 93
11.2 j-不变量 . . . 93
11.3 E 上的 $GF(q)$-有理点 . . . 94
11.4 与 **Q** 上椭圆曲线相关的模形式 . . . 95

12 数域 **97**
12.1 分歧 (Ramification) . . . 97
12.2 类数 (Class numbers) . . . 98
12.3 整基 (Integral basis) . . . 101

13 代数几何 **103**
13.1 曲线上点的计数 . . . 103
13.2 使用 Singular 计算 Riemann-Roch 空间 . . . 112

14 接口问题 **119**
14.1 后台作业 . . . 119
14.2 引用 Sage . . . 119
14.3 将 Sage 会话保存到日志 . . . 119
14.4 LaTeX 转换 . . . 120
14.5 Sage 与其他计算机代数系统 . . . 120
14.6 命令行上的 Sage 帮助 . . . 121
14.7 读取和导入文件到 Sage . . . 121
14.8 Sage 命令的 Python 语言程序代码 . . . 122

14.9 Sage 中的“特殊函数” 123
14.10 Sage 是什么? 124

15 文档贡献 **125**

16 索引与表格 **127**

索引 **129**

本文档收集了一些关于“如何在 Sage 中构建...”相关问题的答案。尽管在使用手册、教程或 Python 代码的文档中可以找到许多这方面的材料，但希望本文档能够为一般用户提供一些如何开始有效使用 Sage 的基本示例。

此作品采用 Creative Commons Attribution-Share Alike 3.0 License 许可。

请将建议、补充、修正发送到 `sage-devel` Google 讨论组!

Sage wiki http://wiki.sagemath.org/ 包含丰富的信息。此外，所有内容均可在 Sage 网页界面 http://sagecell.sagemath.org/ 上测试。

CHAPTER 1

微积分

这里有一些使用 Sage 进行微积分符号计算的例子。

1.1 微分

微分：

```
sage: var('x k w')
(x, k, w)
sage: f = x^3 * e^(k*x) * sin(w*x); f
x^3*e^(k*x)*sin(w*x)
sage: f.diff(x)
w*x^3*cos(w*x)*e^(k*x) + k*x^3*e^(k*x)*sin(w*x) + 3*x^2*e^(k*x)*sin(w*x)
sage: latex(f.diff(x))
w x^{3} \cos\left(w x\right) e^{\left(k x\right)} + k x^{3} e^{\left(k x\right)} \sin\left(w x\
↪right) + 3 \, x^{2} e^{\left(k x\right)} \sin\left(w x\right)
```

```
>>> from sage.all import *
>>> var('x k w')
(x, k, w)
>>> f = x**Integer(3) * e**(k*x) * sin(w*x); f
x^3*e^(k*x)*sin(w*x)
>>> f.diff(x)
w*x^3*cos(w*x)*e^(k*x) + k*x^3*e^(k*x)*sin(w*x) + 3*x^2*e^(k*x)*sin(w*x)
>>> latex(f.diff(x))
w x^{3} \cos\left(w x\right) e^{\left(k x\right)} + k x^{3} e^{\left(k x\right)} \sin\left(w x\
↪right) + 3 \, x^{2} e^{\left(k x\right)} \sin\left(w x\right)
```

如果你输入 `view(f.diff(x))`，将会打开一个新的窗口显示编译后的输出。在 notebook 中，你可以在单元格中输入

```
var('x k w')
f = x^3 * e^(k*x) * sin(w*x)
```

(续下页)

(接上页)

```
show(f)
show(f.diff(x))
```

然后按 shift-enter 来获得类似的结果。

你还可以在 notebook 单元格中，使用以下命令进行微分和积分

```
R  = PolynomialRing(QQ,"x")
x  = R.gen()
p  = x^2  +  1
show(p.derivative())
show(p.integral())
```

或者在命令行中，使用以下命令进行微分和积分

```
sage: R  = PolynomialRing(QQ,"x")
sage: x  = R.gen()
sage: p  = x^2  +  1
sage: p.derivative()
2*x
sage: p.integral()
1/3*x^3 + x
```

```
>>>  from  sage.all  import  *
>>> R  = PolynomialRing(QQ,"x")
>>> x  = R.gen()
>>> p  = x**Integer(2)  + Integer(1)
>>> p.derivative()
2*x
>>> p.integral()
1/3*x^3 + x
```

此时你也可以输入 view(p.derivative()) 或 view(p.integral()) 以打开一个新窗口，并由 LaTeX 对输出进行排版。

1.1.1 临界点

你可以找到分段定义函数的临界点：

```
sage: x  = PolynomialRing(RationalField(),  'x').gen()
sage: f1  = x^0
sage: f2  =  1-x
sage: f3  =  2*x
sage: f4  =  10*x-x^2
sage: f  = piecewise([((0,1),f1), ((1,2),f2), ((2,3),f3), ((3,10),f4)])
sage: f.critical_points()
[5.0]
```

```
>>>  from  sage.all  import  *
>>> x  = PolynomialRing(RationalField(),  'x').gen()
>>> f1  = x**Integer(0)
>>> f2  = Integer(1)-x
>>> f3  = Integer(2)*x
>>> f4  = Integer(10)*x-x**Integer(2)
>>> f  = piecewise([((Integer(0),Integer(1)),f1), ((Integer(1),Integer(2)),f2), ((Integer(2),
↪Integer(3)),f3), ((Integer(3),Integer(10)),f4)])
```

(续下页)

(接上页)

```
>>> f.critical_points()
[5.0]
```

1.1.2 幂级数

Sage 提供了几种构建和处理幂级数的方法。

为了从函数表达式中获取泰勒级数，请在表达式上使用.taylor() 方法:

```
sage: var('f0 k x')
(f0, k, x)
sage: g  = f0/sinh(k*x)^4
sage: g.taylor(x,  0,  3)
-62/945*f0*k^2*x^2 + 11/45*f0 - 2/3*f0/(k^2*x^2) + f0/(k^4*x^4)
```

```
>>>  from  sage.all  import  *
>>> var('f0 k x')
(f0, k, x)
>>> g  = f0/sinh(k*x)**Integer(4)
>>> g.taylor(x, Integer(0), Integer(3))
-62/945*f0*k^2*x^2 + 11/45*f0 - 2/3*f0/(k^2*x^2) + f0/(k^4*x^4)
```

可以通过.series() 方法获得函数的形式幂级数展开:

```
sage: (1/(2-cos(x))).series(x,7)
1 + (-1/2)*x^2 + 7/24*x^4 + (-121/720)*x^6 + Order(x^7)
```

```
>>>  from  sage.all  import  *
>>> (Integer(1)/(Integer(2)-cos(x))).series(x,Integer(7))
1 + (-1/2)*x^2 + 7/24*x^4 + (-121/720)*x^6 + Order(x^7)
```

然而，目前对此类序列的某些操作很难执行。有两种替代方案：要么使用 Sage 的 Maxima 子系统以获取完整符号功能:

```
sage: f  = log(sin(x)/x)
sage: f.taylor(x,  0,  10)
-1/467775*x^10 - 1/37800*x^8 - 1/2835*x^6 - 1/180*x^4 - 1/6*x^2
sage: maxima(f).powerseries(x,0)._sage_()
sum(2^(2*i... - 1)*(-1)^i...*x^(2*i...)*bern(2*i...)/(i...*factorial(2*i...)), i..., 1,␣
↪+Infinity)
```

```
>>>  from  sage.all  import  *
>>> f  = log(sin(x)/x)
>>> f.taylor(x, Integer(0), Integer(10))
-1/467775*x^10 - 1/37800*x^8 - 1/2835*x^6 - 1/180*x^4 - 1/6*x^2
>>> maxima(f).powerseries(x,Integer(0))._sage_()
sum(2^(2*i... - 1)*(-1)^i...*x^(2*i...)*bern(2*i...)/(i...*factorial(2*i...)), i..., 1,␣
↪+Infinity)
```

要么你可以使用形式幂级数环进行快速计算。这些环缺乏符号函数:

```
sage: R.<w>  = QQ[[]]
sage: ps  = w  +  17/2*w^2  +  15/4*w^4  + O(w^6); ps
w + 17/2*w^2 + 15/4*w^4 + O(w^6)
sage: ps.exp()
```

(续下页)

(接上页)

```
1 + w + 9*w^2 + 26/3*w^3 + 265/6*w^4 + 413/10*w^5 + O(w^6)
sage: (1+ps).log()
w + 8*w^2 - 49/6*w^3 - 193/8*w^4 + 301/5*w^5 + O(w^6)
sage: (ps^1000).coefficients()
[1, 8500, 36088875, 102047312625, 1729600092867375/8]
```

```
>>> from sage.all import *
>>> R = QQ[['w']]; (w,) = R._first_ngens(1)
>>> ps = w + Integer(17)/Integer(2)*w**Integer(2) + Integer(15)/Integer(4)*w**Integer(4) +␣
↪O(w**Integer(6)); ps
w + 17/2*w^2 + 15/4*w^4 + O(w^6)
>>> ps.exp()
1 + w + 9*w^2 + 26/3*w^3 + 265/6*w^4 + 413/10*w^5 + O(w^6)
>>> (Integer(1)+ps).log()
w + 8*w^2 - 49/6*w^3 - 193/8*w^4 + 301/5*w^5 + O(w^6)
>>> (ps**Integer(1000)).coefficients()
[1, 8500, 36088875, 102047312625, 1729600092867375/8]
```

1.2 积分

下面的黎曼和与梯形法积分讨论了数值积分。

Sage 可以自行对一些简单函数进行积分：

```
sage: f = x^3
sage: f.integral(x)
1/4*x^4
sage: integral(x^3,x)
1/4*x^4
sage: f = x*sin(x^2)
sage: integral(f,x)
-1/2*cos(x^2)
```

```
>>> from sage.all import *
>>> f = x**Integer(3)
>>> f.integral(x)
1/4*x^4
>>> integral(x**Integer(3),x)
1/4*x^4
>>> f = x*sin(x**Integer(2))
>>> integral(f,x)
-1/2*cos(x^2)
```

Sage 还可以计算涉及极限的符号定积分。

```
sage: var('x, k, w')
(x, k, w)
sage: f = x^3 * e^(k*x) * sin(w*x)
sage: f.integrate(x)
((24*k^3*w*e^(k*x) - 24*k*w^3*e^(k*x) - (k^6*w*e^(k*x) + 3*k^4*w^3*e^(k*x) + 3*k^2*w^5*e^(k*x)␣
↪+ w^7*e^(k*x))*x^3 + 6*(k^5*w*e^(k*x) + 2*k^3*w^3*e^(k*x) + k*w^5*e^(k*x))*x^2 - 6*(3*k^4*w*e^
↪(k*x) + 2*k^2*w^3*e^(k*x) - w^5*e^(k*x))*x)*cos(w*x) - (6*k^4*e^(k*x) - 36*k^2*w^2*e^(k*x) +␣
↪6*w^4*e^(k*x) - (k^7*e^(k*x) + 3*k^5*w^2*e^(k*x) + 3*k^3*w^4*e^(k*x) + k*w^6*e^(k*x))*x^3 +␣
↪3*(k^6*e^(k*x) + k^4*w^2*e^(k*x) - k^2*w^4*e^(k*x) - w^6*e^(k*x))*x^2 - 6*(k^5*e^(k*x) - 2*k^
```

(续下页)

(接上页)

```
↪3*w^2*e^(k*x) - 3*k*w^4*e^(k*x))*x)*sin(w*x))/(k^8 + 4*k^6*w^2 + 6*k^4*w^4 + 4*k^2*w^6 + w^8)
sage: integrate(1/x^2, x,  1, infinity)
1
```

```
>>>  from  sage.all  import  *
>>> var('x, k, w')
(x, k, w)
>>> f  = x**Integer(3)  * e**(k*x)  * sin(w*x)
>>> f.integrate(x)
((24*k^3*w*e^(k*x) - 24*k*w^3*e^(k*x) - (k^6*w*e^(k*x) + 3*k^4*w^3*e^(k*x) + 3*k^2*w^5*e^(k*x)␣
↪+ w^7*e^(k*x))*x^3 + 6*(k^5*w*e^(k*x) + 2*k^3*w^3*e^(k*x) + k*w^5*e^(k*x))*x^2 - 6*(3*k^4*w*e^
↪(k*x) + 2*k^2*w^3*e^(k*x) - w^5*e^(k*x))*x)*cos(w*x) - (6*k^4*e^(k*x) - 36*k^2*w^2*e^(k*x) +␣
↪6*w^4*e^(k*x) - (k^7*e^(k*x) + 3*k^5*w^2*e^(k*x) + 3*k^3*w^4*e^(k*x) + k*w^6*e^(k*x))*x^3 +␣
↪3*(k^6*e^(k*x) + k^4*w^2*e^(k*x) - k^2*w^4*e^(k*x) - w^6*e^(k*x))*x^2 - 6*(k^5*e^(k*x) - 2*k^
↪3*w^2*e^(k*x) - 3*k*w^4*e^(k*x))*x)*sin(w*x))/(k^8 + 4*k^6*w^2 + 6*k^4*w^4 + 4*k^2*w^6 + w^8)
>>> integrate(Integer(1)/x**Integer(2), x, Integer(1), infinity)
1
```

1.2.1 卷积

你可以计算任意分段函数与另一个函数的卷积（在定义域之外，它们被假定为零）。以下是 $f, f*f$ 和 $f*f*f$ 的定义，其中 $f(x) = 1, 0 < x < 1$:

```
sage: x  = PolynomialRing(QQ,  'x').gen()
sage: f  = piecewise([((0,1),1*x^0)])
sage: g  = f.convolution(f)
sage: h  = f.convolution(g)
sage: set_verbose(-1)
sage: P  = f.plot(); Q  = g.plot(rgbcolor=(1,1,0)); R  = h.plot(rgbcolor=(0,1,1))
```

```
>>>  from  sage.all  import  *
>>> x  = PolynomialRing(QQ,  'x').gen()
>>> f  = piecewise([((Integer(0),Integer(1)),Integer(1)*x**Integer(0))])
>>> g  = f.convolution(f)
>>> h  = f.convolution(g)
>>> set_verbose(-Integer(1))
>>> P  = f.plot(); Q  = g.plot(rgbcolor=(Integer(1),Integer(1),Integer(0))); R  = h.
↪plot(rgbcolor=(Integer(0),Integer(1),Integer(1)))
```

要查看此内容，请输入 `show(P+Q+R)`。

1.2.2 黎曼和与梯形法积分

关于 $\int_a^b f(x)\,dx$ 的数值近似，其中 f 是分段函数，可以：

- 计算（用于绘图目的）根据梯形法则定义的分段线性函数，基于将其分割为 N 个子区间进行数值积分；
- 梯形法则给出的近似值；
- 计算（用于绘图目的）根据黎曼和（左端点、右端点或中点）定义的分段常数函数，基于将其分割为 N 个子区间进行数值积分；
- 黎曼和近似值给出的近似值。

```
sage: f1(x)  = x^2
sage: f2(x)  =  5-x^2
```

(续下页)

(接上页)

```
sage: f  = piecewise([[[0,1], f1], [RealSet.open_closed(1,2), f2]])
sage: t  = f.trapezoid(2); t
piecewise(x|-->1/2*x on (0, 1/2), x|-->3/2*x - 1/2 on (1/2, 1), x|-->7/2*x - 5/2 on (1, 3/2),␣
↪x|-->-7/2*x + 8 on (3/2, 2); x)
sage: t.integral()
piecewise(x|-->1/4*x^2 on (0, 1/2), x|-->3/4*x^2 - 1/2*x + 1/8 on (1/2, 1), x|-->7/4*x^2 - 5/
↪2*x + 9/8 on (1, 3/2), x|-->-7/4*x^2 + 8*x - 27/4 on (3/2, 2); x)
sage: t.integral(definite=True)
9/4
```

```
>>> from sage.all import *
>>> __tmp__=var("x"); f1 = symbolic_expression(x**Integer(2)).function(x)
>>> __tmp__=var("x"); f2 = symbolic_expression(Integer(5)-x**Integer(2)).function(x)
>>> f  = piecewise([[[Integer(0),Integer(1)], f1], [RealSet.open_closed(Integer(1),Integer(2)),␣
↪f2]])
>>> t  = f.trapezoid(Integer(2)); t
piecewise(x|-->1/2*x on (0, 1/2), x|-->3/2*x - 1/2 on (1/2, 1), x|-->7/2*x - 5/2 on (1, 3/2),␣
↪x|-->-7/2*x + 8 on (3/2, 2); x)
>>> t.integral()
piecewise(x|-->1/4*x^2 on (0, 1/2), x|-->3/4*x^2 - 1/2*x + 1/8 on (1/2, 1), x|-->7/4*x^2 - 5/
↪2*x + 9/8 on (1, 3/2), x|-->-7/4*x^2 + 8*x - 27/4 on (3/2, 2); x)
>>> t.integral(definite=True)
9/4
```

1.2.3 拉普拉斯变换

如果你有一个分段定义的多项式函数，那么有一个“原生”命令用于计算拉普拉斯变换。这将调用 Maxima，但值得注意的是，Maxima 无法（使用最后几个示例中的直接接口）处理这种类型的计算。

```
sage: var('x s')
(x, s)
sage: f1(x)  =  1
sage: f2(x)  =  1-x
sage: f  = piecewise([((0,1),f1), ((1,2),f2)])
sage: f.laplace(x, s)
-e^(-s)/s + (s + 1)*e^(-2*s)/s^2 + 1/s - e^(-s)/s^2
```

```
>>> from sage.all import *
>>> var('x s')
(x, s)
>>> __tmp__=var("x"); f1 = symbolic_expression(Integer(1)).function(x)
>>> __tmp__=var("x"); f2 = symbolic_expression(Integer(1)-x).function(x)
>>> f  = piecewise([((Integer(0),Integer(1)),f1), ((Integer(1),Integer(2)),f2)])
>>> f.laplace(x, s)
-e^(-s)/s + (s + 1)*e^(-2*s)/s^2 + 1/s - e^(-s)/s^2
```

对于其他“合理”的函数，可以使用 Maxima 接口计算拉普拉斯变换：

```
sage: var('k, s, t')
(k, s, t)
sage: f  =  1/exp(k*t)
sage: f.laplace(t,s)
1/(k + s)
```

```
>>> from sage.all import *
>>> var('k, s, t')
(k, s, t)
>>> f = Integer(1)/exp(k*t)
>>> f.laplace(t,s)
1/(k + s)
```

上面是计算拉普拉斯变换的一种方法

```
sage: var('s, t')
(s, t)
sage: f = t^5*exp(t)*sin(t)
sage: L = laplace(f, t, s); L
240*(3*s^5 - 15*s^4 + 20*s^3 - 12*s + 4)/(s^12 - 12*s^11 + 72*s^10 - 280*s^9 + 780*s^8 - 1632*s^
↪7 + 2624*s^6 - 3264*s^5 + 3120*s^4 - 2240*s^3 + 1152*s^2 - 384*s + 64)
```

```
>>> from sage.all import *
>>> var('s, t')
(s, t)
>>> f = t**Integer(5)*exp(t)*sin(t)
>>> L = laplace(f, t, s); L
240*(3*s^5 - 15*s^4 + 20*s^3 - 12*s + 4)/(s^12 - 12*s^11 + 72*s^10 - 280*s^9 + 780*s^8 - 1632*s^
↪7 + 2624*s^6 - 3264*s^5 + 3120*s^4 - 2240*s^3 + 1152*s^2 - 384*s + 64)
```

上面是另一种方法。

1.3 常微分方程

使用 Sage 接口与 Maxima 可以符号化地求解常微分方程。参见

```
sage: desolvers?
```

```
>>> from sage.all import *
>>> desolvers?
```

获取可用命令。可以使用 Sage 接口与 Octave（一个实验性包）或 GSL（Gnu 科学库）中的例程来数值求解常微分方程。

例如，通过 Sage 的 Maxima 接口符号化地求解常微分方程（请勿输入....:）：

```
sage: y=function('y')(x); desolve(diff(y,x,2) + 3*x == y, dvar = y, ics = [1,1,1])
3*x - 2*e^(x - 1)
sage: desolve(diff(y,x,2) + 3*x == y, dvar = y)
_K2*e^(-x) + _K1*e^x + 3*x
sage: desolve(diff(y,x) + 3*x == y, dvar = y)
(3*(x + 1)*e^(-x) + _C)*e^x
sage: desolve(diff(y,x) + 3*x == y, dvar = y, ics = [1,1]).expand()
3*x - 5*e^(x - 1) + 3

sage: f=function('f')(x); desolve_laplace(diff(f,x,2) == 2*diff(f,x)-f, dvar = f, ics = [0,
↪1,2])
x*e^x + e^x

sage: desolve_laplace(diff(f,x,2) == 2*diff(f,x)-f, dvar = f)
-x*e^x*f(0) + x*e^x*D[0](f)(0) + e^x*f(0)
```

```
>>> from sage.all import *
>>> y=function('y')(x); desolve(diff(y,x,Integer(2)) + Integer(3)*x == y, dvar = y, ics =␣
↪[Integer(1),Integer(1),Integer(1)])
3*x - 2*e^(x - 1)
>>> desolve(diff(y,x,Integer(2)) + Integer(3)*x == y, dvar = y)
_K2*e^(-x) + _K1*e^x + 3*x
>>> desolve(diff(y,x) + Integer(3)*x == y, dvar = y)
(3*(x + 1)*e^(-x) + _C)*e^x
>>> desolve(diff(y,x) + Integer(3)*x == y, dvar = y, ics = [Integer(1),Integer(1)]).expand()
3*x - 5*e^(x - 1) + 3

>>> f=function('f')(x); desolve_laplace(diff(f,x,Integer(2)) == Integer(2)*diff(f,x)-f, dvar␣
↪ = f, ics = [Integer(0),Integer(1),Integer(2)])
x*e^x + e^x

>>> desolve_laplace(diff(f,x,Integer(2)) == Integer(2)*diff(f,x)-f, dvar = f)
-x*e^x*f(0) + x*e^x*D[0](f)(0) + e^x*f(0)
```

如果你已经安装了 Octave 和 gnuplot，

```
sage: octave.de_system_plot(['x+y','x-y'], [1,-1], [0,2])  # optional - octave
```

```
>>> from sage.all import *
>>> octave.de_system_plot(['x+y','x-y'], [Integer(1),-Integer(1)], [Integer(0),Integer(2)])  #␣
↪optional - octave
```

将在同一个图中绘制常微分方程组的两个图像 $(t, x(t)), (t, y(t))$ （t-轴为横轴）

$$x' = x + y, x(0) = 1; y' = x - y, y(0) = -1,$$

对于 $0 \le t \le 2$。使用 desolve_system_rk4 也可以获得相同的结果:

```
sage: x, y, t = var('x y t')
sage: P=desolve_system_rk4([x+y, x-y], [x,y], ics=[0,1,-1], ivar=t, end_points=2)
sage: p1 = list_plot([[i,j] for i,j,k in P], plotjoined=True)
sage: p2 = list_plot([[i,k] for i,j,k in P], plotjoined=True, color='red')
sage: p1+p2
Graphics object consisting of 2 graphics primitives
```

```
>>> from sage.all import *
>>> x, y, t = var('x y t')
>>> P=desolve_system_rk4([x+y, x-y], [x,y], ics=[Integer(0),Integer(1),-Integer(1)], ivar=t,␣
↪end_points=Integer(2))
>>> p1 = list_plot([[i,j] for i,j,k in P], plotjoined=True)
>>> p2 = list_plot([[i,k] for i,j,k in P], plotjoined=True, color='red')
>>> p1+p2
Graphics object consisting of 2 graphics primitives
```

该方程组也可以通过使用命令 desolve_system 来求解。

```
sage: t=var('t'); x=function('x',t); y=function('y',t)
sage: des = [diff(x,t) == x+y, diff(y,t) == x-y]
sage: desolve_system(des, [x,y], ics = [0, 1, -1])
[x(t) == cosh(sqrt(2)*t), y(t) == sqrt(2)*sinh(sqrt(2)*t) - cosh(sqrt(2)*t)]
```

```
>>> from sage.all import *
>>> t=var('t'); x=function('x',t); y=function('y',t)
>>> des = [diff(x,t) == x+y, diff(y,t) == x-y]
>>> desolve_system(des, [x,y], ics = [Integer(0), Integer(1), -Integer(1)])
[x(t) == cosh(sqrt(2)*t), y(t) == sqrt(2)*sinh(sqrt(2)*t) - cosh(sqrt(2)*t)]
```

此命令的输出不是一对函数。

最后，可以使用幂级数求解线性微分方程：

```
sage: R.<t> = PowerSeriesRing(QQ, default_prec=10)
sage: a = 2 - 3*t + 4*t^2 + O(t^10)
sage: b = 3 - 4*t^2 + O(t^7)
sage: f = a.solve_linear_de(prec=5, b=b, f0=3/5)
sage: f
3/5 + 21/5*t + 33/10*t^2 - 38/15*t^3 + 11/24*t^4 + O(t^5)
sage: f.derivative() - a*f - b
O(t^4)
```

```
>>> from sage.all import *
>>> R = PowerSeriesRing(QQ, default_prec=Integer(10), names=('t',)); (t,) = R._first_ngens(1)
>>> a = Integer(2) - Integer(3)*t + Integer(4)*t**Integer(2) + O(t**Integer(10))
>>> b = Integer(3) - Integer(4)*t**Integer(2) + O(t**Integer(7))
>>> f = a.solve_linear_de(prec=Integer(5), b=b, f0=Integer(3)/Integer(5))
>>> f
3/5 + 21/5*t + 33/10*t^2 - 38/15*t^3 + 11/24*t^4 + O(t^5)
>>> f.derivative() - a*f - b
O(t^4)
```

1.4 周期函数的傅里叶级数

设 f 是一个周期为 $2L$ 的实周期函数。f 的傅里叶级数是

$$S(x) = \frac{a_0}{2} + \sum_{n=1}^{\infty} \left[a_n \cos\left(\frac{n\pi x}{L}\right) + b_n \sin\left(\frac{n\pi x}{L}\right) \right]$$

其中

$$a_n = \frac{1}{L} \int_{-L}^{L} f(x) \cos\left(\frac{n\pi x}{L}\right) dx,$$

并且

$$b_n = \frac{1}{L} \int_{-L}^{L} f(x) \sin\left(\frac{n\pi x}{L}\right) dx,$$

傅里叶系数 a_n 和 b_n 是通过声明 f 在一个周期内分段定义的函数并调用方法 `fourier_series_cosine_coefficient` 和 `fourier_series_sine_coefficient` 来计算的，而部分和是通过 `fourier_series_partial_sum` 获得的：

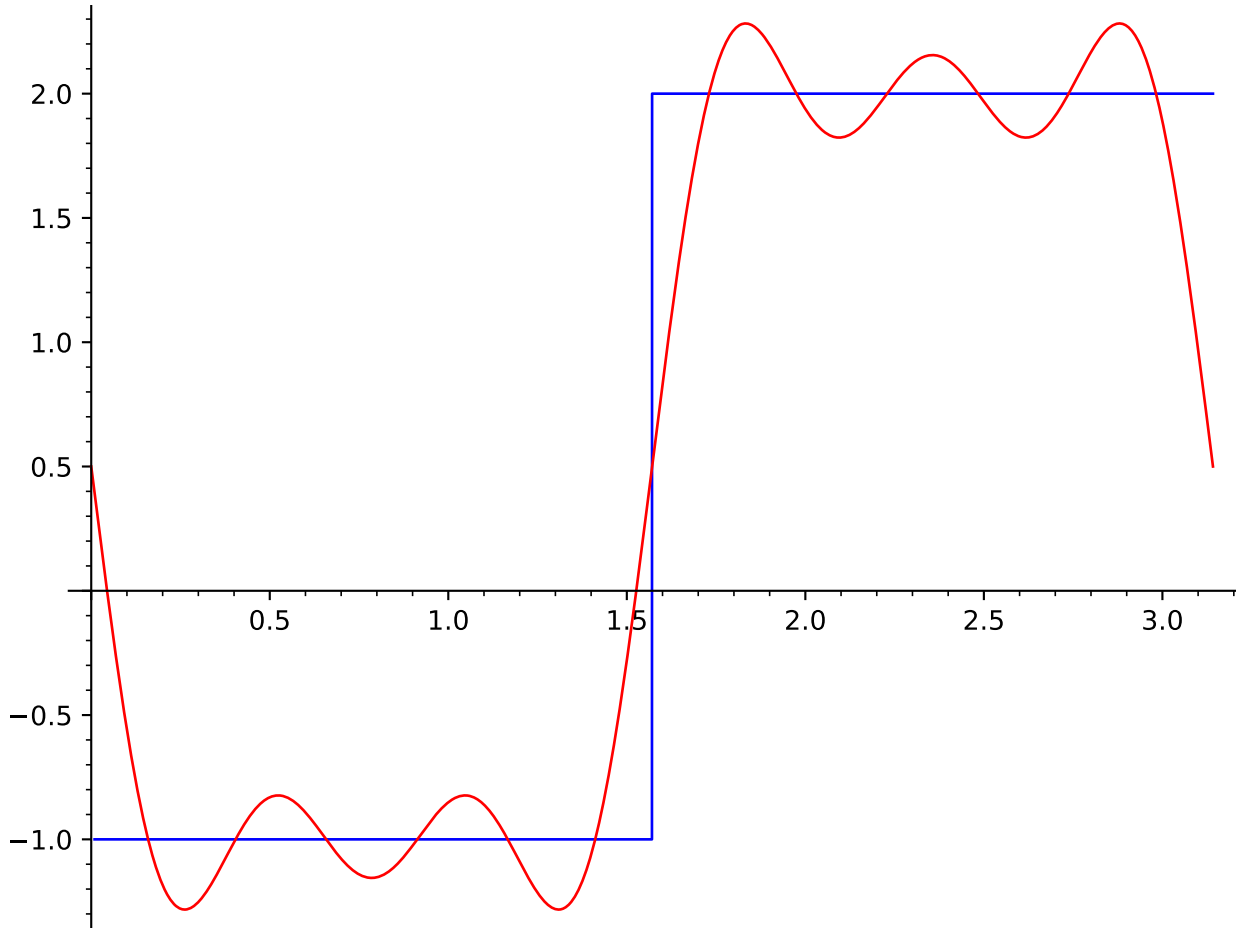
```
sage: f = piecewise([((0,pi/2), -1), ((pi/2,pi), 2)])
sage: f.fourier_series_cosine_coefficient(0)
1
sage: f.fourier_series_sine_coefficient(5)
-6/5/pi
```

(续下页)

(接上页)

```
sage: s5  = f.fourier_series_partial_sum(5); s5
-6/5*sin(10*x)/pi - 2*sin(6*x)/pi - 6*sin(2*x)/pi + 1/2
sage: plot(f, (0,pi))  + plot(s5, (x,0,pi), color='red')
Graphics object consisting of 2 graphics primitives
```

```
>>> from sage.all import *
>>> f  = piecewise([((Integer(0),pi/Integer(2)),  -Integer(1)), ((pi/Integer(2),pi),␣
↪Integer(2))])
>>> f.fourier_series_cosine_coefficient(Integer(0))
1
>>> f.fourier_series_sine_coefficient(Integer(5))
-6/5/pi
>>> s5  = f.fourier_series_partial_sum(Integer(5)); s5
-6/5*sin(10*x)/pi - 2*sin(6*x)/pi - 6*sin(2*x)/pi + 1/2
>>> plot(f, (Integer(0),pi))  + plot(s5, (x,Integer(0),pi), color='red')
Graphics object consisting of 2 graphics primitives
```



CHAPTER 2

绘图

Sage 可以使用 matplotlib, openmath, gnuplot 或 surf 进行绘图，但在标准发行版中只有 matplotlib 和 openmath 包含在内。有关 surf 的示例，请参见绘制曲面。

在 Sage 中有许多不同的绘图方式。你可以使用 gnuplot 绘制函数（二维或三维）或者点集（仅支持二维），你可以使用 Maxima 绘制微分方程的解（调用 gnuplot 或 openmath），或者使用 Singular 的接口与绘图包 surf 进行绘图（不包含在 Sage 中）。gnuplot 没有隐式绘图命令，所以如果你想使用隐式绘图绘制曲线或曲面，如在章节‘ch:AG，代数几何’中所述，最好使用 Singular 的 surf 接口。

2.1 二维函数绘图

默认的绘图方法使用优秀的 `matplotlib` 包。

要查看绘图，请输入 `P.save("<path>/myplot.png")`，然后在诸如 gimp 之类的图形查看器中打开它。

如下例所示，可以绘制分段函数：

```
sage: f1  =  1
sage: f2  =  1-x
sage: f3  = exp(x)
sage: f4  = sin(2*x)
sage: f  = piecewise([((0,1),f1), ((1,2),f2), ((2,3),f3), ((3,10),f4)])
sage: f.plot(x,0,10)
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> f1  = Integer(1)
>>> f2  = Integer(1)-x
>>> f3  = exp(x)
>>> f4  = sin(Integer(2)*x)
>>> f  = piecewise([((Integer(0),Integer(1)),f1), ((Integer(1),Integer(2)),f2), ((Integer(2),
↪Integer(3)),f3), ((Integer(3),Integer(10)),f4)])
>>> f.plot(x,Integer(0),Integer(10))
Graphics object consisting of 1 graphics primitive
```

还可以生成其他函数图像：

雅可比椭圆函数 $\mathrm{sn}(x, 2)$, $-3 < x < 3$ 的红色图像（请勿输入....:）：

```
sage: L  = [(i/100.0, maxima.eval('jacobi_sn (%s/100.0,2.0)'%i))
....:       for i  in  range(-300,300)]
sage: show(line(L, rgbcolor=(3/4,1/4,1/8)))
```

```
>>>  from  sage.all  import  *
>>> L  = [(i/RealNumber('100.0'), maxima.eval('jacobi_sn (%s/100.0,2.0)'%i))
...       for i  in  range(-Integer(300),Integer(300))]
>>> show(line(L, rgbcolor=(Integer(3)/Integer(4),Integer(1)/Integer(4),Integer(1)/Integer(8))))
```

J-贝塞尔函数 $J_2(x)$, $0 < x < 10$ 的红色图像：

```
sage: L  = [(i/10.0, maxima.eval('bessel_j (2,%s/10.0)'%i)) for i in range(100)]
sage: show(line(L, rgbcolor=(3/4,1/4,5/8)))
```

```
>>>  from  sage.all  import  *
>>> L  = [(i/RealNumber('10.0'), maxima.eval('bessel_j (2,%s/10.0)'%i)) for i in␣
↪range(Integer(100))]
>>> show(line(L, rgbcolor=(Integer(3)/Integer(4),Integer(1)/Integer(4),Integer(5)/Integer(8))))
```

黎曼 ζ 函数 $\zeta(1/2 + it)$, $0 < t < 30$ 的紫色图像：

```
sage: I  = CDF.0
sage: show(line([zeta(1/2  + k*I/6)  for k  in  range(180)], rgbcolor=(3/4,1/2,5/8)))
```

```
>>>  from  sage.all  import  *
>>> I  = CDF.gen(0)
>>> show(line([zeta(Integer(1)/Integer(2)  + k*I/Integer(6))  for k  in  range(Integer(180))],␣
↪rgbcolor=(Integer(3)/Integer(4),Integer(1)/Integer(2),Integer(5)/Integer(8))))
```

2.2 绘制曲线

在 Sage 中绘制曲线，可以使用 Singular 和 surf (http://surf.sourceforge.net/ 也可以作为实验包使用) 或使用 matplotlib (Sage 自带)。

2.2.1 matplotlib

这里有几个例子。要查看绘图，请输入 `p.save("<path>/my_plot.png")` (其中<path> 是你拥有写权限且希望保存绘图的目录路径) 并在查看器中查看 (例如 GIMP)。

蓝色 Nicomedes 共轭线：

```
sage: L  = [[1+5*cos(pi/2+pi*i/100), tan(pi/2+pi*i/100)*
....:      (1+5*cos(pi/2+pi*i/100))]  for i  in  range(1,100)]
sage: line(L, rgbcolor=(1/4,1/8,3/4))
Graphics object consisting of 1 graphics primitive
```

```
>>>  from  sage.all  import  *
>>> L  = [[Integer(1)+Integer(5)*cos(pi/Integer(2)+pi*i/Integer(100)), tan(pi/Integer(2)+pi*i/
↪Integer(100))*
...       (Integer(1)+Integer(5)*cos(pi/Integer(2)+pi*i/Integer(100)))]  for i  in␣
↪ range(Integer(1),Integer(100))]
```

(续下页)

(接上页)

```
>>> line(L, rgbcolor=(Integer(1)/Integer(4),Integer(1)/Integer(8),Integer(3)/Integer(4)))
Graphics object consisting of 1 graphics primitive
```

蓝色（三叶）内旋轮线 (hypotrochoid)：

```
sage: n = 4; h = 3; b = 2
sage: L = [[n*cos(pi*i/100)+h*cos((n/b)*pi*i/100),
....:     n*sin(pi*i/100)-h*sin((n/b)*pi*i/100)] for i in range(200)]
sage: line(L, rgbcolor=(1/4,1/4,3/4))
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> n = Integer(4); h = Integer(3); b = Integer(2)
>>> L = [[n*cos(pi*i/Integer(100))+h*cos((n/b)*pi*i/Integer(100)),
...     n*sin(pi*i/Integer(100))-h*sin((n/b)*pi*i/Integer(100))] for i in␣
↪ range(Integer(200))]
>>> line(L, rgbcolor=(Integer(1)/Integer(4),Integer(1)/Integer(4),Integer(3)/Integer(4)))
Graphics object consisting of 1 graphics primitive
```

蓝色（四叶）内旋轮线 (hypotrochoid)：

```
sage: n = 6; h = 5; b = 2
sage: L = [[n*cos(pi*i/100)+h*cos((n/b)*pi*i/100),
....:     n*sin(pi*i/100)-h*sin((n/b)*pi*i/100)] for i in range(200)]
sage: line(L, rgbcolor=(1/4,1/4,3/4))
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> n = Integer(6); h = Integer(5); b = Integer(2)
>>> L = [[n*cos(pi*i/Integer(100))+h*cos((n/b)*pi*i/Integer(100)),
...     n*sin(pi*i/Integer(100))-h*sin((n/b)*pi*i/Integer(100))] for i in␣
↪ range(Integer(200))]
>>> line(L, rgbcolor=(Integer(1)/Integer(4),Integer(1)/Integer(4),Integer(3)/Integer(4)))
Graphics object consisting of 1 graphics primitive
```

红色帕斯卡蜗线 (limaçon of Pascal)：

```
sage: L = [[sin(pi*i/100)+sin(pi*i/50),-(1+cos(pi*i/100)+cos(pi*i/50))]
....:     for i in range(-100,101)]
sage: line(L, rgbcolor=(1,1/4,1/2))
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> L = [[sin(pi*i/Integer(100))+sin(pi*i/Integer(50)),-(Integer(1)+cos(pi*i/
↪Integer(100))+cos(pi*i/Integer(50)))]
...     for i in range(-Integer(100),Integer(101))]
>>> line(L, rgbcolor=(Integer(1),Integer(1)/Integer(4),Integer(1)/Integer(2)))
Graphics object consisting of 1 graphics primitive
```

浅绿色麦克劳林三等分角线 (trisectrix of Maclaurin)：

```
sage: L = [[2*(1-4*cos(-pi/2+pi*i/100)^2),10*tan(-pi/2+pi*i/100)*
....:     (1-4*cos(-pi/2+pi*i/100)^2)] for i in range(1,100)]
sage: line(L, rgbcolor=(1/4,1,1/8))
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> L = [[Integer(2)*(Integer(1)-Integer(4)*cos(-pi/Integer(2)+pi*i/Integer(100))**Integer(2)),
↪Integer(10)*tan(-pi/Integer(2)+pi*i/Integer(100))*
...       (Integer(1)-Integer(4)*cos(-pi/Integer(2)+pi*i/Integer(100))**Integer(2))] for i in␣
↪ range(Integer(1),Integer(100))]
>>> line(L, rgbcolor=(Integer(1)/Integer(4),Integer(1),Integer(1)/Integer(8)))
Graphics object consisting of 1 graphics primitive
```

浅绿色伯努利双纽线 (lemniscate of Bernoulli，我们忽略 i=100，因为这样会导致除 0 错误):

```
sage: v = [(1/cos(-pi/2+pi*i/100), tan(-pi/2+pi*i/100)) for i in range(1,200) if i!=100 ]
sage: L = [(a/(a^2+b^2), b/(a^2+b^2)) for a,b in v]
sage: line(L, rgbcolor=(1/4,3/4,1/8))
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> v = [(Integer(1)/cos(-pi/Integer(2)+pi*i/Integer(100)), tan(-pi/Integer(2)+pi*i/
↪Integer(100))) for i in range(Integer(1),Integer(200)) if i!=Integer(100) ]
>>> L = [(a/(a**Integer(2)+b**Integer(2)), b/(a**Integer(2)+b**Integer(2))) for a,b in v]
>>> line(L, rgbcolor=(Integer(1)/Integer(4),Integer(3)/Integer(4),Integer(1)/Integer(8)))
Graphics object consisting of 1 graphics primitive
```

2.2.2 surf

由于 `surf` 仅在类 UNIX 操作系统上可用（并且不包含在 Sage 中），因此在 Sage 中使用以下命令绘图仅在类 UNIX 操作系统上可用。顺带一提，surf 已包含在几个流行的 Linux 发行版中。

```
sage: s = singular.eval
sage: s('LIB "surf.lib";')
...
sage: s("ring rr0 = 0,(x1,x2),dp;")
''
sage: s("ideal I = x1^3 - x2^2;")
''
sage: s("plot(I);")
...
```

```
>>> from sage.all import *
>>> s = singular.eval
>>> s('LIB "surf.lib";')
...
>>> s("ring rr0 = 0,(x1,x2),dp;")
''
>>> s("ideal I = x1^3 - x2^2;")
''
>>> s("plot(I);")
...
```

在 surf 窗口处于活动状态时按下 `q` 退出 surf 并返回 Sage。

你可以将此绘图保存为 surf 脚本。在弹出的 surf 窗口中，只需选择 `file`, `save as` 等等。（输入 `q` 或选择 `file`, `quit` 关闭窗口。）

生成的图省略，但鼓励读者亲自尝试。

```
s = singular
s('LIB "surf.lib";')
s("ring rr0 = 0,(x1,x2),dp;")
s("ideal I = x13 - x22;")
s("plot(I);")
s('ring rr1 = 0,(x,y,z),dp;')
s('ideal I(1) = 2x2-1/2x3 +1-y+1;')
s('plot(I(1));')
s('poly logo = ((x+3)3 + 2\*(x+3)2 - y2)\*(x3 -y2)\*((x-3)3-2\*(x-3)2-y2);')
s('plot(logo);') Steiner surface
s('ideal J(2) = x2\*y2+x2\*z2+y2\*z2-17\*x\*y\*z;')
s('plot(J(2));')
```

2.3 openmath

Openmath 是由 W. Schelter 编写的 TCL/Tk GUI 绘图程序。

以下命令绘制函数 $\cos(2x) + 2e^{-x}$

```
sage: maxima.plot2d('cos(2*x) + 2*exp(-x)','[x,0,1]',    # not tested (pops up a window)
....:       '[plot_format,openmath]')
```

```
>>> from sage.all import *
>>> maxima.plot2d('cos(2*x) + 2*exp(-x)','[x,0,1]',    # not tested (pops up a window)
...       '[plot_format,openmath]')
```

(Mac OS X 用户请注意：这些 `openmath` 命令是在一个 xterm shell 会话中运行的，而不是使用标准的 Mac 终端应用程序。)

```
sage: maxima.eval('load("plotdf");')
'"...share/maxima.../share/dynamics/plotdf.lisp"'
sage: maxima.eval('plotdf(x+y,[trajectory_at,2,-0.1]);  ')    # not tested
```

```
>>> from sage.all import *
>>> maxima.eval('load("plotdf");')
'"...share/maxima.../share/dynamics/plotdf.lisp"'
>>> maxima.eval('plotdf(x+y,[trajectory_at,2,-0.1]);  ')    # not tested
```

这里绘制了一个方向场（plotdf Maxima 包也由 W. Schelter 编写。）

多个函数的二维绘图：

```
sage: maxima.plot2d('[x,x^2,x^3]','[x,-1,1]','[plot_format,openmath]')    # not tested
```

```
>>> from sage.all import *
>>> maxima.plot2d('[x,x^2,x^3]','[x,-1,1]','[plot_format,openmath]')    # not tested
```

Openmath 还可以绘制 $z = f(x, y)$ 形式的三维曲面，其中 x 和 y 在矩形范围内。例如，这里有一个可以用鼠标移动的“实时”三维图：

```
sage: maxima.plot3d ("sin(x^2 + y^2)",  "[x, -3, 3]",  "[y, -3, 3]",    # not tested
....:       '[plot_format, openmath]')
```

```
>>> from sage.all import *
>>> maxima.plot3d ("sin(x^2 + y^2)", "[x, -3, 3]", "[y, -3, 3]", # not tested
...       '[plot_format, openmath]')
```

通过适当旋转，你可以查看轮廓图。

2.4 Tachyon 3D 绘图

光线追踪包 Tachyon 随 Sage 一起分发。3D 图看起来非常漂亮，但通常需要更多的配置。下面是一个参数空间曲线的示例：

```
sage: f = lambda t: (t,t^2,t^3)
sage: t = Tachyon(camera_center=(5,0,4))
sage: t.texture('t')
sage: t.light((-20,-20,40), 0.2, (1,1,1))
sage: t.parametric_plot(f,-5,5,'t',min_depth=6)
```

```
>>> from sage.all import *
>>> f = lambda t: (t,t**Integer(2),t**Integer(3))
>>> t = Tachyon(camera_center=(Integer(5),Integer(0),Integer(4)))
>>> t.texture('t')
>>> t.light((-Integer(20),-Integer(20),Integer(40)), RealNumber('0.2'), (Integer(1),Integer(1),
↪Integer(1)))
>>> t.parametric_plot(f,-Integer(5),Integer(5),'t',min_depth=Integer(6))
```

输入 `t.show()` 查看该曲线。

参考手册中有其他示例。

2.5 gnuplot

必须安装 `gnuplot` 才能运行这些命令。

首先，这是绘制函数的方法：{plot!a function}

```
sage: maxima.plot2d('sin(x)','[x,-5,5]')
sage: opts = '[gnuplot_term, ps], [gnuplot_out_file, "sin-plot.eps"]'
sage: maxima.plot2d('sin(x)','[x,-5,5]',opts)
sage: opts = '[gnuplot_term, ps], [gnuplot_out_file, "/tmp/sin-plot.eps"]'
sage: maxima.plot2d('sin(x)','[x,-5,5]',opts)
```

```
>>> from sage.all import *
>>> maxima.plot2d('sin(x)','[x,-5,5]')
>>> opts = '[gnuplot_term, ps], [gnuplot_out_file, "sin-plot.eps"]'
>>> maxima.plot2d('sin(x)','[x,-5,5]',opts)
>>> opts = '[gnuplot_term, ps], [gnuplot_out_file, "/tmp/sin-plot.eps"]'
>>> maxima.plot2d('sin(x)','[x,-5,5]',opts)
```

默认情况下 eps 文件保存在当前目录，但你可以指定保存路径。

下面是在平面上绘制参数曲线的示例：

```
sage: maxima.plot2d_parametric(["sin(t)","cos(t)"], "t",[-3.1,3.1])
sage: opts = '[gnuplot_preamble, "set nokey"], [gnuplot_term, ps],
```

(续下页)

(接上页)

```
....:     [gnuplot_out_file,  "circle-plot.eps"]'
sage: maxima.plot2d_parametric(["sin(t)","cos(t)"],  "t", [-3.1,3.1], options=opts)
```

```
>>>  from  sage.all  import  *
>>> maxima.plot2d_parametric(["sin(t)","cos(t)"],  "t",[-RealNumber('3.1'),RealNumber('3.1')])
>>> opts  =  '[gnuplot_preamble,  "set nokey"], [gnuplot_term, ps],
...     [gnuplot_out_file,  "circle-plot.eps"]'
>>> maxima.plot2d_parametric(["sin(t)","cos(t)"],  "t", [-RealNumber('3.1'),RealNumber('3.1')],␣
↪options=opts)
```

下面是在三维空间中绘制参数曲面的示例: {plot!a parametric surface}

```
sage: maxima.plot3d_parametric(["v*sin(u)","v*cos(u)","v"], ["u","v"],
....:     [-3.2,3.2],[0,3])
sage: opts  =  '[gnuplot_term, ps], [gnuplot_out_file,  "sin-cos-plot.eps"]'
sage: maxima.plot3d_parametric(["v*sin(u)","v*cos(u)","v"], ["u","v"],
....:     [-3.2,3.2],[0,3],opts)
```

```
>>>  from  sage.all  import  *
>>> maxima.plot3d_parametric(["v*sin(u)","v*cos(u)","v"], ["u","v"],
...     [-RealNumber('3.2'),RealNumber('3.2')],[Integer(0),Integer(3)])
>>> opts  =  '[gnuplot_term, ps], [gnuplot_out_file,  "sin-cos-plot.eps"]'
>>> maxima.plot3d_parametric(["v*sin(u)","v*cos(u)","v"], ["u","v"],
...     [-RealNumber('3.2'),RealNumber('3.2')],[Integer(0),Integer(3)],opts)
```

为了演示如何传递 gnuplot 选项，下面是关于绘制黎曼 ζ 函数 $\zeta(s)$ 上一系列点的示例（使用 Pari 计算，但使用 Maxima 和 Gnuplot 绘图）：{plot!points} {Riemann zeta function}

```
sage: zeta_ptsx  = [ (pari(1/2  + i*I/10).zeta().real()).precision(1)
....:      for i  in  range(70,150)]
sage: zeta_ptsy  = [ (pari(1/2  + i*I/10).zeta().imag()).precision(1)
....:      for i  in  range(70,150)]
sage: maxima.plot_list(zeta_ptsx, zeta_ptsy)   # optional -- pops up a window.
sage: opts='[gnuplot_preamble,  "set nokey"], [gnuplot_term, ps],
....:     [gnuplot_out_file,  "zeta.eps"]'
sage: maxima.plot_list(zeta_ptsx, zeta_ptsy, opts)  # optional -- pops up a window.
```

```
>>>  from  sage.all  import  *
>>> zeta_ptsx  = [ (pari(Integer(1)/Integer(2)  + i*I/Integer(10)).zeta().real()).
↪precision(Integer(1))
...      for i  in  range(Integer(70),Integer(150))]
>>> zeta_ptsy  = [ (pari(Integer(1)/Integer(2)  + i*I/Integer(10)).zeta().imag()).
↪precision(Integer(1))
...      for i  in  range(Integer(70),Integer(150))]
>>> maxima.plot_list(zeta_ptsx, zeta_ptsy)   # optional -- pops up a window.
>>> opts='[gnuplot_preamble,  "set nokey"], [gnuplot_term, ps],
...     [gnuplot_out_file,  "zeta.eps"]'
>>> maxima.plot_list(zeta_ptsx, zeta_ptsy, opts)  # optional -- pops up a window.
```

2.6 绘制曲面

绘制曲面与绘制曲线并无太大区别，尽管语法稍有不同。特别是，你需要加载 `surf`。{plot!surface using surf}

```
sage: singular.eval('ring rr1 = 0,(x,y,z),dp;')
''
sage: singular.eval('ideal I(1) = 2x2-1/2x3 +1-y+1;')
''
sage: singular.eval('plot(I(1));')
...
```

```
>>> from sage.all import *
>>> singular.eval('ring rr1 = 0,(x,y,z),dp;')
''
>>> singular.eval('ideal I(1) = 2x2-1/2x3 +1-y+1;')
''
>>> singular.eval('plot(I(1));')
...
```

CHAPTER 3

群

3.1 置换群

置换群是某一对称群 S_n 的子群。Sage 有一个 Python 类 `PermutationGroup`，因此你可以直接使用此类群:

```
sage: G  = PermutationGroup(['(1,2,3)(4,5)'])
sage: G
Permutation Group with generators [(1,2,3)(4,5)]
sage: g  = G.gens()[0]; g
(1,2,3)(4,5)
sage: g*g
(1,3,2)
sage: G  = PermutationGroup(['(1,2,3)'])
sage: g  = G.gens()[0]; g
(1,2,3)
sage: g.order()
3
```

```
>>> from sage.all import *
>>> G  = PermutationGroup(['(1,2,3)(4,5)'])
>>> G
Permutation Group with generators [(1,2,3)(4,5)]
>>> g  = G.gens()[Integer(0)]; g
(1,2,3)(4,5)
>>> g*g
(1,3,2)
>>> G  = PermutationGroup(['(1,2,3)'])
>>> g  = G.gens()[Integer(0)]; g
(1,2,3)
>>> g.order()
3
```

对于魔方群（S_{48} 的置换子群，其中魔方的非中心面以某种固定方式标记为 $1, 2, ..., 48$），你可以按如下方式使用 GAP-Sage 接口。

```
sage: cube = "cubegp := Group(
( 1, 3, 8, 6)( 2, 5, 7, 4)( 9,33,25,17)(10,34,26,18)(11,35,27,19),
( 9,11,16,14)(10,13,15,12)( 1,17,41,40)( 4,20,44,37)( 6,22,46,35),
(17,19,24,22)(18,21,23,20)( 6,25,43,16)( 7,28,42,13)( 8,30,41,11),
(25,27,32,30)(26,29,31,28)( 3,38,43,19)( 5,36,45,21)( 8,33,48,24),
(33,35,40,38)(34,37,39,36)( 3, 9,46,32)( 2,12,47,29)( 1,14,48,27),
(41,43,48,46)(42,45,47,44)(14,22,30,38)(15,23,31,39)(16,24,32,40) )"
sage: gap(cube)
'permutation group with 6 generators'
sage: gap("Size(cubegp)")
43252003274489856000'
```

```
>>> from sage.all import *
>>> cube = "cubegp := Group(
( 1, 3, 8, 6)( 2, 5, 7, 4)( 9,33,25,17)(10,34,26,18)(11,35,27,19),
( 9,11,16,14)(10,13,15,12)( 1,17,41,40)( 4,20,44,37)( 6,22,46,35),
(17,19,24,22)(18,21,23,20)( 6,25,43,16)( 7,28,42,13)( 8,30,41,11),
(25,27,32,30)(26,29,31,28)( 3,38,43,19)( 5,36,45,21)( 8,33,48,24),
(33,35,40,38)(34,37,39,36)( 3, 9,46,32)( 2,12,47,29)( 1,14,48,27),
(41,43,48,46)(42,45,47,44)(14,22,30,38)(15,23,31,39)(16,24,32,40) )"
>>> gap(cube)
'permutation group with 6 generators'
>>> gap("Size(cubegp)")
43252003274489856000'
```

你还可以选择另一种方式来实现：

- 创建一个包含以下内容的文件 `cubegroup.py`:

```
cube = "cubegp := Group(
( 1, 3, 8, 6)( 2, 5, 7, 4)( 9,33,25,17)(10,34,26,18)(11,35,27,19),
( 9,11,16,14)(10,13,15,12)( 1,17,41,40)( 4,20,44,37)( 6,22,46,35),
(17,19,24,22)(18,21,23,20)( 6,25,43,16)( 7,28,42,13)( 8,30,41,11),
(25,27,32,30)(26,29,31,28)( 3,38,43,19)( 5,36,45,21)( 8,33,48,24),
(33,35,40,38)(34,37,39,36)( 3, 9,46,32)( 2,12,47,29)( 1,14,48,27),
(41,43,48,46)(42,45,47,44)(14,22,30,38)(15,23,31,39)(16,24,32,40) )"
```

然后将该文件放置在 Sage 目录的 `$SAGE_ROOT/local/lib/python2.4/site-packages/sage` 子目录中。最后，读取（即 `import`）该文件到 Sage 中：

```
sage: import sage.cubegroup
sage: sage.cubegroup.cube
'cubegp := Group(( 1, 3, 8, 6)( 2, 5, 7, 4)( 9,33,25,17)(10,34,26,18)
(11,35,27,19),( 9,11,16,14)(10,13,15,12)( 1,17,41,40)( 4,20,44,37)
( 6,22,46,35),(17,19,24,22)(18,21,23,20)( 6,25,43,16)( 7,28,42,13)
( 8,30,41,11),(25,27,32,30)(26,29,31,28)( 3,38,43,19)( 5,36,45,21)
( 8,33,48,24),(33,35,40,38)(34,37,39,36)( 3, 9,46,32)( 2,12,47,29)
( 1,14,48,27),(41,43,48,46)(42,45,47,44)(14,22,30,38)(15,23,31,39)
(16,24,32,40) )'
sage: gap(sage.cubegroup.cube)
'permutation group with 6 generators'
sage: gap("Size(cubegp)")
'43252003274489856000'
```

```
>>> from sage.all import *
>>> import sage.cubegroup
>>> sage.cubegroup.cube
```

(续下页)

(接上页)

```
'cubegp := Group(( 1, 3, 8, 6)( 2, 5, 7, 4)( 9,33,25,17)(10,34,26,18)
(11,35,27,19),( 9,11,16,14)(10,13,15,12)( 1,17,41,40)( 4,20,44,37)
( 6,22,46,35),(17,19,24,22)(18,21,23,20)( 6,25,43,16)( 7,28,42,13)
( 8,30,41,11),(25,27,32,30)(26,29,31,28)( 3,38,43,19)( 5,36,45,21)
( 8,33,48,24),(33,35,40,38)(34,37,39,36)( 3, 9,46,32)( 2,12,47,29)
( 1,14,48,27),(41,43,48,46)(42,45,47,44)(14,22,30,38)(15,23,31,39)
(16,24,32,40) )'
>>> gap(sage.cubegroup.cube)
'permutation group with 6 generators'
>>> gap("Size(cubegp)")
'43252003274489856000'
```

（在 Sage 输出中，会使用换行符来代替上面的回车符。）

- 使用 CubeGroup 类:

```
sage: rubik  = CubeGroup()
sage: rubik
The Rubik's cube group with generators R,L,F,B,U,D in SymmetricGroup(48).
sage: rubik.order()
43252003274489856000
```

```
>>>  from  sage.all  import  *
>>> rubik  = CubeGroup()
>>> rubik
The Rubik's cube group with generators R,L,F,B,U,D in SymmetricGroup(48).
>>> rubik.order()
43252003274489856000
```

(1) Sage 实现了经典群（如 $GU(3, \mathbf{F}_5)$）和具有用户定义生成器的有限域上的矩阵群。

(2) Sage 还实现了有限和无限（但有限生成）阿贝尔群。

3.2 共轭类

你可以“原生地”计算有限群的共轭类:

```
sage: G  = PermutationGroup(['(1,2,3)',  '(1,2)(3,4)',  '(1,7)'])
sage: CG  = G.conjugacy_classes_representatives()
sage: gamma  = CG[2]
sage: CG; gamma
[(), (4,7), (3,4,7), (2,3)(4,7), (2,3,4,7), (1,2)(3,4,7), (1,2,3,4,7)]
(3,4,7)
```

```
>>>  from  sage.all  import  *
>>> G  = PermutationGroup(['(1,2,3)',  '(1,2)(3,4)',  '(1,7)'])
>>> CG  = G.conjugacy_classes_representatives()
>>> gamma  = CG[Integer(2)]
>>> CG; gamma
[(), (4,7), (3,4,7), (2,3)(4,7), (2,3,4,7), (1,2)(3,4,7), (1,2,3,4,7)]
(3,4,7)
```

你可以使用 Sage-GAP 接口完成这一任务:

```
sage: libgap.eval("G := Group((1,2)(3,4),(1,2,3))")
Group([ (1,2)(3,4), (1,2,3) ])
sage: libgap.eval("CG := ConjugacyClasses(G)")
[ ()^G, (2,3,4)^G, (2,4,3)^G, (1,2)(3,4)^G ]
sage: libgap.eval("gamma := CG[3]")
(2,4,3)^G
sage: libgap.eval("g := Representative(gamma)")
(2,4,3)
```

```
>>> from sage.all import *
>>> libgap.eval("G := Group((1,2)(3,4),(1,2,3))")
Group([ (1,2)(3,4), (1,2,3) ])
>>> libgap.eval("CG := ConjugacyClasses(G)")
[ ()^G, (2,3,4)^G, (2,4,3)^G, (1,2)(3,4)^G ]
>>> libgap.eval("gamma := CG[3]")
(2,4,3)^G
>>> libgap.eval("g := Representative(gamma)")
(2,4,3)
```

或者，这里有另一种（更符合 Python 风格的）方法来进行该计算:

```
sage: G = libgap.eval("Group([(1,2,3), (1,2)(3,4), (1,7)])")
sage: CG = G.ConjugacyClasses()
sage: gamma = CG[2]
sage: g = gamma.Representative()
sage: CG; gamma; g
[ ()^G, (4,7)^G, (3,4,7)^G, (2,3)(4,7)^G, (2,3,4,7)^G, (1,2)(3,4,7)^G, (1,2,3,4,7)^G ]
(3,4,7)^G
(3,4,7)
```

```
>>> from sage.all import *
>>> G = libgap.eval("Group([(1,2,3), (1,2)(3,4), (1,7)])")
>>> CG = G.ConjugacyClasses()
>>> gamma = CG[Integer(2)]
>>> g = gamma.Representative()
>>> CG; gamma; g
[ ()^G, (4,7)^G, (3,4,7)^G, (2,3)(4,7)^G, (2,3,4,7)^G, (1,2)(3,4,7)^G, (1,2,3,4,7)^G ]
(3,4,7)^G
(3,4,7)
```

3.3 正规子群

如果想要找到置换群 G （从共轭角度）的所有正规子群，可以使用 Sage 的 GAP 接口:

```
sage: G = AlternatingGroup( 5 )
sage: libgap(G).NormalSubgroups()
[ Alt( [ 1 .. 5 ] ), Group(()) ]
```

```
>>> from sage.all import *
>>> G = AlternatingGroup( Integer(5) )
>>> libgap(G).NormalSubgroups()
[ Alt( [ 1 .. 5 ] ), Group(()) ]
```

或者

```
sage: G  = libgap.AlternatingGroup(  5 )
sage: G.NormalSubgroups()
[ Alt( [ 1 .. 5 ] ), Group(()) ]
```

```
>>>  from  sage.all  import  *
>>> G  = libgap.AlternatingGroup( Integer(5) )
>>> G.NormalSubgroups()
[ Alt( [ 1 .. 5 ] ), Group(()) ]
```

这里有另一种更直接使用 GAP 的方法:

```
sage: libgap.eval("G := AlternatingGroup( 5 )")
Alt( [ 1 .. 5 ] )
sage: libgap.eval("normal := NormalSubgroups( G )")
[ Alt( [ 1 .. 5 ] ), Group(()) ]
sage: G  = libgap.eval("DihedralGroup( 10 )")
sage: G.NormalSubgroups().SortedList()
[ Group([  ]), Group([ f2 ]), <pc group of size 10 with 2 generators> ]
sage: libgap.eval("G := SymmetricGroup( 4 )")
Sym( [ 1 .. 4 ] )
sage: libgap.eval("normal := NormalSubgroups( G );")
[ Sym( [ 1 .. 4 ] ), Alt( [ 1 .. 4 ] ), Group([ (1,4)(2,3),  ... ]),
      Group(()) ]
```

```
>>>  from  sage.all  import  *
>>> libgap.eval("G := AlternatingGroup( 5 )")
Alt( [ 1 .. 5 ] )
>>> libgap.eval("normal := NormalSubgroups( G )")
[ Alt( [ 1 .. 5 ] ), Group(()) ]
>>> G  = libgap.eval("DihedralGroup( 10 )")
>>> G.NormalSubgroups().SortedList()
[ Group([  ]), Group([ f2 ]), <pc group of size 10 with 2 generators> ]
>>> libgap.eval("G := SymmetricGroup( 4 )")
Sym( [ 1 .. 4 ] )
>>> libgap.eval("normal := NormalSubgroups( G );")
[ Sym( [ 1 .. 4 ] ), Alt( [ 1 .. 4 ] ), Group([ (1,4)(2,3),  ... ]),
      Group(()) ]
```

3.4 中心

如何在 Sage 中计算群的中心?

虽然 Sage 调用 GAP 来计算群的中心，但 `center` 是“封装”过的方法（即 Sage 有一个类 PermutationGroup 关联”center” 方法），因此用户不需要使用 `libgap` 命令。这里有一个例子:

```
sage: G  = PermutationGroup(['(1,2,3)(4,5)',  '(3,4)'])
sage: G.center()
Subgroup generated by [()] of (Permutation Group with generators [(3,4), (1,2,3)(4,5)])
```

```
>>>  from  sage.all  import  *
>>> G  = PermutationGroup(['(1,2,3)(4,5)',  '(3,4)'])
>>> G.center()
Subgroup generated by [()] of (Permutation Group with generators [(3,4), (1,2,3)(4,5)])
```

类似的语法也适用于矩阵群:

```
sage: G  = SL(2, GF(5) )
sage: G.center()
Subgroup with 1 generators (
[4 0]
[0 4]
) of Special Linear Group of degree 2 over Finite Field of size 5
sage: G  = PSL(2,  5 )
sage: G.center()
Subgroup generated by [()] of (The projective special linear group of degree 2 over Finite␣
↪Field of size 5)
```

```
>>>  from  sage.all  import  *
>>> G  = SL(Integer(2), GF(Integer(5)) )
>>> G.center()
Subgroup with 1 generators (
[4 0]
[0 4]
) of Special Linear Group of degree 2 over Finite Field of size 5
>>> G  = PSL(Integer(2), Integer(5) )
>>> G.center()
Subgroup generated by [()] of (The projective special linear group of degree 2 over Finite␣
↪Field of size 5)
```

备注

在 GAP 中 `center` 有两种拼写方式，但在 Sage 中不行。

3.5 群 id 数据库

函数 `group_id` 使用了 E. A. O'Brien、B. Eick 和 H. U. Besche 的小群库，它是 GAP 的一部分。

```
sage: G  = PermutationGroup(['(1,2,3)(4,5)',  '(3,4)'])
sage: G.order()
120
sage: G.group_id()
[120, 34]
```

```
>>>  from  sage.all  import  *
>>> G  = PermutationGroup(['(1,2,3)(4,5)',  '(3,4)'])
>>> G.order()
120
>>> G.group_id()
[120, 34]
```

另一个使用小型群数据库的例子：`group_id`

```
sage: gap_console()
┌─────┐   GAP 4.10.0 of 01-Nov-2018
│ GAP │   https://www.gap-system.org
└─────┘   Architecture: x86_64-pc-linux-gnu-default64
Configuration:  gmp 6.0.0, readline
Loading the library and packages ...
Packages:   GAPDoc 1.6.2, PrimGrp 3.3.2, SmallGrp 1.3, TransGrp 2.0.4
```

(续下页)

(接上页)

```
Try '??help' for help. See also '?copyright', '?cite' and '?authors'
gap> G:=Group((4,6,5)(7,8,9),(1,7,2,4,6,9,5,3));
Group([ (4,6,5)(7,8,9), (1,7,2,4,6,9,5,3) ])
gap> StructureDescription(G);
"(C3 x C3) : GL(2,3)"
```

```
>>> from sage.all import *
>>> gap_console()
┌─────────┐   GAP 4.10.0 of 01-Nov-2018
│  GAP    │   https://www.gap-system.org
└─────────┘   Architecture: x86_64-pc-linux-gnu-default64
Configuration:  gmp 6.0.0, readline
Loading the library and packages ...
Packages:   GAPDoc 1.6.2, PrimGrp 3.3.2, SmallGrp 1.3, TransGrp 2.0.4
Try '??help' for help. See also '?copyright', '?cite' and '?authors'
gap> G:=Group((4,6,5)(7,8,9),(1,7,2,4,6,9,5,3));
Group([ (4,6,5)(7,8,9), (1,7,2,4,6,9,5,3) ])
gap> StructureDescription(G);
"(C3 x C3) : GL(2,3)"
```

3.6 小于 32 阶的群的构建指令

作者：

- Davis Shurbert

每个小于 32 阶的群都在 Sage 中实现为置换群。这些群的构建都非常简单。我们首先展示如何构建直积和半直积，然后给出构建这些小群所需的命令。

设 G1, G2, ..., Gn 是已经在 Sage 中初始化的置换群。可以使用以下命令取它们的直积（当然，这里省略号只是作为符号使用，实际上必须显式输入所求乘积中的每个因子）。

```
sage: G = direct_product_permgroups([G1, G2, ..., Gn])
```

```
>>> from sage.all import *
>>> G = direct_product_permgroups([G1, G2, Ellipsis, Gn])
```

半直积运算可以被视为直积运算的推广。给定两个群 H 和 K，它们的半直积 $H \rtimes_\phi K$（其中 $\phi : H \to Aut(K)$ 是一个同态）是一个群，其基础集合是 H 和 K 的笛卡尔积，但具有以下运算：

$$(h_1, k_1)(h_2, k_2) = (h_1 h_2, k_1^{\phi(h_2)} k_2).$$

输出不是运算定义中明确描述的群，而是一个同构的置换群。在下面的例程中，假设 H 和 K 已经在 Sage 中定义且初始化。此外，phi 是一个包含两个子列表的列表，通过给出 H 的生成器集合的像来定义底层同态。对于下表中的每个半直积群，我们将展示如何构建 phi，然后假设你已经阅读此段落并理解如何从那里开始。

```
sage: G = H.semidirect_product(K, phi)
```

```
>>> from sage.all import *
>>> G = H.semidirect_product(K, phi)
```

为了避免不必要的重复，我们现在将给出创建 n 阶循环群 C_n 的命令和 n 个字母的二面体群 D_n 的命令。我们还会为每个命令展示一个例子以确保读者理解这些命令，然后不再重复。

```
sage: G  = CyclicPermutationGroup(n)

sage: G  = DihedralGroup(n)
```

```
>>> from sage.all import *
>>> G  = CyclicPermutationGroup(n)

>>> G  = DihedralGroup(n)
```

请注意，直积运算中将使用指数表示法。例如 $C_2{}^2 = C_2 \times C_2$。该表格是在 AD Thomas 和 GV Wood 的 *Group Tables* (1980, Shiva Publishing) 的帮助下制作的。

阶	群描述	命令	GAP ID

1	平凡群	`sage: G = SymmetricGroup(1)`
		`>>> from sage.all import *`
		`>>> G = SymmetricGroup(Integer(1))`
			[1,1]

2	C_2	`sage: G = SymmetricGroup(2)`
		`>>> from sage.all import *`
		`>>> G = SymmetricGroup(Integer(2))`
			[2,1]

3	C_3	`sage: G = CyclicPermutationGroup(3)`
		`>>> from sage.all import *`
		`>>> G = CyclicPermutationGroup(Integer(3))`
			[3,1]

4	C_4		[4,1]

4	$C_2 \times C_2$	`sage: G = KleinFourGroup()`
		`>>> from sage.all import *`
		`>>> G = KleinFourGroup()`
			[4,2]

5	C_5		[5,1]
6	C_6		[6,2]

续下页

表 1 – 接上页

阶	群描述	命令	GAP ID

6	S_3 （三字母对称群）	`sage: G = SymmetricGroup(3)`
		`>>> from sage.all import *`
		`>>> G = SymmetricGroup(Integer(3))`
			[6,1]

7	C_7		[7,1]
8	C_8		[8,1]
8	$C_4 \times C_2$		[8,2]
8	$C_2 \times C_2 \times C_2$		[8,5]

8	D_4	`sage: G = DihedralGroup(4)`
		`>>> from sage.all import *`
		`>>> G = DihedralGroup(Integer(4))`
			[8,3]

8	四元群 (Q)	`sage: G = QuaternionGroup()`
		`>>> from sage.all import *`
		`>>> G = QuaternionGroup()`
			[8,4]

9	C_9		[9,1]
9	$C_3 \times C_3$		[9,2]
10	C_{10}		[10,2]
10	D_5		[10,1]
11	C_{11}		[11,1]
12	C_{12}		[12,2]
12	$C_6 \times C_2$		[12,5]
12	D_6		[12,4]

12	A_4 （四字母交错群）	`sage: G = AlternatingGroup(4)`
		`>>> from sage.all import *`
		`>>> G = AlternatingGroup(Integer(4))`
			[12,3]

12	Q_6 （12 阶双环群）	`sage: G = DiCyclicGroup(3)`
		`>>> from sage.all import *`
		`>>> G = DiCyclicGroup(Integer(3))`
			[12,1]

续下页

表 1 – 接上页

阶	群描述	命令	GAP ID
13	C_{13}		[13,1]
14	C_{14}		[14,2]
14	D_7		[14,1]
15	C_{15}		[15,1]
16	C_{16}		[16,1]
16	$C_8 \times C_2$		[16,5]
16	$C_4 \times C_4$		[16,2]
16	$C_4 \times C_2 \times C_2$		[16,10]
16	${C_2}^4$		[16,14]
16	$D_4 \times C_2$		[16,11]
16	$Q \times C_2$		[16,12]
16	D_8		[16,7]

16	Q_8 （16 阶双环群）	`sage: G = DiCyclicGroup(4)`
		`>>> from sage.all import *`
		`>>> G = DiCyclicGroup(Integer(4))`
			[16,9]

16	2^4 阶半二面体群	`sage: G = SemidihedralGroup(4)`
		`>>> from sage.all import *`
		`>>> G = SemidihedralGroup(Integer(4))`
			[16,8]

16	2^4 阶分裂亚循环群	`sage: G = SplitMetacyclicGroup(2,4)`
		`>>> from sage.all import *`
		`>>> G = SplitMetacyclicGroup(Integer(2), Integer(4))`
			[16,6]

续下页

表 1 – 接上页

阶	群描述	命令	GAP ID

16	$(C_4 \times C_2) \rtimes_\phi C_2$	`sage: C2 = SymmetricGroup(2); C4 = CyclicPermutationGroup(4)`
		`sage: A = direct_product_permgroups([C2,C4])`
		`sage: alpha = PermutationGroupMorphism(A,A,[A.gens()[0],A.gens()[0]^2*A.gens()[1]])`
		`sage: phi = [[(1,2)],[alpha]]`
		
		`>>> from sage.all import *`
		`>>> C2 = SymmetricGroup(Integer(2)); C4 = CyclicPermutationGroup(Integer(4))`
		`>>> A = direct_product_permgroups([C2,C4])`
		`>>> alpha = PermutationGroupMorphism(A,A,[A.gens()[Integer(0)],A.gens()[Integer(0)]**Integer(2)*A.gens()[Integer(1)]])`
		`>>> phi = [[(Integer(1),Integer(2))],[alpha]]`
			[16,13]

续下页

表 1 – 接上页

阶	群描述	命令	GAP ID

16	$(C_4 \times C_2) \rtimes_\phi C_2$	sage: C2 = SymmetricGroup(2); C4 = CyclicPermutationGroup(4)
		sage: A = direct_product_permgroups([C2,C4])
		sage: alpha = PermutationGroupMorphism(A,A,[A.gens()[0]^3*A.gens()[1],A.gens()[1]])
		sage: phi = [[(1,2)],[alpha]]
		
		>>> from sage.all import *
		>>> C2 = SymmetricGroup(Integer(2)); C4 = CyclicPermutationGroup(Integer(4))
		>>> A = direct_product_permgroups([C2,C4])
		>>> alpha = PermutationGroupMorphism(A,A,[A.gens()[Integer(0)]**Integer(3)*A.gens()[Integer(1)],A.gens()[Integer(1)]])
		>>> phi = [[(Integer(1),Integer(2))],[alpha]]
			[16,3]

续下页

表 1 – 接上页

阶	群描述	命令	GAP ID

16	$C_4 \rtimes_\phi C_4$	sage: C4 = CyclicPermutationGroup(4)
		sage: alpha = PermutationGroupMorphism(C4,C4,[C4.gen().inverse()])
		sage: phi = [[(1,2,3,4)],[alpha]]
		
		>>> from sage.all import *
		>>> C4 = CyclicPermutationGroup(Integer(4))
		>>> alpha = PermutationGroupMorphism(C4,C4,[C4.gen().inverse()])
		>>> phi = [[(Integer(1),Integer(2),Integer(3),Integer(4))],[alpha]]
			[16,4]

17	C_{17}		[17,1]
18	C_{18}		[18,2]
18	$C_6 \times C_3$		[18,5]
18	D_9		[18,1]
18	$S_3 \times C_3$		[18,3]

18	$Dih(C_3 \times C_3)$	sage: G = GeneralDihedralGroup([3,3])
		
		>>> from sage.all import *
		>>> G = GeneralDihedralGroup([Integer(3),Integer(3)])
			[18,4]

19	C_{19}		[19,1]
20	C_{20}		[20,2]
20	$C_{10} \times C_2$		[20,5]
20	D_{10}		[20,4]
20	Q_{10} （20 阶双环群）	[20,1]	

续下页

表 1 – 接上页

阶	群描述	命令	GAP ID

20	$Hol(C_5)$	sage: C5 = CyclicPermutationGroup(5)
		sage: G = C5.holomorph()
		
		>>> from sage.all import *
		>>> C5 = CyclicPermutationGroup(Integer(5))
		>>> G = C5.holomorph()
			[20,3]

21	C_{21}		[21,2]

21	$C_7 \rtimes_\phi C_3$	sage: C7 = CyclicPermutationGroup(7)
		sage: alpha = PermutationGroupMorphism(C7,C7,[C7.gen()**4])
		sage: phi = [[(1,2,3)],[alpha]]
		
		>>> from sage.all import *
		>>> C7 = CyclicPermutationGroup(Integer(7))
		>>> alpha = PermutationGroupMorphism(C7,C7,[C7.gen()**Integer(4)])
		>>> phi = [[(Integer(1),Integer(2),Integer(3))],[alpha]]
			[21,1]

22	C_{22}		[22,2]
22	D_{11}		[22,1]
23	C_{23}		[23,1]
24	C_{24}		[24,2]
24	D_{12}		[24,6]
24	Q_{12} （24 阶双环群）	[24,4]	
24	$C_{12} \times C_2$		[24,9]
24	$C_6 \times C_2 \times C_2$		[24,15]

24	S_4 （四字母对称群）	sage: G = SymmetricGroup(4)
		
		>>> from sage.all import *
		>>> G = SymmetricGroup(Integer(4))
			[24,12]

24	$S_3 \times C_4$		[24,5]

续下页

表 1 – 接上页

阶	群描述	命令	GAP ID
24	$S_3 \times C_2 \times C_2$		[24,14]
24	$D_4 \times C_3$		[24,10]
24	$Q \times C_3$		[24,11]
24	$A_4 \times C_2$		[24,13]
24	$Q_6 \times C_2$		[24,7]

24	$Q \rtimes_\phi C_3$	`sage: Q = QuaternionGroup()`
		`sage: alpha = PermutationGroupMorphism(Q,Q,[Q.gens()[0]*Q.gens()[1],Q.gens()[0].inverse()])`
		`sage: phi = [[(1,2,3)],[alpha]]`
		`>>> from sage.all import *`
		`>>> Q = QuaternionGroup()`
		`>>> alpha = PermutationGroupMorphism(Q,Q,[Q.gens()[Integer(0)]*Q.gens()[Integer(1)],Q.gens()[Integer(0)].inverse()])`
		`>>> phi = [[(Integer(1),Integer(2),Integer(3))],[alpha]]`
			[24,3]

续下页

表 1 – 接上页

阶	群描述	命令	GAP ID

24	$C_3 \rtimes_\phi C_8$	sage: C3 = CyclicPermutationGroup(3)
		sage: alpha = PermutationGroupMorphism(C3,C3,[C3.gen().inverse()])
		sage: phi = [[(1,2,3,4,5,6,7,8)],[alpha]]
		>>> from sage.all import *
		>>> C3 = CyclicPermutationGroup(Integer(3))
		>>> alpha = PermutationGroupMorphism(C3,C3,[C3.gen().inverse()])
		>>> phi = [[(Integer(1),Integer(2),Integer(3),Integer(4),Integer(5),Integer(6),Integer(7),Integer(8))],[alpha]]
			[24,1]

续下页

表 1 – 接上页

阶	群描述	命令	GAP ID

24	$C_3 \rtimes_\phi D_4$	sage: C3 = CyclicPermutationGroup(3)
		sage: alpha1 = PermutationGroupMorphism(C3,C3,[C3.gen().inverse()])
		sage: alpha2 = PermutationGroupMorphism(C3,C3,[C3.gen()])
		sage: phi = [[(1,2,3,4),(1,3)],[alpha1,alpha2]]
		>>> from sage.all import *
		>>> C3 = CyclicPermutationGroup(Integer(3))
		>>> alpha1 = PermutationGroupMorphism(C3,C3,[C3.gen().inverse()])
		>>> alpha2 = PermutationGroupMorphism(C3,C3,[C3.gen()])
		>>> phi = [[(Integer(1),Integer(2),Integer(3),Integer(4)),(Integer(1),Integer(3))],[alpha1,alpha2]]
			[24,8]

25	C_{25}		[25,1]
25	$C_5 \times C_5$		[25,2]
26	C_{26}		[26,2]
26	D_{13}		[26,1]
27	C_{27}		[27,1]
27	$C_9 \times C_3$		[27,2]
27	$C_3 \times C_3 \times C_3$		[27,5]

27	3^3 阶分裂亚循环群	sage: G = SplitMetacyclicGroup(3,3)
		>>> from sage.all import *
		>>> G = SplitMetacyclicGroup(Integer(3),Integer(3))
			[27,4]

续下页

表 1 – 接上页

阶	群描述	命令	GAP ID

27	$(C_3 \times C_3) \rtimes_\phi C_3$	sage: C3 = CyclicPermutationGroup(3)
		sage: A = direct_product_permgroups([C3,C3])
		sage: alpha = PermutationGroupMorphism(A,A,[A.gens()[0]*A.gens()[1].inverse(),A.gens()[1]])
		sage: phi = [[(1,2,3)],[alpha]]
		>>> from sage.all import *
		>>> C3 = CyclicPermutationGroup(Integer(3))
		>>> A = direct_product_permgroups([C3,C3])
		>>> alpha = PermutationGroupMorphism(A,A,[A.gens()[Integer(0)]*A.gens()[Integer(1)].inverse(),A.gens()[Integer(1)]])
		>>> phi = [[(Integer(1),Integer(2),Integer(3))],[alpha]]
			[27,3]

28	C_{28}		[28,2]
28	$C_{14} \times C_2$		[28,4]
28	D_{14}		[28,3]
28	Q_{14} （28 阶双环群）	[28,1]	
29	C_{29}		[29,1]
30	C_{30}		[30,4]
30	D_{15}		[30,3]
30	$D_5 \times C_3$		[30,2]
30	$D_3 \times C_5$		[30,1]
31	C_{31}		[31,1]

该表由 Kevin Halasz 提供。

3.7 小于等于 15 阶的有限呈示群的构建说明

Sage 能够轻松构建阶数小于等于 15 的有限呈示群。我们将首先探讨创建有限生成阿贝尔群，以及有限呈示群的直积和半直积。

所有有限生成阿贝尔群都可以使用 `groups.presentation.FGAbelian(ls)` 命令创建，其中 `ls` 是一个非负整数列表，该列表被简化为定义要返回群的不变量。例如，要构建 $C_4 \times C_2 \times C_2 \times C_2$，我们可以简单地使用:

```
sage: A  = groups.presentation.FGAbelian([4,2,2,2])
```

```
>>> from sage.all import *
>>> A  = groups.presentation.FGAbelian([Integer(4),Integer(2),Integer(2),Integer(2)])
```

无论输入的整数列表如何，对于给定群的输出都是相同的。以下示例为阶数为 30 的循环群产生相同的表示。

```
sage: A  = groups.presentation.FGAbelian([2,3,5])
sage: B  = groups.presentation.FGAbelian([30])
```

```
>>> from sage.all import *
>>> A  = groups.presentation.FGAbelian([Integer(2),Integer(3),Integer(5)])
>>> B  = groups.presentation.FGAbelian([Integer(30)])
```

如果 G 和 H 是有限呈示群，我们可以使用以下代码来创建 G 和 H 的直积，$G \times H$。

```
sage: D  = G.direct_product(H)
```

```
>>> from sage.all import *
>>> D  = G.direct_product(H)
```

假设存在从群 G 到群 H 的自同构群的同态 ϕ。通过 ϕ 将 G 与 H 的半直积定义为 G 和 H 的笛卡尔积，运算为 $(g_1, h_1)(g_2, h_2) = (g_1 g_2, \phi_{h_1}(g_2) h_2)$ 其中 $\phi_h = \phi(h)$。要在 Sage 中为两个有限呈示群构造此乘积，我们必须使用一对列表手动定义 ϕ。第一个列表由群 G 的生成器组成，而第二个列表由第一个列表中相应生成器的像组成。这些自同构同样定义为一对列表，一个列表为生成器，另一个列表为像。作为示例，我们将阶数为 16 的二面体群构造为循环群的半直积。

```
sage: C2  = groups.presentation.Cyclic(2)
sage: C8  = groups.presentation.Cyclic(8)
sage: hom  = (C2.gens(), [ ([C8([1])], [C8([-1])]) ])
sage: D  = C2.semidirect_product(C8, hom)
```

```
>>> from sage.all import *
>>> C2  = groups.presentation.Cyclic(Integer(2))
>>> C8  = groups.presentation.Cyclic(Integer(8))
>>> hom  = (C2.gens(), [ ([C8([Integer(1)])], [C8([-Integer(1)])]) ])
>>> D  = C2.semidirect_product(C8, hom)
```

下表显示了阶数小于等于 15 的群，以及如何在 Sage 中构造它们。重复命令已被省略，但通过以下示例进行了描述。

阶数为 n 的循环群可以通过单个命令创建：

```
sage: C  = groups.presentation.Cyclic(n)
```

```
>>> from sage.all import *
>>> C  = groups.presentation.Cyclic(n)
```

对于阶数为 $2n$ 的二面体群也类似：

```
sage: D  = groups.presentation.Dihedral(n)
```

```
>>> from sage.all import *
>>> D  = groups.presentation.Dihedral(n)
```

该表是根据前面 Kevin Halasz 创建的表格构造的。

阶	群描述	命令	GAP ID

1	平凡群	`sage: G = groups.presentation.Symmetric(1)`
		`>>> from sage.all import *`
		`>>> G = groups.presentation.Symmetric(Integer(1))`
			[1,1]

2	C_2	`sage: G = groups.presentation.Symmetric(2)`
		`>>> from sage.all import *`
		`>>> G = groups.presentation.Symmetric(Integer(2))`
			[2,1]

3	C_3	`sage: G = groups.presentation.Cyclic(3)`
		`>>> from sage.all import *`
		`>>> G = groups.presentation.Cyclic(Integer(3))`
			[3,1]

4	C_4		[4,1]

4	$C_2 \times C_2$	`sage: G = groups.presentation.Klein()`
		`>>> from sage.all import *`
		`>>> G = groups.presentation.Klein()`
			[4,2]

5	C_5		[5,1]
6	C_6		[6,2]

6	S_3 （三字母对称群）	`sage: G = groups.presentation.Symmetric(3)`
		`>>> from sage.all import *`
		`>>> G = groups.presentation.Symmetric(Integer(3))`
			[6,1]

7	C_7		[7,1]
8	C_8		[8,1]

8	$C_4 \times C_2$	`sage: G = groups.presentation.FGAbelian([4,2])`
		`>>> from sage.all import *`
		`>>> G = groups.presentation.FGAbelian([Integer(4)Integer(2)])`
			[8,2]

8	$C_2 \times C_2 \times C_2$	`sage: G = groups.`	[8,5]

CHAPTER 4

线性代数

4.1 向量空间

VectorSpace 命令用于创建一个向量空间类，从中可以创建一个子空间。注意，Sage 计算的基是“行最简”的。

```
sage: V  = VectorSpace(GF(2),8)
sage: S  = V.subspace([V([1,1,0,0,0,0,0,0]),V([1,0,0,0,0,1,1,0])])
sage: S.basis()
[(1, 0, 0, 0, 0, 1, 1, 0), (0, 1, 0, 0, 0, 1, 1, 0)]
sage: S.dimension()
2
```

```
>>> from sage.all import *
>>> V  = VectorSpace(GF(Integer(2)),Integer(8))
>>> S  = V.subspace([V([Integer(1),Integer(1),Integer(0),Integer(0),Integer(0),Integer(0),
↪Integer(0),Integer(0)]),V([Integer(1),Integer(0),Integer(0),Integer(0),Integer(0),Integer(1),
↪Integer(1),Integer(0)])])
>>> S.basis()
[(1, 0, 0, 0, 0, 1, 1, 0), (0, 1, 0, 0, 0, 1, 1, 0)]
>>> S.dimension()
2
```

4.2 矩阵幂

如何在 Sage 中计算矩阵幂？语法如下例所示。

```
sage: R  = IntegerModRing(51)
sage: M  = MatrixSpace(R,3,3)
sage: A  = M([1,2,3,  4,5,6,  7,8,9])
sage: A^1000*A^1007
```

(续下页)

(接上页)

```
[ 3  3  3]
[18  0 33]
[33 48 12]
sage: A^2007

[ 3  3  3]
[18  0 33]
[33 48 12]
```

```
>>> from sage.all import *
>>> R = IntegerModRing(Integer(51))
>>> M = MatrixSpace(R,Integer(3),Integer(3))
>>> A = M([Integer(1),Integer(2),Integer(3), Integer(4),Integer(5),Integer(6), Integer(7),
↪Integer(8),Integer(9)])
>>> A**Integer(1000)*A**Integer(1007)
<BLANKLINE>
[ 3  3  3]
[18  0 33]
[33 48 12]
>>> A**Integer(2007)
<BLANKLINE>
[ 3  3  3]
[18  0 33]
[33 48 12]
```

4.3 核

通过对矩阵对象应用 kernel 方法来计算核。语法如下例所示。

```
sage: M = MatrixSpace(IntegerRing(),4,2)(range(8))
sage: M.kernel()
Free module of degree 4 and rank 2 over Integer Ring
Echelon basis matrix:
[ 1  0 -3  2]
[ 0  1 -2  1]
```

```
>>> from sage.all import *
>>> M = MatrixSpace(IntegerRing(),Integer(4),Integer(2))(range(Integer(8)))
>>> M.kernel()
Free module of degree 4 and rank 2 over Integer Ring
Echelon basis matrix:
[ 1  0 -3  2]
[ 0  1 -2  1]
```

Q 上的一维核：

```
sage: A = MatrixSpace(RationalField(),3)(range(9))
sage: A.kernel()
Vector space of degree 3 and dimension 1 over Rational Field
Basis matrix:
[ 1 -2  1]
```

```
>>> from sage.all import *
>>> A = MatrixSpace(RationalField(),Integer(3))(range(Integer(9)))
```

(续下页)

(接上页)

```
>>> A.kernel()
Vector space of degree 3 and dimension 1 over Rational Field
Basis matrix:
[ 1 -2  1]
```

平凡核：

```
sage: A  = MatrixSpace(RationalField(),2)([1,2,3,4])
sage: A.kernel()
Vector space of degree 2 and dimension 0 over Rational Field
Basis matrix:
[]
sage: M  = MatrixSpace(RationalField(),0,2)(0)
sage: M
[]
sage: M.kernel()
Vector space of degree 0 and dimension 0 over Rational Field
Basis matrix:
[]
sage: M  = MatrixSpace(RationalField(),2,0)(0)
sage: M.kernel()
Vector space of degree 2 and dimension 2 over Rational Field
Basis matrix:
[1 0]
[0 1]
```

```
>>>  from  sage.all  import  *
>>> A  = MatrixSpace(RationalField(),Integer(2))([Integer(1),Integer(2),Integer(3),Integer(4)])
>>> A.kernel()
Vector space of degree 2 and dimension 0 over Rational Field
Basis matrix:
[]
>>> M  = MatrixSpace(RationalField(),Integer(0),Integer(2))(Integer(0))
>>> M
[]
>>> M.kernel()
Vector space of degree 0 and dimension 0 over Rational Field
Basis matrix:
[]
>>> M  = MatrixSpace(RationalField(),Integer(2),Integer(0))(Integer(0))
>>> M.kernel()
Vector space of degree 2 and dimension 2 over Rational Field
Basis matrix:
[1 0]
[0 1]
```

零矩阵的核：

```
sage: A  = MatrixSpace(RationalField(),2)(0)
sage: A.kernel()
Vector space of degree 2 and dimension 2 over Rational Field
Basis matrix:
[1 0]
[0 1]
```

```
>>> from sage.all import *
>>> A = MatrixSpace(RationalField(),Integer(2))(Integer(0))
>>> A.kernel()
Vector space of degree 2 and dimension 2 over Rational Field
Basis matrix:
[1 0]
[0 1]
```

非方阵的核：

```
sage: A = MatrixSpace(RationalField(),3,2)(range(6))
sage: A.kernel()
Vector space of degree 3 and dimension 1 over Rational Field
Basis matrix:
[ 1 -2  1]
```

```
>>> from sage.all import *
>>> A = MatrixSpace(RationalField(),Integer(3),Integer(2))(range(Integer(6)))
>>> A.kernel()
Vector space of degree 3 and dimension 1 over Rational Field
Basis matrix:
[ 1 -2  1]
```

分圆域上矩阵的二维核：

```
sage: K = CyclotomicField(12); a = K.gen()
sage: M = MatrixSpace(K,4,2)([1,-1, 0,-2, 0,-a^2-1, 0,a^2-1])
sage: M
[             1            -1]
[             0            -2]
[             0 -zeta12^2 - 1]
[             0  zeta12^2 - 1]
sage: M.kernel()
Vector space of degree 4 and dimension 2 over Cyclotomic Field of order 12
 and degree 4
Basis matrix:
[               0                1                0     -2*zeta12^2]
[               0                0                1 -2*zeta12^2 + 1]
```

```
>>> from sage.all import *
>>> K = CyclotomicField(Integer(12)); a = K.gen()
>>> M = MatrixSpace(K,Integer(4),Integer(2))([Integer(1),-Integer(1), Integer(0),-Integer(2),␣
↪Integer(0),-a**Integer(2)-Integer(1), Integer(0),a**Integer(2)-Integer(1)])
>>> M
[             1            -1]
[             0            -2]
[             0 -zeta12^2 - 1]
[             0  zeta12^2 - 1]
>>> M.kernel()
Vector space of degree 4 and dimension 2 over Cyclotomic Field of order 12
 and degree 4
Basis matrix:
[               0                1                0     -2*zeta12^2]
[               0                0                1 -2*zeta12^2 + 1]
```

复杂基域上的非平凡核。

```
sage: K  = FractionField(PolynomialRing(RationalField(),2,'x'))
sage: M  = MatrixSpace(K,  2)([[K.gen(1),K.gen(0)], [K.gen(1), K.gen(0)]])
sage: M
[x1 x0]
[x1 x0]
sage: M.kernel()
Vector space of degree 2 and dimension 1 over Fraction Field of Multivariate
Polynomial Ring in x0, x1 over Rational Field
Basis matrix:
 [ 1 -1]
```

```
>>>  from  sage.all  import  *
>>> K  = FractionField(PolynomialRing(RationalField(),Integer(2),'x'))
>>> M  = MatrixSpace(K, Integer(2))([[K.gen(Integer(1)),K.gen(Integer(0))], [K.gen(Integer(1)),␣
↪K.gen(Integer(0))]])
>>> M
[x1 x0]
[x1 x0]
>>> M.kernel()
Vector space of degree 2 and dimension 1 over Fraction Field of Multivariate
Polynomial Ring in x0, x1 over Rational Field
Basis matrix:
 [ 1 -1]
```

其他一些适用于整数矩阵的方法包括 `elementary_divisors`, `smith_form` （用于 Smith 标准型），`echelon_form` 用于 Hermite 标准型，`frobenius` 用于 Frobenius 标准型（有理规范型）。

有许多域（例如: **Q**）或有限域上的矩阵方法: `row_span`, `nullity`, `transpose`, `swap_rows`, `matrix_from_columns`, `matrix_from_rows` 等等。

请参阅文件 `matrix.py` 了解更多详情。

4.4 特征向量和特征值

如何使用 Sage 计算特征值和特征向量?

Sage 提供了一系列完整的函数来计算特征值和左右特征向量以及特征子空间。如果我们的矩阵为 A，那么 `eigenmatrix_right`（相对的为 `eightmatrix_left`）命令会给出矩阵 D 和 P, 使得 $AP = PD$ [illegible] $PA = DP$）。

```
sage: A  = matrix(QQ, [[1,1,0],[0,2,0],[0,0,3]])
sage: A
[1 1 0]
[0 2 0]
[0 0 3]
sage: A.eigenvalues()
[3, 2, 1]
sage: A.eigenvectors_right()
[(3, [(0, 0, 1)], 1), (2, [(1, 1, 0)], 1), (1, [(1, 0, 0)], 1)]

sage: A.eigenspaces_right()
[(3,
  Vector space of degree 3 and dimension 1 over Rational Field
  User basis matrix:
  [0 0 1]),
 (2,
```

(续下页)

(接上页)

```
  Vector space of degree 3 and dimension 1 over Rational Field
  User basis matrix:
  [1 1 0]),
 (1,
  Vector space of degree 3 and dimension 1 over Rational Field
  User basis matrix:
  [1 0 0])]

sage: D, P  = A.eigenmatrix_right()
sage: D
[3 0 0]
[0 2 0]
[0 0 1]
sage: P
[0 1 1]
[0 1 0]
[1 0 0]
sage: A*P  == P*D
True
```

```
>>>  from  sage.all  import  *
>>> A  = matrix(QQ, [[Integer(1),Integer(1),Integer(0)],[Integer(0),Integer(2),Integer(0)],
↪[Integer(0),Integer(0),Integer(3)]])
>>> A
[1 1 0]
[0 2 0]
[0 0 3]
>>> A.eigenvalues()
[3, 2, 1]
>>> A.eigenvectors_right()
[(3, [(0, 0, 1)], 1), (2, [(1, 1, 0)], 1), (1, [(1, 0, 0)], 1)]

>>> A.eigenspaces_right()
[(3,
  Vector space of degree 3 and dimension 1 over Rational Field
  User basis matrix:
  [0 0 1]),
 (2,
  Vector space of degree 3 and dimension 1 over Rational Field
  User basis matrix:
  [1 1 0]),
 (1,
  Vector space of degree 3 and dimension 1 over Rational Field
  User basis matrix:
  [1 0 0])]

>>> D, P  = A.eigenmatrix_right()
>>> D
[3 0 0]
[0 2 0]
[0 0 1]
>>> P
[0 1 1]
[0 1 0]
[1 0 0]
>>> A*P  == P*D
True
```

对于矩阵基环分数域之外的特征值，可以选择在实现域的代数闭包（例如代数数 QQbar）时，输出所有特征子空间。也可以为特征多项式的每个不可约因子请求独立的特征子空间，因为其他特征子空间可以通过伽罗瓦共轭形成。下列矩阵的特征值为 $pmsqrt{3}$，我们展示了每种可能的输出。

此外，目前 Sage 尚未实现多重精度数值特征值和特征向量，因此在 CC 或 RR 上调用特征函数可能会给出不准确且无意义的结果（还会打印警告）。具有浮点项的矩阵（在 CDF 和 RDF 上）的特征值和特征向量可以通过”eigenmatrix” 命令获得。

```
sage: MS  = MatrixSpace(QQ,  2,  2)
sage: A  = MS([1,-4,1,  -1])
sage: A.eigenspaces_left(format='all')
[(-1.732050807568878?*I,
  Vector space of degree 2 and dimension 1 over Algebraic Field
  User basis matrix:
  [                       1 -1 - 1.732050807568878?*I]),
 (1.732050807568878?*I,
  Vector space of degree 2 and dimension 1 over Algebraic Field
  User basis matrix:
  [                       1 -1 + 1.732050807568878?*I])]
sage: A.eigenspaces_left(format='galois')
[(a0,
  Vector space of degree 2 and dimension 1 over Number Field in a0 with defining polynomial x^2␣
↪+ 3
  User basis matrix:
  [     1 a0 - 1])]
```

```
>>>  from  sage.all  import  *
>>> MS  = MatrixSpace(QQ, Integer(2), Integer(2))
>>> A  = MS([Integer(1),-Integer(4),Integer(1),  -Integer(1)])
>>> A.eigenspaces_left(format='all')
[(-1.732050807568878?*I,
  Vector space of degree 2 and dimension 1 over Algebraic Field
  User basis matrix:
  [                       1 -1 - 1.732050807568878?*I]),
 (1.732050807568878?*I,
  Vector space of degree 2 and dimension 1 over Algebraic Field
  User basis matrix:
  [                       1 -1 + 1.732050807568878?*I])]
>>> A.eigenspaces_left(format='galois')
[(a0,
  Vector space of degree 2 and dimension 1 over Number Field in a0 with defining polynomial x^2␣
↪+ 3
  User basis matrix:
  [     1 a0 - 1])]
```

另一种方法是通过接口调用 Maxima：

```
sage: A  = maxima("matrix ([1, -4], [1, -1])")
sage: eig  = A.eigenvectors()
sage: eig.sage()
[[[-I*sqrt(3), I*sqrt(3)], [1, 1]], [[[1, 1/4*I*sqrt(3) + 1/4]], [[1, -1/4*I*sqrt(3) + 1/4]]]]
```

```
>>>  from  sage.all  import  *
>>> A  = maxima("matrix ([1, -4], [1, -1])")
>>> eig  = A.eigenvectors()
>>> eig.sage()
[[[-I*sqrt(3), I*sqrt(3)], [1, 1]], [[[1, 1/4*I*sqrt(3) + 1/4]], [[1, -1/4*I*sqrt(3) + 1/4]]]]
```

这告诉我们 $\vec{v}_1 = [1, (\sqrt{3}i + 1)/4]$ 是 $\lambda_1 = -\sqrt{3}i$ （重数为 1）的特征向量，而 $\vec{v}_2 = [1, (-\sqrt{3}i + 1)/4]$ 是 $\lambda_2 = \sqrt{3}i$ （重数也为 1）的特征向量。

以下是另外两个例子：

```
sage: A  = maxima("matrix ([11, 0, 0], [1, 11, 0], [1, 3, 2])")
sage: A.eigenvectors()
[[[2,11],[1,2]],[[[0,0,1]],[[0,1,1/3]]]]
sage: A  = maxima("matrix ([-1, 0, 0], [1, -1, 0], [1, 3, 2])")
sage: A.eigenvectors()
[[[-1,2],[2,1]],[[[0,1,-1]],[[0,0,1]]]]
```

```
>>> from sage.all import *
>>> A  = maxima("matrix ([11, 0, 0], [1, 11, 0], [1, 3, 2])")
>>> A.eigenvectors()
[[[2,11],[1,2]],[[[0,0,1]],[[0,1,1/3]]]]
>>> A  = maxima("matrix ([-1, 0, 0], [1, -1, 0], [1, 3, 2])")
>>> A.eigenvectors()
[[[-1,2],[2,1]],[[[0,1,-1]],[[0,0,1]]]]
```

警告：请注意输出的顺序是相反的，尽管矩阵几乎相同。

最后，你还可以使用 Sage 的 GAP 接口来计算“有理”特征值和特征向量：

```
sage: A  = libgap([[1,2,3],[4,5,6],[7,8,9]]); A
[ [ 1, 2, 3 ], [ 4, 5, 6 ], [ 7, 8, 9 ] ]
sage: libgap(QQ).Eigenvectors(A)
[ [ 1, -2, 1 ] ]
sage: libgap(QQ).Eigenvalues(A)
[ 0 ]
```

```
>>> from sage.all import *
>>> A  = libgap([[Integer(1),Integer(2),Integer(3)],[Integer(4),Integer(5),Integer(6)],
↪[Integer(7),Integer(8),Integer(9)]]); A
[ [ 1, 2, 3 ], [ 4, 5, 6 ], [ 7, 8, 9 ] ]
>>> libgap(QQ).Eigenvectors(A)
[ [ 1, -2, 1 ] ]
>>> libgap(QQ).Eigenvalues(A)
[ 0 ]
```

4.5 行化简

矩阵的行最简阶梯形式按以下示例计算。

```
sage: M  = MatrixSpace(RationalField(),2,3)
sage: A  = M([1,2,3,  4,5,6])
sage: A
[1 2 3]
[4 5 6]
sage: A.parent()
Full MatrixSpace of 2 by 3 dense matrices over Rational Field
sage: A[0,2]  =  389
sage: A
[  1   2 389]
[  4   5   6]
sage: A.echelon_form()
```

(续下页)

(接上页)

```
[      1       0 -1933/3]
[      0       1  1550/3]
```

```
>>> from sage.all import *
>>> M = MatrixSpace(RationalField(),Integer(2),Integer(3))
>>> A = M([Integer(1),Integer(2),Integer(3), Integer(4),Integer(5),Integer(6)])
>>> A
[1 2 3]
[4 5 6]
>>> A.parent()
Full MatrixSpace of 2 by 3 dense matrices over Rational Field
>>> A[Integer(0),Integer(2)] = Integer(389)
>>> A
[  1   2 389]
[  4   5   6]
>>> A.echelon_form()
[      1       0 -1933/3]
[      0       1  1550/3]
```

4.6 特征多项式

特征多项式是一个适用于方阵的 Sage 方法。

首先是 **Z** 上的矩阵：

```
sage: A = MatrixSpace(IntegerRing(),2)( [[1,2], [3,4]] )
sage: f = A.charpoly()
sage: f
x^2 - 5*x - 2
sage: f.parent()
Univariate Polynomial Ring in x over Integer Ring
```

```
>>> from sage.all import *
>>> A = MatrixSpace(IntegerRing(),Integer(2))( [[Integer(1),Integer(2)], [Integer(3),
↪Integer(4)]] )
>>> f = A.charpoly()
>>> f
x^2 - 5*x - 2
>>> f.parent()
Univariate Polynomial Ring in x over Integer Ring
```

我们计算一个定义在多项式环 $\mathbf{Z}[a]$ 上的矩阵的特征多项式：

```
sage: R = PolynomialRing(IntegerRing(),'a'); a = R.gen()
sage: M = MatrixSpace(R,2)([[a,1], [a,a+1]])
sage: M
[    a     1]
[    a a + 1]
sage: f = M.charpoly()
sage: f
x^2 + (-2*a - 1)*x + a^2
sage: f.parent()
Univariate Polynomial Ring in x over Univariate Polynomial Ring in a over
Integer Ring
```

(续下页)

(接上页)

```
sage: M.trace()
2*a + 1
sage: M.determinant()
a^2
```

```
>>> from sage.all import *
>>> R = PolynomialRing(IntegerRing(),'a'); a = R.gen()
>>> M = MatrixSpace(R,Integer(2))([[a,Integer(1)], [a,a+Integer(1)]])
>>> M
[    a     1]
[    a a + 1]
>>> f = M.charpoly()
>>> f
x^2 + (-2*a - 1)*x + a^2
>>> f.parent()
Univariate Polynomial Ring in x over Univariate Polynomial Ring in a over
Integer Ring

>>> M.trace()
2*a + 1
>>> M.determinant()
a^2
```

我们计算一个定义在多元多项式环 $\mathbf{Z}[u, v]$ 上的矩阵的特征多项式：

```
sage: R.<u,v> = PolynomialRing(ZZ,2)
sage: A = MatrixSpace(R,2)([u,v,u^2,v^2])
sage: f = A.charpoly(); f
x^2 + (-v^2 - u)*x - u^2*v + u*v^2
```

```
>>> from sage.all import *
>>> R = PolynomialRing(ZZ,Integer(2), names=('u', 'v',)); (u, v,) = R._first_ngens(2)
>>> A = MatrixSpace(R,Integer(2))([u,v,u**Integer(2),v**Integer(2)])
>>> f = A.charpoly(); f
x^2 + (-v^2 - u)*x - u^2*v + u*v^2
```

区分变量有点困难。为了解决这个问题，我们可能需要重命名不定变量”Z”，我们可以轻松地执行如下操作：

```
sage: f = A.charpoly('Z'); f
Z^2 + (-v^2 - u)*Z - u^2*v + u*v^2
```

```
>>> from sage.all import *
>>> f = A.charpoly('Z'); f
Z^2 + (-v^2 - u)*Z - u^2*v + u*v^2
```

4.7 求解线性方程组

使用 Maxima，可以轻松求解线性方程：

```
sage: var('a,b,c')
(a, b, c)
sage: eqn = [a+b*c==1, b-a*c==0, a+b==5]
sage: s = solve(eqn, a,b,c); s
```

(续下页)

(接上页)

```
[[a == -1/4*I*sqrt(79) + 11/4, b == 1/4*I*sqrt(79) + 9/4, c == 1/10*I*sqrt(79) + 1/10], [a == 1/
↪4*I*sqrt(79) + 11/4, b == -1/4*I*sqrt(79) + 9/4, c == -1/10*I*sqrt(79) + 1/10]]
```

```
>>> from sage.all import *
>>> var('a,b,c')
(a, b, c)
>>> eqn = [a+b*c==Integer(1), b-a*c==Integer(0), a+b==Integer(5)]
>>> s = solve(eqn, a,b,c); s
[[a == -1/4*I*sqrt(79) + 11/4, b == 1/4*I*sqrt(79) + 9/4, c == 1/10*I*sqrt(79) + 1/10], [a == 1/
↪4*I*sqrt(79) + 11/4, b == -1/4*I*sqrt(79) + 9/4, c == -1/10*I*sqrt(79) + 1/10]]
```

你甚至可以用 LaTeX 很好地排版方程的解:

```
sage.: print(latex(s))
...
```

要通过 xdvi 在屏幕上显示上述内容，请输入 `view(s)`。

你还可以通过 `solve` 命令符号化求解线性方程:

```
sage: var('x,y,z,a')
(x, y, z, a)
sage: eqns = [x + z == y, 2*a*x - y == 2*a^2, y - 2*z == 2]
sage: solve(eqns, x, y, z)
[[x == a + 1, y == 2*a, z == a - 1]]
```

```
>>> from sage.all import *
>>> var('x,y,z,a')
(x, y, z, a)
>>> eqns = [x + z == y, Integer(2)*a*x - y == Integer(2)*a**Integer(2), y - Integer(2)*z␣
↪ == Integer(2)]
>>> solve(eqns, x, y, z)
[[x == a + 1, y == 2*a, z == a - 1]]
```

这是一个数值化 Numpy 示例:

```
sage: from numpy import arange, eye, linalg
sage: A = eye(10)          ##   the 10x10 identity matrix
sage: b = arange(1,11)
sage: x = linalg.solve(A,b)
```

```
>>> from sage.all import *
>>> from numpy import arange, eye, linalg
>>> A = eye(Integer(10))          ##   the 10x10 identity matrix
>>> b = arange(Integer(1),Integer(11))
>>> x = linalg.solve(A,b)
```

另一种数值化求解方程组的方法是使用 Sage 的 Octave 接口:

```
sage: M33 = MatrixSpace(QQ,3,3)
sage: A = M33([1,2,3,4,5,6,7,8,0])
sage: V3 = VectorSpace(QQ,3)
sage: b = V3([1,2,3])
sage: octave.solve_linear_system(A,b)      # optional - octave
[-0.333333, 0.666667, 0]
```

```
>>> from sage.all import *
>>> M33 = MatrixSpace(QQ,Integer(3),Integer(3))
>>> A = M33([Integer(1),Integer(2),Integer(3),Integer(4),Integer(5),Integer(6),Integer(7),
↪Integer(8),Integer(0)])
>>> V3 = VectorSpace(QQ,Integer(3))
>>> b = V3([Integer(1),Integer(2),Integer(3)])
>>> octave.solve_linear_system(A,b)      # optional - octave
[-0.333333, 0.666667, 0]
```

CHAPTER 5

线性码与密码

5.1 码

长度为 n 的线性码是 $GF(q)^n$ 的有限维子空间。Sage 能够在一定程度上计算线性纠错码。它基本上对 GAP 和 GUAVA 命令进行了一些封装。GUAVA 2.8 未包含在 Sage 4.0 的 GAP 安装中，但可以作为可选包安装。

Sage 能够计算汉明 (Hamming) 码

```
sage: C  = codes.HammingCode(GF(3),  3)
sage: C
[13, 10] Hamming Code over GF(3)
sage: C.minimum_distance()
3
sage: C.generator_matrix()
[1 0 0 0 0 0 0 0 0 0 1 2 0]
[0 1 0 0 0 0 0 0 0 0 0 1 2]
[0 0 1 0 0 0 0 0 0 0 1 0 2]
[0 0 0 1 0 0 0 0 0 0 1 1 1]
[0 0 0 0 1 0 0 0 0 0 1 1 2]
[0 0 0 0 0 1 0 0 0 0 2 0 2]
[0 0 0 0 0 0 1 0 0 0 1 2 1]
[0 0 0 0 0 0 0 1 0 0 2 1 1]
[0 0 0 0 0 0 0 0 1 0 2 2 0]
[0 0 0 0 0 0 0 0 0 1 0 1 1]
```

```
>>>  from  sage.all  import  *
>>> C  = codes.HammingCode(GF(Integer(3)), Integer(3))
>>> C
[13, 10] Hamming Code over GF(3)
>>> C.minimum_distance()
3
>>> C.generator_matrix()
[1 0 0 0 0 0 0 0 0 0 1 2 0]
[0 1 0 0 0 0 0 0 0 0 0 1 2]
```

(续下页)

(接上页)

```
[0 0 1 0 0 0 0 0 0 0 1 0 2]
[0 0 0 1 0 0 0 0 0 0 1 1 1]
[0 0 0 0 1 0 0 0 0 0 1 1 2]
[0 0 0 0 0 1 0 0 0 0 2 0 2]
[0 0 0 0 0 0 1 0 0 0 1 2 1]
[0 0 0 0 0 0 0 1 0 0 2 1 1]
[0 0 0 0 0 0 0 0 1 0 2 2 0]
[0 0 0 0 0 0 0 0 0 1 0 1 1]
```

四种戈莱 (Golay) 码

```
sage: C  = codes.GolayCode(GF(3))
sage: C
[12, 6, 6] Extended Golay code over GF(3)
sage: C.minimum_distance()
6
sage: C.generator_matrix()
[1 0 0 0 0 0 2 0 1 2 1 2]
[0 1 0 0 0 0 1 2 2 2 1 0]
[0 0 1 0 0 0 1 1 1 0 1 1]
[0 0 0 1 0 0 1 1 0 2 2 2]
[0 0 0 0 1 0 2 1 2 2 0 1]
[0 0 0 0 0 1 0 2 1 2 2 1]
```

```
>>> from sage.all import *
>>> C  = codes.GolayCode(GF(Integer(3)))
>>> C
[12, 6, 6] Extended Golay code over GF(3)
>>> C.minimum_distance()
6
>>> C.generator_matrix()
[1 0 0 0 0 0 2 0 1 2 1 2]
[0 1 0 0 0 0 1 2 2 2 1 0]
[0 0 1 0 0 0 1 1 1 0 1 1]
[0 0 0 1 0 0 1 1 0 2 2 2]
[0 0 0 0 1 0 2 1 2 2 0 1]
[0 0 0 0 0 1 0 2 1 2 2 1]
```

以及二进制 Reed-Muller 码、二次剩余码、准二次剩余码、“随机”线性码和由满秩矩阵生成的码（通常使用行作为基）。

对于给定的码 C，Sage 可以返回生成矩阵、检验矩阵和对偶码：

```
sage: C  = codes.HammingCode(GF(2),  3)
sage: Cperp  = C.dual_code()
sage: C; Cperp
[7, 4] Hamming Code over GF(2)
[7, 3] linear code over GF(2)
sage: C.generator_matrix()
  [1 0 0 0 0 1 1]
  [0 1 0 0 1 0 1]
  [0 0 1 0 1 1 0]
  [0 0 0 1 1 1 1]
sage: C.parity_check_matrix()
  [1 0 1 0 1 0 1]
  [0 1 1 0 0 1 1]
```

(续下页)

(接上页)

```
  [0 0 0 1 1 1 1]
sage: C.dual_code()
[7, 3] linear code over GF(2)
sage: C  = codes.HammingCode(GF(4,'a'),  3)
sage: C.dual_code()
[21, 3] linear code over GF(4)
```

```
>>>  from  sage.all  import  *
>>> C  = codes.HammingCode(GF(Integer(2)), Integer(3))
>>> Cperp  = C.dual_code()
>>> C; Cperp
[7, 4] Hamming Code over GF(2)
[7, 3] linear code over GF(2)
>>> C.generator_matrix()
  [1 0 0 0 0 1 1]
  [0 1 0 0 1 0 1]
  [0 0 1 0 1 1 0]
  [0 0 0 1 1 1 1]
>>> C.parity_check_matrix()
  [1 0 1 0 1 0 1]
  [0 1 1 0 0 1 1]
  [0 0 0 1 1 1 1]
>>> C.dual_code()
[7, 3] linear code over GF(2)
>>> C  = codes.HammingCode(GF(Integer(4),'a'), Integer(3))
>>> C.dual_code()
[21, 3] linear code over GF(4)
```

对于 C 和向量 $v \in GF(q)^n$，Sage 可以尝试使用校验子解码 (syndrome decoding) 来解码 v （即，在汉明度量下找到最接近 v 的码字 $c \in C$）。到目前为止，尚未实现任何特殊的解码方法。

```
sage: C  = codes.HammingCode(GF(2),  3)
sage: MS  = MatrixSpace(GF(2),1,7)
sage: F  = GF(2); a  = F.gen()
sage: v  = vector([a,a,F(0),a,a,F(0),a])
sage: c  = C.decode_to_code(v,  "Syndrome"); c
(1, 1, 0, 1, 0, 0, 1)
sage: c  in C
True
```

```
>>>  from  sage.all  import  *
>>> C  = codes.HammingCode(GF(Integer(2)), Integer(3))
>>> MS  = MatrixSpace(GF(Integer(2)),Integer(1),Integer(7))
>>> F  = GF(Integer(2)); a  = F.gen()
>>> v  = vector([a,a,F(Integer(0)),a,a,F(Integer(0)),a])
>>> c  = C.decode_to_code(v,  "Syndrome"); c
(1, 1, 0, 1, 0, 0, 1)
>>> c  in C
True
```

要绘制码的权重分布（直方图），可以使用 Sage 附带的 matplotlib 包：

```
sage: C  = codes.HammingCode(GF(2),  4)
sage: C
[15, 11] Hamming Code over GF(2)
sage: w  = C.weight_distribution(); w
```

(续下页)

(接上页)

```
 [1, 0, 0, 35, 105, 168, 280, 435, 435, 280, 168, 105, 35, 0, 0, 1]
sage: J  =  range(len(w))
sage: W  = IndexedSequence([ZZ(w[i])  for i  in J],J)
sage: P  = W.plot_histogram()
```

```
>>>  from  sage.all  import  *
>>> C  = codes.HammingCode(GF(Integer(2)), Integer(4))
>>> C
[15, 11] Hamming Code over GF(2)
>>> w  = C.weight_distribution(); w
 [1, 0, 0, 35, 105, 168, 280, 435, 435, 280, 168, 105, 35, 0, 0, 1]
>>> J  =  range(len(w))
>>> W  = IndexedSequence([ZZ(w[i])  for i  in J],J)
>>> P  = W.plot_histogram()
```

现在输入 `show(P)` 来展示此内容。

我们完全跳过了几个编码理论函数。请参阅参考手册或文件 `coding/linear_codes.py` 以获取示例。

Sage 还可以通过 Singular 接口 § sec:agcodes 计算代数几何码，称为 AG 码。还可以通过 Sage 接口将 GUAVA 的 AG 码实现用于 GAP `gap_console()`。更多细节请参阅 GUAVA 手册。{GUAVA}

5.2 密码

5.2.1 线性反馈移位寄存器 (LFSR)

在 Sage 中实现了一种特殊类型的流密码，即定义在有限域上的线性反馈移位寄存器（LFSR）序列。流密码长期以来一直被用作伪随机数生成器的来源。{linear feedback shift register}

S. Golomb {G} 给出了数字序列 $\mathbf{a} = \{a_n\}_{n=1}^{\infty}$ ($a_n \in \{0, 1\}$) 被认为是“随机”的三个统计特征。将 $\mathbf{a}$ 的自相关定义为：

$$C(k) = C(k, \mathbf{a}) = \lim_{N\to\infty} \frac{1}{N} \sum_{n=1}^{N} (-1)^{a_n + a_{n+k}}.$$

如果 a 是周期性的，周期为 P，则该公式可以简化为：

$$C(k) = \frac{1}{P} \sum_{n=1}^{P} (-1)^{a_n + a_{n+k}}$$

假设 a 是周期性的，周期为 P。

- 平衡：$|\sum_{n=1}^{P} (-1)^{a_n}| \leq 1$.
- 低自相关：

$$C(k) = \begin{cases} 1, & k = 0, \\ \epsilon, & k \neq 0. \end{cases}$$

  （对于满足头两个特性的序列，已知 $\epsilon = -1/P$ 必然成立。）

- 比例运行特性：在每个周期中，一半的递推序列长度为 1, 四分之一长度为 2, 等等。此外，1 的递推次数和 0 的递推次数相等。

满足这些特性的序列将被称为伪随机数列。{pseudo-random}

一般反馈移位寄存器是一个映射 $f : \mathbf{F}_q^d \to \mathbf{F}_q^d$，形式为：

$$f(x_0, ..., x_{n-1}) = (x_1, x_2, ..., x_n),$$
$$x_n = C(x_0, ..., x_{n-1}),$$

其中 $C : \mathbf{F}_q^d \to \mathbf{F}_q$ 是一个给定的函数。当 C 形式为：

$$C(x_0, ..., x_{n-1}) = c_0 x_0 + ... + c_{n-1} x_{n-1},$$

对于某些给定常数 $c_i \in \mathbf{F}_q$, 该映射被称为线性反馈移位寄存器（LFSR）。系数序列 c_i 被称为密钥，多项式

$$C(x) = 1 + c_0 x + \cdots + c_{n-1} x^n$$

有时被称为连接多项式。

例如：在 $GF(2)$ 上，如果 $[c_0, c_1, c_2, c_3] = [1, 0, 0, 1]$ 则 $C(x) = 1 + x + x^4$,

x_n = x_{n-4} + x_{n-1},ngeq 4.

那么 LFSR 序列为

$$1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1,$$
$$1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1, ...$$

此 0, 1 序列是周期性的，周期为 $P = 2^4 - 1 = 15$，并满足 Golomb 的三个随机性条件。然而，这个以 15 为周期的序列可以通过只知道 8 个项被“破解”（即生成 $g(x)$ 的过程）！这是 Berlekamp-Massey 算法 {M} 的功能，已通过 `lfsr_connection_polynomial` 实现（产生 `berlekamp_massey` 的反向结果）。

```
sage: F  = GF(2)
sage: o  = F(0)
sage: l  = F(1)
sage: key  = [l,o,o,l]; fill  = [l,l,o,l]; n  =  20
sage: s  = lfsr_sequence(key,fill,n); s
[1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0]
sage: lfsr_autocorrelation(s,15,7)
4/15
sage: lfsr_autocorrelation(s,15,0)
8/15
sage: lfsr_connection_polynomial(s)
x^4 + x + 1
sage:  from  sage.matrix.berlekamp_massey  import berlekamp_massey
sage: berlekamp_massey(s)
x^4 + x^3 + 1
```

```
>>>  from  sage.all  import  *
>>> F  = GF(Integer(2))
>>> o  = F(Integer(0))
>>> l  = F(Integer(1))
>>> key  = [l,o,o,l]; fill  = [l,l,o,l]; n  = Integer(20)
>>> s  = lfsr_sequence(key,fill,n); s
[1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0]
>>> lfsr_autocorrelation(s,Integer(15),Integer(7))
4/15
>>> lfsr_autocorrelation(s,Integer(15),Integer(0))
8/15
>>> lfsr_connection_polynomial(s)
x^4 + x + 1
>>>  from  sage.matrix.berlekamp_massey  import berlekamp_massey
>>> berlekamp_massey(s)
x^4 + x^3 + 1
```

5.2.2 经典密码

有一个用于加密系统的类型（由 David Kohel 创建，他还编写了下面的示例），实现了经典加密系统。一般接口如下：

```
sage: S  = AlphabeticStrings()
sage: S
Free alphabetic string monoid on A-Z
sage: E  = SubstitutionCryptosystem(S)
sage: E
Substitution cryptosystem on Free alphabetic string monoid on A-Z
sage: K  = S([  25-i  for i  in  range(26) ])
sage: e  = E(K)
sage: m  = S("THECATINTHEHAT")
sage: e(m)
GSVXZGRMGSVSZG
```

```
>>>  from  sage.all  import  *
>>> S  = AlphabeticStrings()
>>> S
Free alphabetic string monoid on A-Z
>>> E  = SubstitutionCryptosystem(S)
>>> E
Substitution cryptosystem on Free alphabetic string monoid on A-Z
>>> K  = S([ Integer(25)-i  for i  in  range(Integer(26)) ])
>>> e  = E(K)
>>> m  = S("THECATINTHEHAT")
>>> e(m)
GSVXZGRMGSVSZG
```

下面是另一个例子：

```
sage: S  = AlphabeticStrings()
sage: E  = TranspositionCryptosystem(S,15);
sage: m  = S("THECATANDTHEHAT")
sage: G  = E.key_space()
sage: G
Symmetric group of order 15! as a permutation group
sage: g  = G([  3,  2,  1,  6,  5,  4,  9,  8,  7,  12,  11,  10,  15,  14,  13 ])
sage: e  = E(g)
sage: e(m)
EHTTACDNAEHTTAH
```

```
>>>  from  sage.all  import  *
>>> S  = AlphabeticStrings()
>>> E  = TranspositionCryptosystem(S,Integer(15));
>>> m  = S("THECATANDTHEHAT")
>>> G  = E.key_space()
>>> G
Symmetric group of order 15! as a permutation group
>>> g  = G([ Integer(3), Integer(2), Integer(1), Integer(6), Integer(5), Integer(4), Integer(9),
↪ Integer(8), Integer(7), Integer(12), Integer(11), Integer(10), Integer(15), Integer(14),␣
↪Integer(13) ])
>>> e  = E(g)
>>> e(m)
EHTTACDNAEHTTAH
```

其思想是加密系统是一个映射 $E : KS \to \mathrm{Hom}_{\mathrm{Set}}(MS, CS)$，其中 math:$KS$, MS 和 CS 分别是密钥空间、明

文（或消息）空间和密文空间。假设 E 为单射，所以 `e.key()` 返回原像密钥。

CHAPTER 6

表示论

6.1 普通特征标 (Ordinary characters)

如何在 Sage 中计算有限群的特征标表？可以使用 Sage-GAP 接口来计算特征标表。

你可以使用 `PermutationGroup` 类的 `character_table` 方法，或通过 GAP 命令 `CharacterTable` 的接口，将置换群 G 的特征标值表构建为 Sage 矩阵。

```
sage: G  = PermutationGroup([[(1,2),(3,4)], [(1,2,3,4)]])
sage: G.order()
8
sage: G.character_table()  # random
[ 1  1  1  1  1]
[ 1 -1 -1  1  1]
[ 1 -1  1 -1  1]
[ 1  1 -1 -1  1]
[ 2  0  0  0 -2]
sage: CT  = libgap(G).CharacterTable()
sage: CT.Display()  # random
CT1

 2  3  2  2  2  3

   1a 2a 2b 4a 2c
2P 1a 1a 1a 2c 1a
3P 1a 2a 2b 4a 2c

X.1     1  1  1  1  1
X.2     1 -1 -1  1  1
X.3     1 -1  1 -1  1
X.4     1  1 -1 -1  1
X.5     2  .  .  . -2
```

```
>>> from sage.all import *
>>> G = PermutationGroup([[(Integer(1),Integer(2)),(Integer(3),Integer(4))], [(Integer(1),
↪Integer(2),Integer(3),Integer(4))]])
>>> G.order()
8
>>> G.character_table()  # random
[ 1  1  1  1  1]
[ 1 -1 -1  1  1]
[ 1 -1  1 -1  1]
[ 1  1 -1 -1  1]
[ 2  0  0  0 -2]
>>> CT = libgap(G).CharacterTable()
>>> CT.Display()  # random
CT1
<BLANKLINE>
 2  3  2  2  2  3
<BLANKLINE>
   1a 2a 2b 4a 2c
2P 1a 1a 1a 2c 1a
3P 1a 2a 2b 4a 2c
<BLANKLINE>
X.1     1  1  1  1  1
X.2     1 -1 -1  1  1
X.3     1 -1  1 -1  1
X.4     1  1 -1 -1  1
X.5     2  .  .  . -2
```

下面是另一个示例:

```
sage: G = PermutationGroup([[(1,2),(3,4)], [(1,2,3)]])
sage: G.character_table()  # random
[         1          1          1          1]
[         1 -zeta3 - 1      zeta3          1]
[         1      zeta3 -zeta3 - 1          1]
[         3          0          0         -1]
sage: G = libgap.eval("Group((1,2)(3,4),(1,2,3))"); G
Group([ (1,2)(3,4), (1,2,3) ])
sage: T = G.CharacterTable()
sage: T.Display()  # random
CT2

     2  2  .  .  2
     3  1  1  1  .

       1a 3a 3b 2a
    2P 1a 3b 3a 1a
    3P 1a 1a 1a 2a

X.1     1  1  1  1
X.2     1  A /A  1
X.3     1 /A  A  1
X.4     3  .  . -1

A = E(3)^2
  = (-1-Sqrt(-3))/2 = -1-b3
```

```
>>> from sage.all import *
```

(续下页)

(接上页)

```
>>> G  = PermutationGroup([[(Integer(1),Integer(2)),(Integer(3),Integer(4))], [(Integer(1),
↪Integer(2),Integer(3))]])
>>> G.character_table()  # random
[         1          1          1          1]
[         1 -zeta3 - 1      zeta3          1]
[         1      zeta3 -zeta3 - 1          1]
[         3          0          0         -1]
>>> G  = libgap.eval("Group((1,2)(3,4),(1,2,3))"); G
Group([ (1,2)(3,4), (1,2,3) ])
>>> T  = G.CharacterTable()
>>> T.Display()  # random
CT2
<BLANKLINE>
     2  2  .  .  2
     3  1  1  1  .
<BLANKLINE>
       1a 3a 3b 2a
    2P 1a 3b 3a 1a
    3P 1a 1a 1a 2a
<BLANKLINE>
X.1     1  1  1  1
X.2     1  A /A  1
X.3     1 /A  A  1
X.4     3  .  . -1
<BLANKLINE>
A = E(3)^2
  = (-1-Sqrt(-3))/2 = -1-b3
```

其中 $E(3)$ 表示单位立方根，$ER(-3)$ 表示 -3 的平方根，即 $i\sqrt{3}$, 而 $b3 = \frac{1}{2}(-1+i\sqrt{3})$。请注意添加的 `print` Python 命令。这会令输出更美观。

```
sage: irr  = G.Irr();  sorted(irr)
[Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, 1, 1, 1 ] ),
 Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, E(3)^2, E(3), 1 ] ),
 Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, E(3), E(3)^2, 1 ] ),
 Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 3, 0, 0, -1 ] )]
sage: irr.Display()  # random
[ [       1,       1,       1,       1 ],
  [       1,  E(3)^2,    E(3),       1 ],
  [       1,    E(3),  E(3)^2,       1 ],
  [       3,       0,       0,      -1 ] ]
sage: CG  = G.ConjugacyClasses(); CG
[ ()^G, (2,3,4)^G, (2,4,3)^G, (1,2)(3,4)^G ]
sage: gamma  = CG[2]; gamma
(2,4,3)^G
sage: g  = gamma.Representative(); g
(2,4,3)
sage: chi  = irr[1]; chi  # random
Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, E(3)^2, E(3), 1 ] )
sage: g^chi  # random
E(3)
```

```
>>>  from  sage.all  import  *
>>> irr  = G.Irr();  sorted(irr)
[Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, 1, 1, 1 ] ),
 Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, E(3)^2, E(3), 1 ] ),
```

(续下页)

(接上页)

```
 Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, E(3), E(3)^2, 1 ] ),
 Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 3, 0, 0, -1 ] )]
>>> irr.Display()  # random
[ [       1,       1,       1,       1 ],
  [       1,  E(3)^2,    E(3),       1 ],
  [       1,    E(3),  E(3)^2,       1 ],
  [       3,       0,       0,      -1 ] ]
>>> CG  = G.ConjugacyClasses(); CG
[ ()^G, (2,3,4)^G, (2,4,3)^G, (1,2)(3,4)^G ]
>>> gamma  = CG[Integer(2)]; gamma
(2,4,3)^G
>>> g  = gamma.Representative(); g
(2,4,3)
>>> chi  = irr[Integer(1)]; chi  # random
Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, E(3)^2, E(3), 1 ] )
>>> g**chi  # random
E(3)
```

最后一个量是特征 `chi` 在群元素 `g` 处的值。

或者，如果你关闭 IPython 的“美观打印”，那么表格将打印得更好。

```
sage:  %Pprint
Pretty printing has been turned OFF
sage: G  = libgap.eval("Group((1,2)(3,4),(1,2,3))"); G
Group([ (1,2)(3,4), (1,2,3) ])
sage: T  = G.CharacterTable(); T
CharacterTable( Alt( [ 1 .. 4 ] ) )
sage: T.Display()
CT3

     2  2  2  .  .
     3  1  .  1  1

       1a 2a 3a 3b
    2P 1a 1a 3b 3a
    3P 1a 2a 1a 1a

X.1     1  1  1  1
X.2     1  1  A /A
X.3     1  1 /A  A
X.4     3 -1  .  .

A = E(3)^2
  = (-1-Sqrt(-3))/2 = -1-b3
sage: irr  = G.Irr(); irr
[ Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, 1, 1, 1 ] ),
  Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, 1, E(3)^2, E(3) ] ),
  Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, 1, E(3), E(3)^2 ] ),
  Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 3, -1, 0, 0 ] ) ]
sage: irr.Display()
[ [       1,       1,       1,       1 ],
  [       1,       1,  E(3)^2,    E(3) ],
  [       1,       1,    E(3),  E(3)^2 ],
  [       3,      -1,       0,       0 ] ]
sage:  %Pprint
Pretty printing has been turned ON
```

```
>>> from sage.all import *
>>> %Pprint
Pretty printing has been turned OFF
>>> G = libgap.eval("Group((1,2)(3,4),(1,2,3))"); G
Group([ (1,2)(3,4), (1,2,3) ])
>>> T = G.CharacterTable(); T
CharacterTable( Alt( [ 1 .. 4 ] ) )
>>> T.Display()
CT3
<BLANKLINE>
     2  2  2  .  .
     3  1  .  1  1
<BLANKLINE>
       1a 2a 3a 3b
    2P 1a 1a 3b 3a
    3P 1a 2a 1a 1a
<BLANKLINE>
X.1     1  1  1  1
X.2     1  1  A /A
X.3     1  1 /A  A
X.4     3 -1  .  .
<BLANKLINE>
A = E(3)^2
  = (-1-Sqrt(-3))/2 = -1-b3
>>> irr = G.Irr(); irr
[ Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, 1, 1, 1 ] ),
  Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, 1, E(3)^2, E(3) ] ),
  Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 1, 1, E(3), E(3)^2 ] ),
  Character( CharacterTable( Alt( [ 1 .. 4 ] ) ), [ 3, -1, 0, 0 ] ) ]
>>> irr.Display()
[ [       1,       1,       1,       1 ],
  [       1,       1,  E(3)^2,    E(3) ],
  [       1,       1,    E(3),  E(3)^2 ],
  [       3,      -1,       0,       0 ] ]
>>> %Pprint
Pretty printing has been turned ON
```

6.2 布劳尔特征标 (Brauer characters)

GAP 中的布劳尔特征标表尚未具有“原生”接口。要访问它们，你可以使用 `libgap.eval` 命令直接与 GAP 交互。

下面的示例通过使用 GAP 接口来说明语法。

```
sage: G = libgap.eval("Group((1,2)(3,4),(1,2,3))"); G
Group([ (1,2)(3,4), (1,2,3) ])
sage: irr = G.IrreducibleRepresentations(GF(7)); irr    # random arch. dependent output
[ [ (1,2)(3,4), (1,2,3) ] -> [ [ [ Z(7)^0 ] ], [ [ Z(7)^4 ] ] ],
  [ (1,2)(3,4), (1,2,3) ] -> [ [ [ Z(7)^0 ] ], [ [ Z(7)^2 ] ] ],
  [ (1,2)(3,4), (1,2,3) ] -> [ [ [ Z(7)^0 ] ], [ [ Z(7)^0 ] ] ],
  [ (1,2)(3,4), (1,2,3) ] ->
    [ [ [ Z(7)^2, Z(7)^5, Z(7) ], [ Z(7)^3, Z(7)^2, Z(7)^3 ],
        [ Z(7), Z(7)^5, Z(7)^2 ] ],
      [ [ 0*Z(7), Z(7)^0, 0*Z(7) ], [ 0*Z(7), 0*Z(7), Z(7)^0 ],
        [ Z(7)^0, 0*Z(7), 0*Z(7) ] ] ] ]
sage: brvals = [[chi.Image(c.Representative()).BrauerCharacterValue()
```

(续下页)

(接上页)

```
....:             for c  in G.ConjugacyClasses()]  for chi  in irr]
sage: brvals          # random architecture dependent output
[ [       1,       1,  E(3)^2,    E(3) ],
  [       1,       1,    E(3),  E(3)^2 ],
  [       1,       1,       1,       1 ],
  [       3,      -1,       0,       0 ] ]
sage: T  = G.CharacterTable()
sage: T.Display()  # random
CT3

     2  2  .  .  2
     3  1  1  1  .

       1a 3a 3b 2a
    2P 1a 3b 3a 1a
    3P 1a 1a 1a 2a

X.1     1  1  1  1
X.2     1  A /A  1
X.3     1 /A  A  1
X.4     3  .  . -1

A = E(3)^2
  = (-1-Sqrt(-3))/2 = -1-b3
```

```
>>>  from  sage.all  import  *
>>> G  = libgap.eval("Group((1,2)(3,4),(1,2,3))"); G
Group([ (1,2)(3,4), (1,2,3) ])
>>> irr  = G.IrreducibleRepresentations(GF(Integer(7))); irr    # random arch. dependent output
[ [ (1,2)(3,4), (1,2,3) ] -> [ [ [ Z(7)^0 ] ], [ [ Z(7)^4 ] ] ],
  [ (1,2)(3,4), (1,2,3) ] -> [ [ [ Z(7)^0 ] ], [ [ Z(7)^2 ] ] ],
  [ (1,2)(3,4), (1,2,3) ] -> [ [ [ Z(7)^0 ] ], [ [ Z(7)^0 ] ] ],
  [ (1,2)(3,4), (1,2,3) ] ->
    [ [ [ Z(7)^2, Z(7)^5, Z(7) ], [ Z(7)^3, Z(7)^2, Z(7)^3 ],
        [ Z(7), Z(7)^5, Z(7)^2 ] ],
      [ [ 0*Z(7), Z(7)^0, 0*Z(7) ], [ 0*Z(7), 0*Z(7), Z(7)^0 ],
        [ Z(7)^0, 0*Z(7), 0*Z(7) ] ] ] ]
>>> brvals  = [[chi.Image(c.Representative()).BrauerCharacterValue()
...             for c  in G.ConjugacyClasses()]  for chi  in irr]
>>> brvals          # random architecture dependent output
[ [       1,       1,  E(3)^2,    E(3) ],
  [       1,       1,    E(3),  E(3)^2 ],
  [       1,       1,       1,       1 ],
  [       3,      -1,       0,       0 ] ]
>>> T  = G.CharacterTable()
>>> T.Display()  # random
CT3
<BLANKLINE>
     2  2  .  .  2
     3  1  1  1  .
<BLANKLINE>
       1a 3a 3b 2a
    2P 1a 3b 3a 1a
    3P 1a 1a 1a 2a
<BLANKLINE>
X.1     1  1  1  1
```

(续下页)

(接上页)

```
X.2     1  A /A  1
X.3     1 /A  A  1
X.4     3  .  . -1
<BLANKLINE>
A = E(3)^2
  = (-1-Sqrt(-3))/2 = -1-b3
```

CHAPTER 7

环

7.1 矩阵环

如何在 Sage 中构建有限环上的矩阵环？`MatrixSpace` 构造函数接受任意环作为基环。下面是语法示例：

```
sage: R  = IntegerModRing(51)
sage: M  = MatrixSpace(R,3,3)
sage: M(0)
[0 0 0]
[0 0 0]
[0 0 0]
sage: M(1)
[1 0 0]
[0 1 0]
[0 0 1]
sage:  5*M(1)
[5 0 0]
[0 5 0]
[0 0 5]
```

```
>>>  from  sage.all  import  *
>>> R  = IntegerModRing(Integer(51))
>>> M  = MatrixSpace(R,Integer(3),Integer(3))
>>> M(Integer(0))
[0 0 0]
[0 0 0]
[0 0 0]
>>> M(Integer(1))
[1 0 0]
[0 1 0]
[0 0 1]
>>> Integer(5)*M(Integer(1))
[5 0 0]
[0 5 0]
```

(续下页)

(接上页)

```
[0 0 5]
```

7.2 多项式环

如何在 Sage 中构建有限域上的多项式环？下面是一个示例：

```
sage: R  = PolynomialRing(GF(97),'x')
sage: x  = R.gen()
sage: f  = x^2+7
sage: f  in R
True
```

```
>>>  from  sage.all  import  *
>>> R  = PolynomialRing(GF(Integer(97)),'x')
>>> x  = R.gen()
>>> f  = x**Integer(2)+Integer(7)
>>> f  in R
True
```

下面是使用 Singular 接口的示例：

```
sage: R  = singular.ring(97,  '(a,b,c,d)',  'lp')
sage: I  = singular.ideal(['a+b+c+d',  'ab+ad+bc+cd',  'abc+abd+acd+bcd',  'abcd-1'])
sage: R
polynomial ring, over a field, global ordering
// coefficients: ZZ/97...
// number of vars : 4
//        block   1 : ordering lp
//                  : names    a b c d
//        block   2 : ordering C
sage: I
a+b+c+d,
a*b+a*d+b*c+c*d,
a*b*c+a*b*d+a*c*d+b*c*d,
a*b*c*d-1
```

```
>>>  from  sage.all  import  *
>>> R  = singular.ring(Integer(97),  '(a,b,c,d)',  'lp')
>>> I  = singular.ideal(['a+b+c+d',  'ab+ad+bc+cd',  'abc+abd+acd+bcd',  'abcd-1'])
>>> R
polynomial ring, over a field, global ordering
// coefficients: ZZ/97...
// number of vars : 4
//        block   1 : ordering lp
//                  : names    a b c d
//        block   2 : ordering C
>>> I
a+b+c+d,
a*b+a*d+b*c+c*d,
a*b*c+a*b*d+a*c*d+b*c*d,
a*b*c*d-1
```

下面是另一个使用 GAP 的方法：

```
sage: R  = libgap.PolynomialRing(GF(97),  4); R
GF(97)[x_1,x_2,x_3,x_4]
sage: I  = R.IndeterminatesOfPolynomialRing(); I
[ x_1, x_2, x_3, x_4 ]
sage: x1, x2, x3, x4  = I
sage: f  = x1*x2  + x3; f
x_1*x_2+x_3
sage: f.Value(I,[1,1,1,1])
Z(97)^34
```

```
>>>  from  sage.all  import  *
>>> R  = libgap.PolynomialRing(GF(Integer(97)), Integer(4)); R
GF(97)[x_1,x_2,x_3,x_4]
>>> I  = R.IndeterminatesOfPolynomialRing(); I
[ x_1, x_2, x_3, x_4 ]
>>> x1, x2, x3, x4  = I
>>> f  = x1*x2  + x3; f
x_1*x_2+x_3
>>> f.Value(I,[Integer(1),Integer(1),Integer(1),Integer(1)])
Z(97)^34
```

7.3 p-进数

如何在 Sage 中构建 p-进数? Sage 在这方面取得了很大的进展（参见 David Harvey 和 David Roe 在 SageDays 上的演讲）。这里只给出一些简单的示例。

要计算 `Qp` 中整数的 `Zp` 环的特征和剩余类域，请使用以下示例所示语法。

```
sage: K  = Qp(3)
sage: K.residue_class_field()
Finite Field of size 3
sage: K.residue_characteristic()
3
sage: a  = K(1); a
1 + O(3^20)
sage:  82*a
1 + 3^4 + O(3^20)
sage:  12*a
3 + 3^2 + O(3^21)
sage: a  in K
True
sage: b  =  82*a
sage: b^4
1 + 3^4 + 3^5 + 2*3^9 + 3^12 + 3^13 + 3^16 + O(3^20)
```

```
>>>  from  sage.all  import  *
>>> K  = Qp(Integer(3))
>>> K.residue_class_field()
Finite Field of size 3
>>> K.residue_characteristic()
3
>>> a  = K(Integer(1)); a
1 + O(3^20)
>>> Integer(82)*a
1 + 3^4 + O(3^20)
```

(续下页)

(接上页)

```
>>> Integer(12)*a
3 + 3^2 + O(3^21)
>>> a  in K
True
>>> b  = Integer(82)*a
>>> b**Integer(4)
1 + 3^4 + 3^5 + 2*3^9 + 3^12 + 3^13 + 3^16 + O(3^20)
```

7.4 多项式的商环

如何在 Sage 中构建商环?

我们创建商环 $GF(97)[x]/(x^3+7)$，并展示许多基本函数。

```
sage: R  = PolynomialRing(GF(97),'x')
sage: x  = R.gen()
sage: S  = R.quotient(x^3  +  7,  'a')
sage: a  = S.gen()
sage: S
Univariate Quotient Polynomial Ring in a over Finite Field of size 97 with
modulus x^3 + 7
sage: S.is_field()
True
sage: a  in S
True
sage: x  in S
True
sage: S.polynomial_ring()
Univariate Polynomial Ring in x over Finite Field of size 97
sage: S.modulus()
x^3 + 7
sage: S.degree()
3
```

```
>>>  from  sage.all  import  *
>>> R  = PolynomialRing(GF(Integer(97)),'x')
>>> x  = R.gen()
>>> S  = R.quotient(x**Integer(3)  + Integer(7),  'a')
>>> a  = S.gen()
>>> S
Univariate Quotient Polynomial Ring in a over Finite Field of size 97 with
modulus x^3 + 7
>>> S.is_field()
True
>>> a  in S
True
>>> x  in S
True
>>> S.polynomial_ring()
Univariate Polynomial Ring in x over Finite Field of size 97
>>> S.modulus()
x^3 + 7
>>> S.degree()
3
```

在 Sage 中，`in` 表示存在对该环的“标准强制转换”。因此整数 x 和 a 都在 S 中，虽然 x 实际上需要被强制

转换。

你还可以在商环中进行计算，而无需实际计算，然后使用 quo_rem 命令，如下所示：

```
sage: R  = PolynomialRing(GF(97),'x')
sage: x  = R.gen()
sage: f  = x^7+1
sage: (f^3).quo_rem(x^7-1)
(x^14 + 4*x^7 + 7, 8)
```

```
>>> from sage.all import *
>>> R  = PolynomialRing(GF(Integer(97)),'x')
>>> x  = R.gen()
>>> f  = x**Integer(7)+Integer(1)
>>> (f**Integer(3)).quo_rem(x**Integer(7)-Integer(1))
(x^14 + 4*x^7 + 7, 8)
```

CHAPTER 8

多项式

8.1 多项式幂

如何在 Sage 中计算模多项式幂?

要计算 $GF(97)[x]$ 中的 $x^{2006} \pmod{x^3+7}$，我们需要创建商环 $GF(97)[x]/(x^3+7)$，并在其中计算 x^{2006}。作为 Sage 符号的一个重要细节，我们必须区分 $GF(97)[x]$ 中的 x 和商环 $GF(97)[x]/(x^3+7)$ 中的的对应元素（我们用 a 来表示）。

```
sage: R = PolynomialRing(GF(97),'x')
sage: x = R.gen()
sage: S = R.quotient(x^3 + 7, 'a')
sage: a = S.gen()
sage: S
Univariate Quotient Polynomial Ring in a over
Finite Field of size 97 with modulus x^3 + 7
sage: a^2006
4*a^2
```

```
>>> from sage.all import *
>>> R = PolynomialRing(GF(Integer(97)),'x')
>>> x = R.gen()
>>> S = R.quotient(x**Integer(3) + Integer(7), 'a')
>>> a = S.gen()
>>> S
Univariate Quotient Polynomial Ring in a over
Finite Field of size 97 with modulus x^3 + 7
>>> a**Integer(2006)
4*a^2
```

另一种计算方法是：

```
sage: R = PolynomialRing(GF(97),'x')
sage: x = R.gen()
```

(续下页)

(接上页)

```
sage: S  = R.quotient(x^3  +  7,  'a')
sage: a  = S.gen()
sage: a^20062006
80*a
sage: libgap.eval("R:= PolynomialRing( GF(97))")
GF(97)[x_1]
sage: libgap.eval("i:= IndeterminatesOfPolynomialRing(R)")
[ x_1 ]
sage: libgap.eval("x:= i[1]"); libgap.eval("f:= x;")
x_1
x_1
sage: libgap.eval("PowerMod( R, x, 20062006, x^3+7 );")
Z(97)^41*x_1
sage: libgap.eval("PowerMod( R, x, 20062006, x^3+7 );")
Z(97)^41*x_1
sage: libgap.eval("PowerMod( R, x, 2006200620062006, x^3+7 );")
Z(97)^4*x_1^2
sage: a^2006200620062006
43*a^2
sage: libgap.eval("PowerMod( R, x, 2006200620062006, x^3+7 );")
Z(97)^4*x_1^2
sage: libgap.eval("Int(Z(97)^4)")
43
```

```
>>>  from  sage.all  import  *
>>> R  = PolynomialRing(GF(Integer(97)),'x')
>>> x  = R.gen()
>>> S  = R.quotient(x**Integer(3)  + Integer(7),  'a')
>>> a  = S.gen()
>>> a**Integer(20062006)
80*a
>>> libgap.eval("R:= PolynomialRing( GF(97))")
GF(97)[x_1]
>>> libgap.eval("i:= IndeterminatesOfPolynomialRing(R)")
[ x_1 ]
>>> libgap.eval("x:= i[1]"); libgap.eval("f:= x;")
x_1
x_1
>>> libgap.eval("PowerMod( R, x, 20062006, x^3+7 );")
Z(97)^41*x_1
>>> libgap.eval("PowerMod( R, x, 20062006, x^3+7 );")
Z(97)^41*x_1
>>> libgap.eval("PowerMod( R, x, 2006200620062006, x^3+7 );")
Z(97)^4*x_1^2
>>> a**Integer(2006200620062006)
43*a^2
>>> libgap.eval("PowerMod( R, x, 2006200620062006, x^3+7 );")
Z(97)^4*x_1^2
>>> libgap.eval("Int(Z(97)^4)")
43
```

8.2 因式分解

你可以使用 Sage 对多项式进行因式分解。

在 Sage 中对一元多项式进行因式分解，只需将方法 `factor` 应用到 PolynomialRingElement 对象 f 上。实际上，该方法会调用 Pari，因此计算相当快。

```
sage: x = PolynomialRing(RationalField(), 'x').gen()
sage: f = (x^3 - 1)^2-(x^2-1)^2
sage: f.factor()
(x - 1)^2 * x^2 * (x^2 + 2*x + 2)
```

```
>>> from sage.all import *
>>> x = PolynomialRing(RationalField(), 'x').gen()
>>> f = (x**Integer(3) - Integer(1))**Integer(2)-(x**Integer(2)-Integer(1))**Integer(2)
>>> f.factor()
(x - 1)^2 * x^2 * (x^2 + 2*x + 2)
```

使用 Singular 接口，Sage 还可以对多元多项式进行因式分解。

```
sage: x, y = PolynomialRing(RationalField(), 2, ['x','y']).gens()
sage: f = (9*y^6 - 9*x^2*y^5 - 18*x^3*y^4 - 9*x^5*y^4 + 9*x^6*y^2 + 9*x^7*y^3
....:       + 18*x^8*y^2 - 9*x^11)
sage: f.factor()
(9) * (-x^5 + y^2) * (x^6 - 2*x^3*y^2 - x^2*y^3 + y^4)
```

```
>>> from sage.all import *
>>> x, y = PolynomialRing(RationalField(), Integer(2), ['x','y']).gens()
>>> f = (Integer(9)*y**Integer(6) - Integer(9)*x**Integer(2)*y**Integer(5) -␣
↪Integer(18)*x**Integer(3)*y**Integer(4) - Integer(9)*x**Integer(5)*y**Integer(4) +␣
↪Integer(9)*x**Integer(6)*y**Integer(2) + Integer(9)*x**Integer(7)*y**Integer(3)
...       + Integer(18)*x**Integer(8)*y**Integer(2) - Integer(9)*x**Integer(11))
>>> f.factor()
(9) * (-x^5 + y^2) * (x^6 - 2*x^3*y^2 - x^2*y^3 + y^4)
```

8.3 多项式最大公约数

下面这个例子展示了一元多项式最大公约数：

```
sage: x = PolynomialRing(RationalField(), 'x').gen()
sage: f = 3*x^3 + x
sage: g = 9*x*(x+1)
sage: f.gcd(g)
x
```

```
>>> from sage.all import *
>>> x = PolynomialRing(RationalField(), 'x').gen()
>>> f = Integer(3)*x**Integer(3) + x
>>> g = Integer(9)*x*(x+Integer(1))
>>> f.gcd(g)
x
```

下面这个例子展示了多元多项式最大公约数：

```
sage: R.<x,y,z>  = PolynomialRing(RationalField(), order='lex')
sage: f  =  3*x^2*(x+y)
sage: g  =  9*x*(y^2  - x^2)
sage: f.gcd(g)
x^2 + x*y
```

```
>>>  from  sage.all  import  *
>>> R  = PolynomialRing(RationalField(), order='lex', names=('x',  'y',  'z',)); (x, y, z,)  =␣
↪R._first_ngens(3)
>>> f  = Integer(3)*x**Integer(2)*(x+y)
>>> g  = Integer(9)*x*(y**Integer(2)  - x**Integer(2))
>>> f.gcd(g)
x^2 + x*y
```

下面是另一种方法：

```
sage: R2  = singular.ring(0,  '(x,y,z)',  'lp')
sage: a  = singular.new('3x2*(x+y)')
sage: b  = singular.new('9x*(y2-x2)')
sage: g  = a.gcd(b)
sage: g
x^2+x*y
```

```
>>>  from  sage.all  import  *
>>> R2  = singular.ring(Integer(0),  '(x,y,z)',  'lp')
>>> a  = singular.new('3x2*(x+y)')
>>> b  = singular.new('9x*(y2-x2)')
>>> g  = a.gcd(b)
>>> g
x^2+x*y
```

下面这个例子展示了通过 GAP 接口计算一元多项式最大公约数。

```
sage: R  = libgap.PolynomialRing(GF(2)); R
GF(2)[x_1]
sage: i  = R.IndeterminatesOfPolynomialRing(); i
[ x_1 ]
sage: x_1  = i[0]
sage: f  = (x_1^3  - x_1  +  1)*(x_1  + x_1^2); f
x_1^5+x_1^4+x_1^3+x_1
sage: g  = (x_1^3  - x_1  +  1)*(x_1  +  1); g
x_1^4+x_1^3+x_1^2+Z(2)^0
sage: f.Gcd(g)
x_1^4+x_1^3+x_1^2+Z(2)^0
```

```
>>>  from  sage.all  import  *
>>> R  = libgap.PolynomialRing(GF(Integer(2))); R
GF(2)[x_1]
>>> i  = R.IndeterminatesOfPolynomialRing(); i
[ x_1 ]
>>> x_1  = i[Integer(0)]
>>> f  = (x_1**Integer(3)  - x_1  + Integer(1))*(x_1  + x_1**Integer(2)); f
x_1^5+x_1^4+x_1^3+x_1
>>> g  = (x_1**Integer(3)  - x_1  + Integer(1))*(x_1  + Integer(1)); g
x_1^4+x_1^3+x_1^2+Z(2)^0
>>> f.Gcd(g)
x_1^4+x_1^3+x_1^2+Z(2)^0
```

当然，我们也可以在生成器上执行相同的计算，它使用 NTL 库（该库能够非常快速地处理有限域上的大规模多项式最大公约数计算）。

```
sage: x = PolynomialRing(GF(2), 'x').gen()
sage: f = (x^3 + x + 1)*(x + x^2); f
x^5 + x^4 + x^3 + x
sage: g = (x^3 + x + 1)*(x + 1)
sage: f.gcd(g)
x^4 + x^3 + x^2 + 1
```

```
>>> from sage.all import *
>>> x = PolynomialRing(GF(Integer(2)), 'x').gen()
>>> f = (x**Integer(3) + x + Integer(1))*(x + x**Integer(2)); f
x^5 + x^4 + x^3 + x
>>> g = (x**Integer(3) + x + Integer(1))*(x + Integer(1))
>>> f.gcd(g)
x^4 + x^3 + x^2 + 1
```

8.4 多项式的根

Sage 可以计算一元多项式的根。

```
sage: x = PolynomialRing(RationalField(), 'x').gen()
sage: f = x^3 - 1
sage: f.roots()
[(1, 1)]
sage: f = (x^3 - 1)^2
sage: f.roots()
[(1, 2)]
sage: x = PolynomialRing(CyclotomicField(3), 'x').gen()
sage: f = x^3 - 1
sage: f.roots()
[(1, 1), (zeta3, 1), (-zeta3 - 1, 1)]
```

```
>>> from sage.all import *
>>> x = PolynomialRing(RationalField(), 'x').gen()
>>> f = x**Integer(3) - Integer(1)
>>> f.roots()
[(1, 1)]
>>> f = (x**Integer(3) - Integer(1))**Integer(2)
>>> f.roots()
[(1, 2)]
>>> x = PolynomialRing(CyclotomicField(Integer(3)), 'x').gen()
>>> f = x**Integer(3) - Integer(1)
>>> f.roots()
[(1, 1), (zeta3, 1), (-zeta3 - 1, 1)]
```

第一个元素是根，第二个元素是它的重数。

在某些情况下，GAP 确实可以求解一元多项式的根，但 GAP 通常不会这样做（根必须生成有限域或循环域的子域）。然而，有一个名为 `RadiRoot` 的 GAP 包，必须将其安装到 GAP 中，因为它确实有助于为有理系数的多项式执行此操作（`radiroot` 本身需要安装其他包；请参阅其网页了解更多详情）。`Factors` 命令实际上有一个选项，允许你增加基域，以便因式分解实际返回根。更多详情，请参阅 GAP 参考手册第 64.10 节“多项式因式分解”中给出的示例。

8.5 多元函数求值

你可以像往常一样在 Sage 中通过代入点来计算多项式的值:

```
sage: x = PolynomialRing(RationalField(), 3, 'x').gens()
sage: f = x[0] + x[1] - 2*x[1]*x[2]
sage: f
-2*x1*x2 + x0 + x1
sage: f(1,2,0)
3
sage: f(1,2,5)
-17
```

```
>>> from sage.all import *
>>> x = PolynomialRing(RationalField(), Integer(3), 'x').gens()
>>> f = x[Integer(0)] + x[Integer(1)] - Integer(2)*x[Integer(1)]*x[Integer(2)]
>>> f
-2*x1*x2 + x0 + x1
>>> f(Integer(1),Integer(2),Integer(0))
3
>>> f(Integer(1),Integer(2),Integer(5))
-17
```

这也适用于有理函数:

```
sage: h = f /(x[1] + x[2])
sage: h
(-2*x1*x2 + x0 + x1)/(x1 + x2)
sage: h(1,2,3)
-9/5
```

```
>>> from sage.all import *
>>> h = f /(x[Integer(1)] + x[Integer(2)])
>>> h
(-2*x1*x2 + x0 + x1)/(x1 + x2)
>>> h(Integer(1),Integer(2),Integer(3))
-9/5
```

Sage 还可以进行符号操作:

```
sage: var('x,y,z')
(x, y, z)
sage: f = (x + 3*y + x^2*y)^3; f
(x^2*y + x + 3*y)^3
sage: f(x=1,y=2,z=3)
729
sage: f.expand()
x^6*y^3 + 3*x^5*y^2 + 9*x^4*y^3 + 3*x^4*y + 18*x^3*y^2 +
27*x^2*y^3 +
x^3 + 9*x^2*y + 27*x*y^2 + 27*y^3
sage: f(x = 5/z)
(3*y + 25*y/z^2 + 5/z)^3
sage: g = f.subs(x = 5/z); g
(3*y + 25*y/z^2 + 5/z)^3
sage: h = g.rational_simplify(); h
(27*y^3*z^6 + 135*y^2*z^5 + 225*(3*y^3 + y)*z^4 + 125*(18*y^2 + 1)*z^3 +
15625*y^3 + 9375*y^2*z + 1875*(3*y^3 + y)*z^2)/z^6
```

```
>>> from sage.all import *
>>> var('x,y,z')
(x, y, z)
>>> f = (x + Integer(3)*y + x**Integer(2)*y)**Integer(3); f
(x^2*y + x + 3*y)^3
>>> f(x=Integer(1),y=Integer(2),z=Integer(3))
729
>>> f.expand()
x^6*y^3 + 3*x^5*y^2 + 9*x^4*y^3 + 3*x^4*y + 18*x^3*y^2 +
27*x^2*y^3 +
x^3 + 9*x^2*y + 27*x*y^2 + 27*y^3
>>> f(x = Integer(5)/z)
(3*y + 25*y/z^2 + 5/z)^3
>>> g = f.subs(x = Integer(5)/z); g
(3*y + 25*y/z^2 + 5/z)^3
>>> h = g.rational_simplify(); h
(27*y^3*z^6 + 135*y^2*z^5 + 225*(3*y^3 + y)*z^4 + 125*(18*y^2 + 1)*z^3 +
15625*y^3 + 9375*y^2*z + 1875*(3*y^3 + y)*z^2)/z^6
```

8.6 多元多项式的根

在某些情况下，Sage（使用 Singular 接口）可以（假设解形成零维代数簇）使用 Gröbner 基求解多元多项式方程。以下是一个简单的示例：

```
sage: R = PolynomialRing(QQ, 2, 'ab', order='lp')
sage: a,b = R.gens()
sage: I = (a^2-b^2-3, a-2*b)*R
sage: B = I.groebner_basis(); B
[a - 2*b, b^2 - 1]
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, Integer(2), 'ab', order='lp')
>>> a,b = R.gens()
>>> I = (a**Integer(2)-b**Integer(2)-Integer(3), a-Integer(2)*b)*R
>>> B = I.groebner_basis(); B
[a - 2*b, b^2 - 1]
```

所以 $b = \pm 1$ 且 $a = 2b$。

8.7 Gröbner 基

此计算在后台使用 Singular 来计算 Gröbner 基。

```
sage: R = PolynomialRing(QQ, 4, 'abcd', order='lp')
sage: a,b,c,d = R.gens()
sage: I = (a+b+c+d, a*b+a*d+b*c+c*d, a*b*c+a*b*d+a*c*d+b*c*d, a*b*c*d-1)*R; I
Ideal (a + b + c + d, a*b + a*d + b*c + c*d, a*b*c + a*b*d + a*c*d + b*c*d,
a*b*c*d - 1) of Multivariate Polynomial Ring in a, b, c, d over Rational Field
sage: B = I.groebner_basis(); B
[a + b + c + d,
 b^2 + 2*b*d + d^2,
 b*c - b*d + c^2*d^4 + c*d - 2*d^2,
 b*d^4 - b + d^5 - d,
```

(续下页)

(接上页)

```
 c^3*d^2 + c^2*d^3 - c - d,
 c^2*d^6 - c^2*d^2 - d^4 + 1]
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, Integer(4), 'abcd', order='lp')
>>> a,b,c,d = R.gens()
>>> I = (a+b+c+d, a*b+a*d+b*c+c*d, a*b*c+a*b*d+a*c*d+b*c*d, a*b*c*d-Integer(1))*R; I
Ideal (a + b + c + d, a*b + a*d + b*c + c*d, a*b*c + a*b*d + a*c*d + b*c*d,
a*b*c*d - 1) of Multivariate Polynomial Ring in a, b, c, d over Rational Field
>>> B = I.groebner_basis(); B
[a + b + c + d,
 b^2 + 2*b*d + d^2,
 b*c - b*d + c^2*d^4 + c*d - 2*d^2,
 b*d^4 - b + d^5 - d,
 c^3*d^2 + c^2*d^3 - c - d,
 c^2*d^6 - c^2*d^2 - d^4 + 1]
```

你可以使用多个环，而不必像在 Singular 中那样来回切换。例如，

```
sage: a,b,c = QQ['a,b,c'].gens()
sage: X,Y = GF(7)['X,Y'].gens()
sage: I = ideal(a, b^2, b^3+c^3)
sage: J = ideal(X^10 + Y^10)

sage: I.minimal_associated_primes ()
[Ideal (c, b, a) of Multivariate Polynomial Ring in a, b, c over Rational Field]

sage: J.minimal_associated_primes ()      # slightly random output
[Ideal (Y^4 + 3*X*Y^3 + 4*X^2*Y^2 + 4*X^3*Y + X^4) of Multivariate Polynomial
Ring in X, Y over Finite Field of size 7,
 Ideal (Y^4 + 4*X*Y^3 + 4*X^2*Y^2 + 3*X^3*Y + X^4) of Multivariate Polynomial
Ring in X, Y over Finite Field of size 7,
 Ideal (Y^2 + X^2) of Multivariate Polynomial Ring in X, Y over Finite Field
of size 7]
```

```
>>> from sage.all import *
>>> a,b,c = QQ['a,b,c'].gens()
>>> X,Y = GF(Integer(7))['X,Y'].gens()
>>> I = ideal(a, b**Integer(2), b**Integer(3)+c**Integer(3))
>>> J = ideal(X**Integer(10) + Y**Integer(10))

>>> I.minimal_associated_primes ()
[Ideal (c, b, a) of Multivariate Polynomial Ring in a, b, c over Rational Field]

>>> J.minimal_associated_primes ()      # slightly random output
[Ideal (Y^4 + 3*X*Y^3 + 4*X^2*Y^2 + 4*X^3*Y + X^4) of Multivariate Polynomial
Ring in X, Y over Finite Field of size 7,
 Ideal (Y^4 + 4*X*Y^3 + 4*X^2*Y^2 + 3*X^3*Y + X^4) of Multivariate Polynomial
Ring in X, Y over Finite Field of size 7,
 Ideal (Y^2 + X^2) of Multivariate Polynomial Ring in X, Y over Finite Field
of size 7]
```

所有实际工作均由 Singular 完成。

Sage 还包括 `gfan`，它提供了计算 Gröbner 基的其他快速算法。更多详情，请参阅参考手册中的"Gröbner fans"部分。

CHAPTER 9

初等数论

9.1 计算模幂

如何在 Sage 中计算模幂?

要想在 Sage 中计算 $51^{2006} \pmod{97}$，请输入

```
sage: R  = Integers(97)
sage: a  = R(51)
sage: a^2006
12
```

```
>>> from sage.all import *
>>> R  = Integers(Integer(97))
>>> a  = R(Integer(51))
>>> a**Integer(2006)
12
```

除了 `R = Integers(97)`，你也可以输入 `R = IntegerModRing(97)`。另一种选择是使用 GMP 接口：

```
sage: 51.powermod(99203843984,97)
96
```

```
>>> from sage.all import *
>>> Integer(51).powermod(Integer(99203843984),Integer(97))
96
```

9.2 离散对数

要找到数 x 使得 $b^x \equiv a \pmod{m}$（$a \pmod{m}$ 的离散对数）可以使用 `log` 命令：

```
sage: r  = Integers(125)
sage: b  = r.multiplicative_generator()^3
sage: a  = b^17
sage: a.log(b)
17
```

```
>>> from sage.all import *
>>> r  = Integers(Integer(125))
>>> b  = r.multiplicative_generator()**Integer(3)
>>> a  = b**Integer(17)
>>> a.log(b)
17
```

这在有限域上也适用：

```
sage: FF  = FiniteField(16,"a")
sage: a  = FF.gen()
sage: c  = a^7
sage: c.log(a)
7
```

```
>>> from sage.all import *
>>> FF  = FiniteField(Integer(16),"a")
>>> a  = FF.gen()
>>> c  = a**Integer(7)
>>> c.log(a)
7
```

9.3 质数

如何在 Sage 中构造质数?

`Primes` 类可以进行质数检测：

```
sage:  2^(2^12)+1  in Primes()
False
sage:  11  in Primes()
True
```

```
>>> from sage.all import *
>>> Integer(2)**(Integer(2)**Integer(12))+Integer(1)  in Primes()
False
>>> Integer(11)  in Primes()
True
```

`next_prime` 的使用一目了然：

```
sage: next_prime(2005)
      2011
```

```
>>> from sage.all import *
>>> next_prime(Integer(2005))
      2011
```

Pari 命令 `primepi` 通过 `pari(x).primepi()` 命令使用。它返回 $\leq x$ 的质数数量，例如：

```
sage: pari(10).primepi()
      4
```

```
>>> from sage.all import *
>>> pari(Integer(10)).primepi()
      4
```

使用 primes_first_n 或 primes 可以检查到，实有 4 个小于等于 10 的质数：

```
sage: primes_first_n(5)
[2, 3, 5, 7, 11]
sage: list(primes(1, 10))
[2, 3, 5, 7]
```

```
>>> from sage.all import *
>>> primes_first_n(Integer(5))
[2, 3, 5, 7, 11]
>>> list(primes(Integer(1), Integer(10)))
[2, 3, 5, 7]
```

9.4 因子

如何在 Sage 中计算整数因子之和?

Sage 使用 divisors(n) 求 n 的因子列表，使用 number_of_divisors(n) 求 n 的因子数量，并使用 sigma(n, k) 求 n 的因子的 k 次幂之和（因此 number_of_divisors(n) 和 sigma(n,0) 等价）。

例如：

```
sage: divisors(28); sum(divisors(28)); 2*28
[1, 2, 4, 7, 14, 28]
56
56
sage: sigma(28,0); sigma(28,1); sigma(28,2)
6
56
1050
```

```
>>> from sage.all import *
>>> divisors(Integer(28)); sum(divisors(Integer(28))); Integer(2)*Integer(28)
[1, 2, 4, 7, 14, 28]
56
56
>>> sigma(Integer(28),Integer(0)); sigma(Integer(28),Integer(1)); sigma(Integer(28),Integer(2))
6
56
1050
```

9.5 二次剩余

尝试一下：

```
sage: Q  = quadratic_residues(23); Q
[0, 1, 2, 3, 4, 6, 8, 9, 12, 13, 16, 18]
sage: N  = [x  for x  in  range(22)  if kronecker(x,23)==-1]; N
[5, 7, 10, 11, 14, 15, 17, 19, 20, 21]
```

```
>>>  from  sage.all  import  *
>>> Q  = quadratic_residues(Integer(23)); Q
[0, 1, 2, 3, 4, 6, 8, 9, 12, 13, 16, 18]
>>> N  = [x  for x  in  range(Integer(22))  if kronecker(x,Integer(23))==-Integer(1)]; N
[5, 7, 10, 11, 14, 15, 17, 19, 20, 21]
```

Q 是模 23 的二次剩余集，N 是非剩余集。

下面是使用 `kronecker` 命令（也称为“勒让德符号”）构造上述集合的另一种方法：

```
sage: [x  for x  in  range(22)  if kronecker(x,23)==1]
[1, 2, 3, 4, 6, 8, 9, 12, 13, 16, 18]
sage: [x  for x  in  range(22)  if kronecker(x,23)==-1]
[5, 7, 10, 11, 14, 15, 17, 19, 20, 21]
```

```
>>>  from  sage.all  import  *
>>> [x  for x  in  range(Integer(22))  if kronecker(x,Integer(23))==Integer(1)]
[1, 2, 3, 4, 6, 8, 9, 12, 13, 16, 18]
>>> [x  for x  in  range(Integer(22))  if kronecker(x,Integer(23))==-Integer(1)]
[5, 7, 10, 11, 14, 15, 17, 19, 20, 21]
```

CHAPTER 10

模形式

SageMath 的计算专长之一是模形式（非常技术性的领域），它能做的远远超出这篇非常简短的介绍所给出的内容。

10.1 尖点形式

如何使用 Sage 计算尖点形式空间的维数?

要计算 Gamma 尖点形式空间的维数，请使用命令 `dimension_cusp_forms`。以下是教程中“模形式”章节的一个示例：

```
sage: from sage.modular.dims import dimension_cusp_forms
sage: dimension_cusp_forms(Gamma0(11),2)
1
sage: dimension_cusp_forms(Gamma0(1),12)
1
sage: dimension_cusp_forms(Gamma1(389),2)
6112
```

```
>>> from sage.all import *
>>> from sage.modular.dims import dimension_cusp_forms
>>> dimension_cusp_forms(Gamma0(Integer(11)),Integer(2))
1
>>> dimension_cusp_forms(Gamma0(Integer(1)),Integer(12))
1
>>> dimension_cusp_forms(Gamma1(Integer(389)),Integer(2))
6112
```

相关命令：`dimension_new__cusp_forms_gamma0`（用于新形式的维数）、`dimension_new__cusp_forms_gamma0`（用于模形式）以及 `dimension_eis`（用于艾森斯坦级数）。这些命令的语法类似 - 可以参阅参考手册中的示例。

10.2 陪集表示

算术商 H/Γ 的基本定义域的显式表示可以通过 Γ 在 $SL_2(\mathbf{Z})$ 上的陪集来确定。这些陪集是如何在 Sage 中计算的呢?

以下是计算 $SL_2(\mathbf{Z})/\Gamma_0(11)$ 的陪集表示的示例:

```
sage: G  = Gamma0(11); G
Congruence Subgroup Gamma0(11)
sage:  list(G.coset_reps())
[
[1 0]  [ 0 -1]  [1 0]  [ 0 -1]  [ 0 -1]  [ 0 -1]  [ 0 -1]  [ 0 -1]
[0 1], [ 1  0], [1 1], [ 1  2], [ 1  3], [ 1  4], [ 1  5], [ 1  6],

[ 0 -1]  [ 0 -1]  [ 0 -1]  [ 0 -1]
[ 1  7], [ 1  8], [ 1  9], [ 1 10]
]
```

```
>>>  from  sage.all  import  *
>>> G  = Gamma0(Integer(11)); G
Congruence Subgroup Gamma0(11)
>>>  list(G.coset_reps())
[
[1 0]  [ 0 -1]  [1 0]  [ 0 -1]  [ 0 -1]  [ 0 -1]  [ 0 -1]  [ 0 -1]
[0 1], [ 1  0], [1 1], [ 1  2], [ 1  3], [ 1  4], [ 1  5], [ 1  6],
<BLANKLINE>
[ 0 -1]  [ 0 -1]  [ 0 -1]  [ 0 -1]
[ 1  7], [ 1  8], [ 1  9], [ 1 10]
]
```

10.3 模符号和 Hecke 算子

接下来我们展示如何在级别为 1、权重为 12 的模符号空间上计算 Hecke 算子。

```
sage: M  = ModularSymbols(1,12)
sage: M.basis()
([X^8*Y^2,(0,0)], [X^9*Y,(0,0)], [X^10,(0,0)])
sage: t2  = M.T(2)
sage: f  = t2.charpoly('x'); f
x^3 - 2001*x^2 - 97776*x - 1180224
sage: factor(f)
(x - 2049) * (x + 24)^2
sage: M.T(11).charpoly('x').factor()
(x - 285311670612) * (x - 534612)^2
```

```
>>>  from  sage.all  import  *
>>> M  = ModularSymbols(Integer(1),Integer(12))
>>> M.basis()
([X^8*Y^2,(0,0)], [X^9*Y,(0,0)], [X^10,(0,0)])
>>> t2  = M.T(Integer(2))
>>> f  = t2.charpoly('x'); f
x^3 - 2001*x^2 - 97776*x - 1180224
>>> factor(f)
(x - 2049) * (x + 24)^2
>>> M.T(Integer(11)).charpoly('x').factor()
(x - 285311670612) * (x - 534612)^2
```

这里 t2 表示 **Q** 上对权重为 12 、符号为 0 且维度为 3 的 $\Gamma_0(1)$ 的完整模符号空间的 Hecke 算子 T_2。

```
sage: M  = ModularSymbols(Gamma1(6),3,sign=0)
sage: M
Modular Symbols space of dimension 4 for Gamma_1(6) of weight 3 with sign 0
over Rational Field
sage: M.basis()
([X,(0,5)], [X,(3,5)], [X,(4,5)], [X,(5,5)])
sage: M._compute_hecke_matrix_prime(2).charpoly()
x^4 - 17*x^2 + 16
sage: M.integral_structure()
Free module of degree 4 and rank 4 over Integer Ring
Echelon basis matrix:
[1 0 0 0]
[0 1 0 0]
[0 0 1 0]
[0 0 0 1]
```

```
>>> from sage.all import *
>>> M  = ModularSymbols(Gamma1(Integer(6)),Integer(3),sign=Integer(0))
>>> M
Modular Symbols space of dimension 4 for Gamma_1(6) of weight 3 with sign 0
over Rational Field
>>> M.basis()
([X,(0,5)], [X,(3,5)], [X,(4,5)], [X,(5,5)])
>>> M._compute_hecke_matrix_prime(Integer(2)).charpoly()
x^4 - 17*x^2 + 16
>>> M.integral_structure()
Free module of degree 4 and rank 4 over Integer Ring
Echelon basis matrix:
[1 0 0 0]
[0 1 0 0]
[0 0 1 0]
[0 0 0 1]
```

更多示例请参阅教程或参考手册中的模形式章节。

10.4 亏格公式 (Genus formulas)

Sage 可以计算 $X_0(N), X_1(N)$ 及相关曲线的亏格 (genus)。以下是一些语法示例：

```
sage: from sage.modular.dims import dimension_cusp_forms
sage: dimension_cusp_forms(Gamma0(22))
2
sage: dimension_cusp_forms(Gamma0(30))
3
sage: dimension_cusp_forms(Gamma1(30))
9
```

```
>>> from sage.all import *
>>> from sage.modular.dims import dimension_cusp_forms
>>> dimension_cusp_forms(Gamma0(Integer(22)))
2
>>> dimension_cusp_forms(Gamma0(Integer(30)))
3
```

(续下页)

(接上页)

```
>>> dimension_cusp_forms(Gamma1(Integer(30)))
9
```

请参阅计算模形式空间维数的代码（在 `sage/modular/dims.py` 中）或 Oesterlé 和 Cohen 的论文 {CO}，获取详细信息。

CHAPTER 11

椭圆曲线

11.1 导子

如何在 Sage 中计算椭圆曲线（在 **Q** 上）的导子?

在 Sage 中使用 `EllipticCurve` 命令定义椭圆曲线 E 后，导子就是与 E 关联的若干“方法”之一。以下是语法示例（来自教程第 2.4 节“模形式”）:

```
sage: E = EllipticCurve([1,2,3,4,5])
sage: E
Elliptic Curve defined by y^2 + x*y + 3*y = x^3 + 2*x^2 + 4*x + 5 over
Rational Field
sage: E.conductor()
10351
```

```
>>> from sage.all import *
>>> E = EllipticCurve([Integer(1),Integer(2),Integer(3),Integer(4),Integer(5)])
>>> E
Elliptic Curve defined by y^2 + x*y + 3*y = x^3 + 2*x^2 + 4*x + 5 over
Rational Field
>>> E.conductor()
10351
```

11.2 j-不变量

如何在 Sage 中计算椭圆曲线的 j-不变量?

与 `EllipticCurve` 类相关的其他方法包括 `j_invariant`, `discriminant` 和 `weierstrass_model`。以下是语法示例:

```
sage: E = EllipticCurve([0, -1, 1, -10, -20])
sage: E
Elliptic Curve defined by y^2 + y = x^3 - x^2 - 10*x - 20 over Rational Field
```

(续下页)

(接上页)

```
sage: E.j_invariant()
-122023936/161051
sage: E.short_weierstrass_model()
Elliptic Curve defined by y^2  = x^3 - 13392*x - 1080432 over Rational Field
sage: E.discriminant()
-161051
sage: E  = EllipticCurve(GF(5),[0,  -1,  1,  -10,  -20])
sage: E.short_weierstrass_model()
Elliptic Curve defined by y^2  = x^3 + 3*x + 3 over Finite Field of size 5
sage: E.j_invariant()
4
```

```
>>>  from  sage.all  import  *
>>> E  = EllipticCurve([Integer(0),  -Integer(1), Integer(1),  -Integer(10),  -Integer(20)])
>>> E
Elliptic Curve defined by y^2 + y = x^3 - x^2 - 10*x - 20 over Rational Field
>>> E.j_invariant()
-122023936/161051
>>> E.short_weierstrass_model()
Elliptic Curve defined by y^2  = x^3 - 13392*x - 1080432 over Rational Field
>>> E.discriminant()
-161051
>>> E  = EllipticCurve(GF(Integer(5)),[Integer(0),  -Integer(1), Integer(1),  -Integer(10),  -
↪Integer(20)])
>>> E.short_weierstrass_model()
Elliptic Curve defined by y^2  = x^3 + 3*x + 3 over Finite Field of size 5
>>> E.j_invariant()
4
```

11.3 E 上的 $GF(q)$-有理点

如何计算有限域上椭圆曲线的点数?

给定一个定义在 $\mathbb{F} = GF(q)$ 上的椭圆曲线，Sage 可以计算其 $\mathbb{F}$-有理点集。

```
sage: E  = EllipticCurve(GF(5),[0,  -1,  1,  -10,  -20])
sage: E
Elliptic Curve defined by y^2 + y = x^3 + 4*x^2 over Finite Field of size 5
sage: E.points()
[(0 : 1 : 0), (0 : 0 : 1), (0 : 4 : 1), (1 : 0 : 1), (1 : 4 : 1)]
sage: E.cardinality()
5
sage: G  = E.abelian_group()
sage: G
Additive abelian group isomorphic to Z/5 embedded in Abelian group of points on Elliptic Curve␣
↪defined by y^2 + y = x^3 + 4*x^2 over Finite Field of size 5
sage: G.permutation_group()
Permutation Group with generators [(1,2,3,4,5)]
```

```
>>>  from  sage.all  import  *
>>> E  = EllipticCurve(GF(Integer(5)),[Integer(0),  -Integer(1), Integer(1),  -Integer(10),  -
↪Integer(20)])
>>> E
Elliptic Curve defined by y^2 + y = x^3 + 4*x^2 over Finite Field of size 5
>>> E.points()
```

(续下页)

(接上页)

```
[(0 : 1 : 0), (0 : 0 : 1), (0 : 4 : 1), (1 : 0 : 1), (1 : 4 : 1)]
>>> E.cardinality()
5
>>> G  = E.abelian_group()
>>> G
Additive abelian group isomorphic to Z/5 embedded in Abelian group of points on Elliptic Curve␣
↪defined by y^2 + y = x^3 + 4*x^2 over Finite Field of size 5
>>> G.permutation_group()
Permutation Group with generators [(1,2,3,4,5)]
```

11.4 与 Q 上椭圆曲线相关的模形式

设 E 是一个“良好”的椭圆曲线，其方程具有整数系数。设 N 为 E 的导子，并且对于每个 n，设 a_n 是出现在 E 的 Hasse-Weil L-函数中的数字。Taniyama-Shimura 猜想（已被 Wiles 证明）表明存在一个权重为 2、级别为 N 的模形式，它是 Hecke 算子下的特征形式，并具有傅里叶级数 $\sum_{n=0}^{\infty} a_n q^n$。Sage 可以计算与 E 相关的序列 a_n。以下是一个示例。

```
sage: E  = EllipticCurve([0,  -1,  1,  -10,  -20])
sage: E
Elliptic Curve defined by y^2 + y = x^3 - x^2 - 10*x - 20 over Rational Field
sage: E.conductor()
11
sage: E.anlist(20)
[0, 1, -2, -1, 2, 1, 2, -2, 0, -2, -2, 1, -2, 4, 4, -1, -4, -2, 4, 0, 2]
sage: E.analytic_rank()
0
```

```
>>> from sage.all import *
>>> E  = EllipticCurve([Integer(0),  -Integer(1), Integer(1),  -Integer(10),  -Integer(20)])
>>> E
Elliptic Curve defined by y^2 + y = x^3 - x^2 - 10*x - 20 over Rational Field
>>> E.conductor()
11
>>> E.anlist(Integer(20))
[0, 1, -2, -1, 2, 1, 2, -2, 0, -2, -2, 1, -2, 4, 4, -1, -4, -2, 4, 0, 2]
>>> E.analytic_rank()
0
```

CHAPTER 12

数域

12.1 分歧 (Ramification)

如何在 Sage 中计算具有给定判别式和分歧的数域?

Sage 可以访问 Jones 数域数据库，该数据库包含有界分歧且度数不超过 6 的数域。该数据库必须单独安装 (`database_jones_numfield`)。

首先加载数据库:

```
sage: J  = JonesDatabase()              # optional - database
sage: J                                 # optional - database
John Jones's table of number fields with bounded ramification and degree <= 6
```

```
>>> from sage.all import *
>>> J  = JonesDatabase()              # optional - database
>>> J                                 # optional - database
John Jones's table of number fields with bounded ramification and degree <= 6
```

列出数据库中所有分歧最多为 2 的数域的度数和判别式:

```
sage: [(k.degree(), k.disc()) for k in J.unramified_outside([2])]  # optional - database
[(4, -2048), (2, 8), (4, -1024), (1, 1), (4, 256), (2, -4), (4, 2048), (4, 512), (4, 2048), (2,␣
↪-8), (4, 2048)]
```

```
>>> from sage.all import *
>>> [(k.degree(), k.disc()) for k in J.unramified_outside([Integer(2)])]  # optional -␣
↪database
[(4, -2048), (2, 8), (4, -1024), (1, 1), (4, 256), (2, -4), (4, 2048), (4, 512), (4, 2048), (2,␣
↪-8), (4, 2048)]
```

列出在 2 之外无分歧且度数恰好为 2 的域的判别式:

```
sage: [k.disc() for k in J.unramified_outside([2],2)]  # optional - database
[8, -4, -8]
```

```
>>> from sage.all import *
>>> [k.disc() for k in J.unramified_outside([Integer(2)],Integer(2))] # optional - database
[8, -4, -8]
```

列出数据库中在 3 和 5 处有分歧的立方域的判别式:

```
sage: [k.disc() for k in J.ramified_at([3,5],3)] # optional - database
[-6075, -6075, -675, -135]
sage: factor(6075)
3^5 * 5^2
sage: factor(675)
3^3 * 5^2
sage: factor(135)
3^3 * 5
```

```
>>> from sage.all import *
>>> [k.disc() for k in J.ramified_at([Integer(3),Integer(5)],Integer(3))] # optional -␣
↪database
[-6075, -6075, -675, -135]
>>> factor(Integer(6075))
3^5 * 5^2
>>> factor(Integer(675))
3^3 * 5^2
>>> factor(Integer(135))
3^3 * 5
```

列出所有在 101 处有分歧的域:

```
sage: J.ramified_at(101)                    # optional - database
[Number Field in a with defining polynomial x^2 - 101,
 Number Field in a with defining polynomial x^4 - x^3 + 13*x^2 - 19*x + 361,
 Number Field in a with defining polynomial x^5 - x^4 - 40*x^3 - 93*x^2 - 21*x + 17,
 Number Field in a with defining polynomial x^5 + x^4 - 6*x^3 - x^2 + 18*x + 4,
 Number Field in a with defining polynomial x^5 + 2*x^4 + 7*x^3 + 4*x^2 + 11*x - 6]
```

```
>>> from sage.all import *
>>> J.ramified_at(Integer(101))                    # optional - database
[Number Field in a with defining polynomial x^2 - 101,
 Number Field in a with defining polynomial x^4 - x^3 + 13*x^2 - 19*x + 361,
 Number Field in a with defining polynomial x^5 - x^4 - 40*x^3 - 93*x^2 - 21*x + 17,
 Number Field in a with defining polynomial x^5 + x^4 - 6*x^3 - x^2 + 18*x + 4,
 Number Field in a with defining polynomial x^5 + 2*x^4 + 7*x^3 + 4*x^2 + 11*x - 6]
```

12.2 类数 (Class numbers)

如何在 Sage 中计算数域的类数 (class number)?

`class_number` 是一个与 QuadraticField 对象相关的方法:

```
sage: K = QuadraticField(29, 'x')
sage: K.class_number()
1
sage: K = QuadraticField(65, 'x')
sage: K.class_number()
2
```

(续下页)

(接上页)

```
sage: K  = QuadraticField(-11,  'x')
sage: K.class_number()
1
sage: K  = QuadraticField(-15,  'x')
sage: K.class_number()
2
sage: K.class_group()
Class group of order 2 with structure C2 of Number Field in x with defining polynomial x^2 + 15␣
↪with x = 3.872983346207417?*I
sage: K  = QuadraticField(401,  'x')
sage: K.class_group()
Class group of order 5 with structure C5 of Number Field in x with defining polynomial x^2 -␣
↪401 with x = 20.02498439450079?
sage: K.class_number()
5
sage: K.discriminant()
401
sage: K  = QuadraticField(-479,  'x')
sage: K.class_group()
Class group of order 25 with structure C25 of Number Field in x with defining polynomial x^2 +␣
↪479 with x = 21.886068628239297*I
sage: K.class_number()
25
sage: K.pari_polynomial()
x^2 + 479
sage: K.degree()
2
```

```
>>>  from  sage.all  import  *
>>> K  = QuadraticField(Integer(29),  'x')
>>> K.class_number()
1
>>> K  = QuadraticField(Integer(65),  'x')
>>> K.class_number()
2
>>> K  = QuadraticField(-Integer(11),  'x')
>>> K.class_number()
1
>>> K  = QuadraticField(-Integer(15),  'x')
>>> K.class_number()
2
>>> K.class_group()
Class group of order 2 with structure C2 of Number Field in x with defining polynomial x^2 + 15␣
↪with x = 3.872983346207417?*I
>>> K  = QuadraticField(Integer(401),  'x')
>>> K.class_group()
Class group of order 5 with structure C5 of Number Field in x with defining polynomial x^2 -␣
↪401 with x = 20.02498439450079?
>>> K.class_number()
5
>>> K.discriminant()
401
>>> K  = QuadraticField(-Integer(479),  'x')
>>> K.class_group()
Class group of order 25 with structure C25 of Number Field in x with defining polynomial x^2 +␣
↪479 with x = 21.886068628239297*I
```

(续下页)

(接上页)

```
>>> K.class_number()
25
>>> K.pari_polynomial()
x^2 + 479
>>> K.degree()
2
```

下面是一个更为一般的数域类型的例子:

```
sage: x  = PolynomialRing(QQ,  'x').gen()
sage: K  = NumberField(x^5+10*x+1,  'a')
sage: K
Number Field in a with defining polynomial x^5 + 10*x + 1
sage: K.degree()
5
sage: K.pari_polynomial()
x^5 + 10*x + 1
sage: K.discriminant()
25603125
sage: K.class_group()
Class group of order 1 of Number Field in a with defining
polynomial x^5 + 10*x + 1
sage: K.class_number()
1
```

```
>>>  from  sage.all  import  *
>>> x  = PolynomialRing(QQ,  'x').gen()
>>> K  = NumberField(x**Integer(5)+Integer(10)*x+Integer(1),  'a')
>>> K
Number Field in a with defining polynomial x^5 + 10*x + 1
>>> K.degree()
5
>>> K.pari_polynomial()
x^5 + 10*x + 1
>>> K.discriminant()
25603125
>>> K.class_group()
Class group of order 1 of Number Field in a with defining
polynomial x^5 + 10*x + 1
>>> K.class_number()
1
```

- 另请参见 Math World 网站上的类数链接 http://mathworld.wolfram.com/ClassNumber.html 获取表格、公式和背景信息。
- 对于循环域，可以尝试:

```
sage: K  = CyclotomicField(19)
sage: K.class_number()      # long time
1
```

```
>>>  from  sage.all  import  *
>>> K  = CyclotomicField(Integer(19))
>>> K.class_number()      # long time
1
```

更多详情，请参见 `ring/number_field.py` 中的文档。

12.3 整基 (Integral basis)

如何在 Sage 中计算数域的整基?

Sage 可以计算数域的元素列表，该列表是该数域的整数全环的基。

```
sage: x = PolynomialRing(QQ, 'x').gen()
sage: K = NumberField(x^5+10*x+1, 'a')
sage: K.integral_basis()
[1, a, a^2, a^3, a^4]
```

```
>>> from sage.all import *
>>> x = PolynomialRing(QQ, 'x').gen()
>>> K = NumberField(x**Integer(5)+Integer(10)*x+Integer(1), 'a')
>>> K.integral_basis()
[1, a, a^2, a^3, a^4]
```

接下来我们计算立方域的整数环，其中 2 是“基本判别式因子”，因此整数环不是由单个元素生成的。

```
sage: x = PolynomialRing(QQ, 'x').gen()
sage: K = NumberField(x^3 + x^2 - 2*x + 8, 'a')
sage: K.integral_basis()
[1, 1/2*a^2 + 1/2*a, a^2]
```

```
>>> from sage.all import *
>>> x = PolynomialRing(QQ, 'x').gen()
>>> K = NumberField(x**Integer(3) + x**Integer(2) - Integer(2)*x + Integer(8), 'a')
>>> K.integral_basis()
[1, 1/2*a^2 + 1/2*a, a^2]
```

CHAPTER 13

代数几何

13.1 曲线上点的计数

如何在 Sage 中计算有限域上椭圆曲线上点的数量?

对于素有限域，有小步大步法 (baby step giant step method) 和 SEA (Schoof-Elkies-Atkin) 算法（由 Christophe Doche 和 Sylvain Duquesne 在 PARI 中实现）。以下是从参考手册中摘取的示例：

```
sage: E = EllipticCurve(GF(10007),[1,2,3,4,5])
sage: E.cardinality()
10076
```

```
>>> from sage.all import *
>>> E = EllipticCurve(GF(Integer(10007)),[Integer(1),Integer(2),Integer(3),Integer(4),
↪Integer(5)])
>>> E.cardinality()
10076
```

`E.points()` 命令将返回有理点的实际列表。

如何在有限域上计算平面曲线上有理点的数量? `rational_points` 命令使用简单枚举算法生成有理点。以下是语法示例：

```
sage: x,y,z = PolynomialRing(GF(5), 3, 'xyz').gens()
sage: C = Curve(y^2*z^7 - x^9 - x*z^8); C
Projective Plane Curve over Finite Field of size 5 defined by -x^9 + y^2*z^7 - x*z^8
sage: C.rational_points()
[(0 : 0 : 1), (0 : 1 : 0), (2 : 2 : 1), (2 : 3 : 1), (3 : 1 : 1), (3 : 4 : 1)]
sage: C.rational_points(algorithm="bn")
[(0 : 0 : 1), (0 : 1 : 0), (2 : 2 : 1), (2 : 3 : 1), (3 : 1 : 1), (3 : 4 : 1)]
```

```
>>> from sage.all import *
>>> x,y,z = PolynomialRing(GF(Integer(5)), Integer(3), 'xyz').gens()
>>> C = Curve(y**Integer(2)*z**Integer(7) - x**Integer(9) - x*z**Integer(8)); C
```

(续下页)

(接上页)

```
Projective Plane Curve over Finite Field of size 5 defined by -x^9 + y^2*z^7 - x*z^8
>>> C.rational_points()
[(0 : 0 : 1), (0 : 1 : 0), (2 : 2 : 1), (2 : 3 : 1), (3 : 1 : 1), (3 : 4 : 1)]
>>> C.rational_points(algorithm="bn")
[(0 : 0 : 1), (0 : 1 : 0), (2 : 2 : 1), (2 : 3 : 1), (3 : 1 : 1), (3 : 4 : 1)]
```

选项 `algorithm="bn"` 使用 Sage 的 Singular 接口并调用 `brnoeth` 包。

下面是将 Sage 的 `rational_points` 应用于 $GF(8)$ 上的 Klein 四次方程的另一个例子。

```
sage: x, y, z  = PolynomialRing(GF(8,'a'),  3,  'xyz').gens()
sage: f  = x^3*y+y^3*z+x*z^3
sage: C  = Curve(f); C
Projective Plane Curve over Finite Field in a of size 2^3 defined by x^3*y + y^3*z + x*z^3
sage: C.rational_points()
[(0 : 0 : 1),
 (0 : 1 : 0),
 (1 : 0 : 0),
 (1 : a : 1),
 (1 : a^2 : 1),
 (1 : a^2 + a : 1),
 (a : 1 : 1),
 (a : a^2 : 1),
 (a : a^2 + 1 : 1),
 (a + 1 : a + 1 : 1),
 (a + 1 : a^2 : 1),
 (a + 1 : a^2 + a + 1 : 1),
 (a^2 : 1 : 1),
 (a^2 : a^2 + a : 1),
 (a^2 : a^2 + a + 1 : 1),
 (a^2 + 1 : a + 1 : 1),
 (a^2 + 1 : a^2 + 1 : 1),
 (a^2 + 1 : a^2 + a : 1),
 (a^2 + a : 1 : 1),
 (a^2 + a : a : 1),
 (a^2 + a : a + 1 : 1),
 (a^2 + a + 1 : a : 1),
 (a^2 + a + 1 : a^2 + 1 : 1),
 (a^2 + a + 1 : a^2 + a + 1 : 1)]
```

```
>>>  from  sage.all  import  *
>>> x, y, z  = PolynomialRing(GF(Integer(8),'a'), Integer(3),  'xyz').gens()
>>> f  = x**Integer(3)*y+y**Integer(3)*z+x*z**Integer(3)
>>> C  = Curve(f); C
Projective Plane Curve over Finite Field in a of size 2^3 defined by x^3*y + y^3*z + x*z^3
>>> C.rational_points()
[(0 : 0 : 1),
 (0 : 1 : 0),
 (1 : 0 : 0),
 (1 : a : 1),
 (1 : a^2 : 1),
 (1 : a^2 + a : 1),
 (a : 1 : 1),
 (a : a^2 : 1),
 (a : a^2 + 1 : 1),
 (a + 1 : a + 1 : 1),
 (a + 1 : a^2 : 1),
```

(续下页)

(接上页)

```
(a + 1 : a^2 + a + 1 : 1),
(a^2 : 1 : 1),
(a^2 : a^2 + a : 1),
(a^2 : a^2 + a + 1 : 1),
(a^2 + 1 : a + 1 : 1),
(a^2 + 1 : a^2 + 1 : 1),
(a^2 + 1 : a^2 + a : 1),
(a^2 + a : 1 : 1),
(a^2 + a : a : 1),
(a^2 + a : a + 1 : 1),
(a^2 + a + 1 : a : 1),
(a^2 + a + 1 : a^2 + 1 : 1),
(a^2 + a + 1 : a^2 + a + 1 : 1)]
```

13.1.1 其他方法

- 对于平面曲线，你可以使用 Singular 的 `closed_points` 命令。输入是 2 变量环 $F[x, y]$ 中曲线 X 的消失理想 I。`closed_points` 命令返回一个素理想列表（每个都是 Gröbner 基），对应于 $V(I)$ 的（不同仿射闭合）点。以下是示例：

```
sage: singular_console()
                     SINGULAR                                /  Development
 A Computer Algebra System for Polynomial Computations   /   version 3-0-1
                                                       0<
     by: G.-M. Greuel, G. Pfister, H. Schoenemann        \   October 2005
FB Mathematik der Universitaet, D-67653 Kaiserslautern    \
// ** executing /home/wdj/sagefiles/sage-0.9.4/local/LIB/.singularrc
> LIB "brnoeth.lib";
> ring s = 2,(x,y),lp;
> ideal I = x4+x,y4+y;
> list L = closed_points(I);
> L;
[1]:
   _[1] = y
   _[2] = x
[2]:
   _[1] = y
   _[2] = x+1
[3]:
   _[1] = y
   _[2] = x2+x+1
[4]:
   _[1] = y+1
   _[2] = x
[5]:
   _[1] = y+1
   _[2] = x+1
[6]:
   _[1] = y+1
   _[2] = x2+x+1
[7]:
   _[1] = y2+y+1
   _[2] = x+1
[8]:
   _[1] = y2+y+1
   _[2] = x
```

(续下页)

(接上页)

```
[9]:
   _[1] = y2+y+1
   _[2] = x+y
[10]:
   _[1] = y2+y+1
   _[2] = x+y+1
> Auf Wiedersehen.
```

```
>>> from sage.all import *
>>> singular_console()
                     SINGULAR                                /  Development
 A Computer Algebra System for Polynomial Computations   /   version 3-0-1
                                                       0<
     by: G.-M. Greuel, G. Pfister, H. Schoenemann        \   October 2005
FB Mathematik der Universitaet, D-67653 Kaiserslautern    \
// ** executing /home/wdj/sagefiles/sage-0.9.4/local/LIB/.singularrc
> LIB "brnoeth.lib";
> ring s = 2,(x,y),lp;
> ideal I = x4+x,y4+y;
> list L = closed_points(I);
> L;
[1]:
   _[1] = y
   _[2] = x
[2]:
   _[1] = y
   _[2] = x+1
[3]:
   _[1] = y
   _[2] = x2+x+1
[4]:
   _[1] = y+1
   _[2] = x
[5]:
   _[1] = y+1
   _[2] = x+1
[6]:
   _[1] = y+1
   _[2] = x2+x+1
[7]:
   _[1] = y2+y+1
   _[2] = x+1
[8]:
   _[1] = y2+y+1
   _[2] = x
[9]:
   _[1] = y2+y+1
   _[2] = x+y
[10]:
   _[1] = y2+y+1
   _[2] = x+y+1
> Auf Wiedersehen.
```

```
sage: singular.lib("brnoeth.lib")
```

(续下页)

(接上页)

```
sage: s  = singular.ring(2,'(x,y)','lp')
sage: I  = singular.ideal('x^4+x',  'y^4+y')
sage: L  = singular.closed_points(I)
sage:  # Here you have all the points :
sage: L        # random
[1]:
   _[1]=y+1
   _[2]=x+1
...
sage: l=[L[k].sage()  for k  in [1..10]];  len(l)  # there are 10 points
10
sage: r=sorted(l[0].ring().gens()); r
[y, x]
sage: r  in [t.gens()  for t  in l]  #  one of them is given by [y,x]
True
```

```
>>>  from  sage.all  import  *
>>> singular.lib("brnoeth.lib")
>>> s  = singular.ring(Integer(2),'(x,y)','lp')
>>> I  = singular.ideal('x^4+x',  'y^4+y')
>>> L  = singular.closed_points(I)
>>>  # Here you have all the points :
>>> L        # random
[1]:
   _[1]=y+1
   _[2]=x+1
...
>>> l=[L[k].sage()  for k  in (ellipsis_range(Integer(1),Ellipsis,Integer(10)))];  len(l)
↪# there are 10 points
10
>>> r=sorted(l[Integer(0)].ring().gens()); r
[y, x]
>>> r  in [t.gens()  for t  in l]  #  one of them is given by [y,x]
True
```

- 另一种计算有理点的方法是使用 Singular 的 `NSplaces` 命令。以下是用该方法在 $GF(8)$ 上计算 Klein 四次方程的示例：

```
sage: singular.LIB("brnoeth.lib")
sage: s  = singular.ring(2,'(x,y)','lp')
sage: f  = singular.poly('x3y+y3+x')
sage: klein1  = f.Adj_div();  print(klein1)
[1]:
   [1]:
      // coefficients: ZZ/2...
// number of vars : 2
//        block   1 : ordering lp
//                  : names    x y
//        block   2 : ordering C
...
sage:  # define a curve X = {f = 0} over GF(2)
sage: klein2  = singular.NSplaces(3,klein1)
sage:  print(singular.eval('extcurve(3,%s)'%klein2.name()))
Total number of rational places : NrRatPl = 23
...
sage: klein3  = singular.extcurve(3, klein2)
```

```
>>>  from  sage.all  import  *
>>> singular.LIB("brnoeth.lib")
>>> s  = singular.ring(Integer(2),'(x,y)','lp')
>>> f  = singular.poly('x3y+y3+x')
>>> klein1  = f.Adj_div();  print(klein1)
[1]:
   [1]:
      // coefficients: ZZ/2...
// number of vars : 2
//        block   1 : ordering lp
//                  : names    x y
//        block   2 : ordering C
...
>>>  # define a curve X = {f = 0} over GF(2)
>>> klein2  = singular.NSplaces(Integer(3),klein1)
>>>  print(singular.eval('extcurve(3,%s)'%klein2.name()))
Total number of rational places : NrRatPl = 23
...
>>> klein3  = singular.extcurve(Integer(3), klein2)
```

上面我们在 Singular 中定义了一条 $GF(8)$ 上的曲线 $X = \{f = 0\}$。

```
sage:  print(klein1)
[1]:
   [1]:
      // coefficients: ZZ/2...
// number of vars : 2
//        block   1 : ordering lp
//                  : names    x y
//        block   2 : ordering C
   [2]:
      // coefficients: ZZ/2...
// number of vars : 3
//        block   1 : ordering lp
//                  : names    x y z
//        block   2 : ordering C
[2]:
   4,3
[3]:
   [1]:
      1,1
   [2]:
      1,2
[4]:
   0
[5]:
   [1]:
      [1]:
         // coefficients: ZZ/2...
// number of vars : 3
//        block   1 : ordering ls
//                  : names    x y t
//        block   2 : ordering C
      [2]:
         1,1
sage:  print(klein1[3])
[1]:
```

(续下页)

(接上页)

```
   1,1
[2]:
   1,2
```

```
>>> from sage.all import *
>>> print(klein1)
[1]:
   [1]:
      // coefficients: ZZ/2...
// number of vars : 2
//        block   1 : ordering lp
//                  : names    x y
//        block   2 : ordering C
   [2]:
      // coefficients: ZZ/2...
// number of vars : 3
//        block   1 : ordering lp
//                  : names    x y z
//        block   2 : ordering C
[2]:
   4,3
[3]:
   [1]:
      1,1
   [2]:
      1,2
[4]:
   0
[5]:
   [1]:
      [1]:
         // coefficients: ZZ/2...
// number of vars : 3
//        block   1 : ordering ls
//                  : names    x y t
//        block   2 : ordering C
      [2]:
         1,1
>>> print(klein1[Integer(3)])
[1]:
   1,1
[2]:
   1,2
```

对于度为 3 的地方:

```
sage: print(klein2[3])
[1]:
   1,1
[2]:
   1,2
[3]:
   3,1
[4]:
   3,2
[5]:
```

(续下页)

(接上页)

```
   3,3
[6]:
   3,4
[7]:
   3,5
[8]:
   3,6
[9]:
   3,7
```

```
>>> from sage.all import *
>>> print(klein2[Integer(3)])
[1]:
   1,1
[2]:
   1,2
[3]:
   3,1
[4]:
   3,2
[5]:
   3,3
[6]:
   3,4
[7]:
   3,5
[8]:
   3,6
[9]:
   3,7
```

下面的每个点都是成对的：（度，点索引号）。

```
sage: print(klein3[3])
[1]:
   1,1
[2]:
   1,2
[3]:
   3,1
[4]:
   3,2
[5]:
   3,3
[6]:
   3,4
[7]:
   3,5
[8]:
   3,6
[9]:
   3,7
```

```
>>> from sage.all import *
>>> print(klein3[Integer(3)])
[1]:
```

(续下页)

(接上页)

```
   1,1
[2]:
   1,2
[3]:
   3,1
[4]:
   3,2
[5]:
   3,3
[6]:
   3,4
[7]:
   3,5
[8]:
   3,6
[9]:
   3,7
```

实际获取 $X(GF(8))$ 的点：

```
sage: R  = klein3[1][5]
sage: R.set_ring()
sage: singular("POINTS;")
[1]:
   [1]:
      0
   [2]:
      1
   [3]:
      0
[2]:
   [1]:
      1
   [2]:
      0
   [3]:
      0
...
```

```
>>>  from  sage.all  import  *
>>> R  = klein3[Integer(1)][Integer(5)]
>>> R.set_ring()
>>> singular("POINTS;")
[1]:
   [1]:
      0
   [2]:
      1
   [3]:
      0
[2]:
   [1]:
      1
   [2]:
      0
   [3]:
```

(续下页)

(接上页)

```
      0
...
```

加上另外 21 个点（已省略）。总共有 23 个有理点。

13.2 使用 Singular 计算 Riemann-Roch 空间

为了计算域 F 上曲线上的因子 D 的 Riemann-Roch 空间基，可以使用 Sage 封装的 `riemann_roch_basis` 方法，它是 Singular 实现的 Brill Noether 算法。注意，这个封装当前仅在 F 是素数且因子 D 在有理点上受支持时才有效。下面是如何使用 `riemann_roch_basis` 的示例，以及如何使用 Singular 本身来帮助理解封装的工作方式。

- 使用 `riemann_roch_basis`:

```
sage: x, y, z  = PolynomialRing(GF(5),  3,  'xyz').gens()
sage: f  = x^7  + y^7  + z^7
sage: X  = Curve(f); pts  = X.rational_points()
sage: D  = X.divisor([ (3, pts[0]), (-1,pts[1]), (10, pts[5]) ])
sage: X.riemann_roch_basis(D)
[(-2*x + y)/(x + y), (-x + z)/(x + y)]
```

```
>>>  from  sage.all  import  *
>>> x, y, z  = PolynomialRing(GF(Integer(5)), Integer(3),  'xyz').gens()
>>> f  = x**Integer(7)  + y**Integer(7)  + z**Integer(7)
>>> X  = Curve(f); pts  = X.rational_points()
>>> D  = X.divisor([ (Integer(3), pts[Integer(0)]), (-Integer(1),pts[Integer(1)]),␣
↪(Integer(10), pts[Integer(5)]) ])
>>> X.riemann_roch_basis(D)
[(-2*x + y)/(x + y), (-x + z)/(x + y)]
```

- 使用 Singular 的 `BrillNoether` 命令（具体内容请参见 Singular 在线文档的 Brill-Noether 章节 https://www.singular.uni-kl.de/Manual/4-3-0/sing_2254.htm 和论文 {CF}）:

```
sage: singular.LIB('brnoeth.lib')
sage: _  = singular.ring(5,'(x,y)','lp')
sage:  print(singular.eval("list X = Adj_div(-x5+y2+x);"))
Computing affine singular points ...
Computing all points at infinity ...
Computing affine singular places ...
Computing singular places at infinity ...
Computing non-singular places at infinity ...
Adjunction divisor computed successfully

The genus of the curve is 2
sage:  print(singular.eval("X = NSplaces(1,X);"))
Computing non-singular affine places of degree 1 ...
sage:  print(singular("X[3];"))
[1]:
   1,1
[2]:
   1,2
[3]:
   1,3
[4]:
   1,4
```

(续下页)

(接上页)

```
[5]:
   1,5
[6]:
   1,6
```

```
>>>  from  sage.all  import  *
>>> singular.LIB('brnoeth.lib')
>>> _  = singular.ring(Integer(5),'(x,y)','lp')
>>>  print(singular.eval("list X = Adj_div(-x5+y2+x);"))
Computing affine singular points ...
Computing all points at infinity ...
Computing affine singular places ...
Computing singular places at infinity ...
Computing non-singular places at infinity ...
Adjunction divisor computed successfully
<BLANKLINE>
The genus of the curve is 2
>>>  print(singular.eval("X = NSplaces(1,X);"))
Computing non-singular affine places of degree 1 ...
>>>  print(singular("X[3];"))
[1]:
   1,1
[2]:
   1,2
[3]:
   1,3
[4]:
   1,4
[5]:
   1,5
[6]:
   1,6
```

上述列表中，每个整数对中的第一个整数表示点的度数 d。第二个整数是该点在环 X[5][d][1] 的 POINTS 列表中的索引。注意，每次运行算法时，这个列表的顺序都不相同，例如上面列表中的 1, 1 每次可能指示不同的有理点。通过定义一个与 X[3] 长度相同的整数列表 G，可以指定一个因子。如果 X[3] 的第 k 项为 d, i，则 G 的第 k 项表示该因子在环 X[5][d][1] 的 POINTS 列表中第 i 个点上的重数。接下来，我们定义一个度为 12 的“随机”因子并计算其 Riemann-Roch 空间基：

```
sage: singular.eval("intvec G = 4,4,4,0,0,0;")
''
sage: singular.eval("def R = X[1][2];")
''
sage: singular.eval("setring R;")
''
sage:  print(singular.eval("list LG = BrillNoether(G,X);"))
Forms of degree 6 :
28

Vector basis successfully computed
```

```
>>>  from  sage.all  import  *
>>> singular.eval("intvec G = 4,4,4,0,0,0;")
''
>>> singular.eval("def R = X[1][2];")
```

(续下页)

(接上页)

```
''
>>> singular.eval("setring R;")
''
>>>  print(singular.eval("list LG = BrillNoether(G,X);"))
Forms of degree 6 :
28
<BLANKLINE>
Vector basis successfully computed
<BLANKLINE>
```

13.2.1 AG 码

Sage 可以通过调用 Singular 的 BrillNoether 算法计算 Riemann-Roch 空间 $L(D) = L_X(D)$ 的基，从而计算 AG 码 $C = C_X(D, E)$。除了曲线 X 和因子 D，还必须指定求值因子 E。

请注意，自从 `riemann_roch_basis` 封装被修复后，本节尚未更新。请参阅上文中，了解如何正确定义 Singular 的 `BrillNoether` 命令的因子。

这里有一个示例，计算相关 AG 码的生成矩阵。这次我们使用 Singular 的 `AGCode_L` 命令：

```
sage: singular.LIB('brnoeth.lib')
sage: singular.eval("ring s = 2,(x,y),lp;")
''
sage:  print(singular.eval("list HC = Adj_div(x3+y2+y);"))
Computing affine singular points ...
Computing all points at infinity ...
Computing affine singular places ...
Computing singular places at infinity ...
Computing non-singular places at infinity ...
Adjunction divisor computed successfully

The genus of the curve is 1
sage:  print(singular.eval("list HC1 = NSplaces(1..2,HC);"))
Computing non-singular affine places of degree 1 ...
Computing non-singular affine places of degree 2 ...
sage:  print(singular.eval("HC = extcurve(2,HC1);"))
Total number of rational places : NrRatPl = 9
```

```
>>>  from  sage.all  import  *
>>> singular.LIB('brnoeth.lib')
>>> singular.eval("ring s = 2,(x,y),lp;")
''
>>>  print(singular.eval("list HC = Adj_div(x3+y2+y);"))
Computing affine singular points ...
Computing all points at infinity ...
Computing affine singular places ...
Computing singular places at infinity ...
Computing non-singular places at infinity ...
Adjunction divisor computed successfully
<BLANKLINE>
The genus of the curve is 1
>>>  print(singular.eval("list HC1 = NSplaces(1..2,HC);"))
Computing non-singular affine places of degree 1 ...
Computing non-singular affine places of degree 2 ...
>>>  print(singular.eval("HC = extcurve(2,HC1);"))
Total number of rational places : NrRatPl = 9
```

我们将以下内容设置为 junk 以丢弃输出:

```
sage: junk  = singular.eval("intvec G = 5;")       # the rational divisor G = 5*HC[3][1]
sage: junk  = singular.eval("def R = HC[1][2];")
sage: singular.eval("setring R;")
''
```

```
>>> from sage.all import *
>>> junk  = singular.eval("intvec G = 5;")       # the rational divisor G = 5*HC[3][1]
>>> junk  = singular.eval("def R = HC[1][2];")
>>> singular.eval("setring R;")
''
```

向量 G 表示因子“无穷远点的 5 倍”。

接下来，我们计算 Riemann-Roch 空间。

```
sage:  print(singular.eval("BrillNoether(G,HC);"))
Forms of degree 3 :
10

Vector basis successfully computed

[1]:
   _[1]=x
   _[2]=z
[2]:
   _[1]=y
   _[2]=z
[3]:
   _[1]=1
   _[2]=1
[4]:
   _[1]=y2+yz
   _[2]=xz
[5]:
   _[1]=y3+y2z
   _[2]=x2z
```

```
>>> from sage.all import *
>>> print(singular.eval("BrillNoether(G,HC);"))
Forms of degree 3 :
10
<BLANKLINE>
Vector basis successfully computed
<BLANKLINE>
[1]:
   _[1]=x
   _[2]=z
[2]:
   _[1]=y
   _[2]=z
[3]:
   _[1]=1
   _[2]=1
[4]:
   _[1]=y2+yz
   _[2]=xz
```

(续下页)

(接上页)

```
[5]:
   _[1]=y3+y2z
   _[2]=x2z
```

这是 Riemann-Roch 空间的基，其中每个函数对表示商（第一个函数除以第二个函数）。每一个基元素都会在特定点进行求值以构建码的生成矩阵。接下来我们构建这些点。

```
sage: singular.eval("def R = HC[1][5];")
'// ** redefining R **'
sage: singular.eval("setring R;")
''
sage:  print(singular.eval("POINTS;"))
[1]:
   [1]:
      0
   [2]:
      1
   [3]:
      0
[2]:
   [1]:
      0
   [2]:
      1
   [3]:
      1
[3]:
   [1]:
      0
   [2]:
      0
   [3]:
      1
[4]:
   [1]:
      (a+1)
   [2]:
      (a)
   [3]:
      1
...
```

```
>>>  from  sage.all  import  *
>>> singular.eval("def R = HC[1][5];")
'// ** redefining R **'
>>> singular.eval("setring R;")
''
>>>  print(singular.eval("POINTS;"))
[1]:
   [1]:
      0
   [2]:
      1
   [3]:
      0
[2]:
```

(续下页)

(接上页)

```
   [1]:
      0
   [2]:
      1
   [3]:
      1
[3]:
   [1]:
      0
   [2]:
      0
   [3]:
      1
[4]:
   [1]:
      (a+1)
   [2]:
      (a)
   [3]:
      1
...
```

再加上 5 个，曲线上总共有 9 个有理点。我们使用这些点的子集（除第一个点外）定义我们的 “求值因子” D:

```
sage: singular.eval("def ER = HC[1][4];")
''
sage: singular.eval("setring ER;")
''
sage:  # D = sum of the rational places no. 2..9 over F_4
sage: singular.eval("intvec D = 2..9;")
''
sage:  # let us construct the corresponding evaluation AG code :
sage:  print(singular.eval("matrix C = AGcode_L(G,D,HC);"))
Forms of degree 3 :
10

Vector basis successfully computed

sage:  # here is a linear code of type [8,5,> = 3] over F_4
sage:  print(singular.eval("print(C);"))
0,0,(a+1),(a),  1,  1,    (a),  (a+1),
1,0,(a),  (a+1),(a),(a+1),(a),  (a+1),
1,1,1,    1,    1,  1,    1,    1,
0,0,(a),  (a+1),1,  1,    (a+1),(a),
0,0,1,    1,    (a),(a+1),(a+1),(a)
```

```
>>>  from  sage.all  import  *
>>> singular.eval("def ER = HC[1][4];")
''
>>> singular.eval("setring ER;")
''
>>>  # D = sum of the rational places no. 2..9 over F_4
>>> singular.eval("intvec D = 2..9;")
''
>>>  # let us construct the corresponding evaluation AG code :
>>>  print(singular.eval("matrix C = AGcode_L(G,D,HC);"))
```

(续下页)

(接上页)

```
Forms of degree 3 :
10
<BLANKLINE>
Vector basis successfully computed
<BLANKLINE>
>>>  # here is a linear code of type [8,5,> = 3] over F_4
>>>  print(singular.eval("print(C);"))
0,0,(a+1),(a),  1,  1,    (a),  (a+1),
1,0,(a),  (a+1),(a),(a+1),(a),  (a+1),
1,1,1,    1,    1,  1,    1,    1,
0,0,(a),  (a+1),1,  1,    (a+1),(a),
0,0,1,    1,    (a),(a+1),(a+1),(a)
```

这就是我们最终想要的生成矩阵，其中 a 表示基域 $GF(2)$ 上度为 2 的域扩张的生成器。

是否可以对其进行“封装”?

CHAPTER 14

接口问题

14.1 后台作业

没错，Sage 作业可以在 UNIX 系统上后台运行。通常做法是输入以下命令

```
$ nohup sage < command_file > output_file &
```

使用 nohup 的优势在于即使你注销后，Sage 仍会继续运行。

目前在 (unix) `top` 命令的输出中，Sage 会显示为”sage-ipython” 或”python”，但在 Sage 的未来版本中将显示为 `sage`。

14.2 引用 Sage

请参阅 citing Sage.

14.3 将 Sage 会话保存到日志

没错，你可以将会话保存到日志。

(a) 你可以通过在后台运行 Sage (后台作业) 将输出写入文件。

(b) 在 KDE konsole 中启动（仅适用于 Linux）。进入 `Settings` → `History ...` 并选择“无限制”。启动你的会话。准备就绪后，进入 `edit` → `save history as ...`。

某些接口（如 Singular 或 GAP 的接口）允许你创建日志文件。对于 Singular，有一个日志文件选项（在 `singular.py` 中）。在 GAP 中，使用命令 `LogTo`。

14.4 LaTeX 转换

没错，你可以将某些结果输出为 LaTeX。

```
sage: M  = MatrixSpace(RealField(),3,3)
sage: A  = M([1,2,3,  4,5,6,  7,8,9])
sage:  print(latex(A))
\left(\begin{array}{rrr}
    1.00000000000000 & 2.00000000000000 & 3.00000000000000 \\
    4.00000000000000 & 5.00000000000000 & 6.00000000000000 \\
    7.00000000000000 & 8.00000000000000 & 9.00000000000000
    \end{array}\right)
```

```
>>>  from  sage.all  import  *
>>> M  = MatrixSpace(RealField(),Integer(3),Integer(3))
>>> A  = M([Integer(1),Integer(2),Integer(3), Integer(4),Integer(5),Integer(6), Integer(7),
↪Integer(8),Integer(9)])
>>>  print(latex(A))
\left(\begin{array}{rrr}
    1.00000000000000 & 2.00000000000000 & 3.00000000000000 \\
    4.00000000000000 & 5.00000000000000 & 6.00000000000000 \\
    7.00000000000000 & 8.00000000000000 & 9.00000000000000
    \end{array}\right)
```

```
sage: view(A)
```

```
>>>  from  sage.all  import  *
>>> view(A)
```

此时会自动调用 dvi 预览，在单独的窗口中显示生成的 LaTeX 输出。

还可以预览多元多项式和有理函数的 LaTeX：

```
sage: x  = PolynomialRing(QQ,3,  'x').gens()
sage: f  = x[0]  + x[1]  -  2*x[1]*x[2]
sage: h  = f  /(x[1]  + x[2])
sage:  print(latex(h))
\frac{-2 x_{1} x_{2} + x_{0} + x_{1}}{x_{1} + x_{2}}
```

```
>>>  from  sage.all  import  *
>>> x  = PolynomialRing(QQ,Integer(3),  'x').gens()
>>> f  = x[Integer(0)]  + x[Integer(1)]  - Integer(2)*x[Integer(1)]*x[Integer(2)]
>>> h  = f  /(x[Integer(1)]  + x[Integer(2)])
>>>  print(latex(h))
\frac{-2 x_{1} x_{2} + x_{0} + x_{1}}{x_{1} + x_{2}}
```

14.5 Sage 与其他计算机代数系统

如果 `foo` 是 Pari、GAP（不以分号结尾）、Singular、Maxima 命令，对于 Pari，请输入 `gp("foo")`，对于 GAP，请输入 `libgap.eval("foo")`，对于 Singular，请输入 `singular.eval("foo")`，对于 Maxima，请输入 `maxima("foo")`。这些程序仅仅是将命令字符串发送到外部程序，并执行它，然后将结果读取回 Sage。因此，如果外部程序没有安装并包含在 PATH 中，这些程序将无法工作。

14.6 命令行上的 Sage 帮助

如果你只知道 Sage 命令的部分名称，但想知道它在 Sage 中的位置，只需输入 `sage -grep <string>` 即可在 Sage 源代码中查找所有出现`<string>` 的地方。例如，

```
$ sage -grep berlekamp_massey
matrix/all.py:from berlekamp_massey import berlekamp_massey
matrix/berlekamp_massey.py:def berlekamp_massey(a):
matrix/matrix.py:import berlekamp_massey
matrix/matrix.py:            g =
berlekamp_massey.berlekamp_massey(cols[i].list())
```

输入 `help(foo)` 或 `foo??` 获取帮助，输入 `foo.[tab]` 来搜索 Sage 命令。输入 `help()` 获取 Python 命令的帮助。

例如

```
help(Matrix)
```

会在新屏幕中返回

```
Help on cython_function_or_method in module sage.matrix.constructor:

matrix(*args, **kwds)
    matrix(*args, **kwds)
    File: sage/matrix/constructor.pyx (starting at line 21)

        Create a matrix.

        This implements the ``matrix`` constructor::

            sage: matrix([[1,2],[3,4]])
            [1 2]
            [3 4]

        It also contains methods to create special types of matrices, see
        ``matrix.[tab]`` for more options. For example::
--More--
```

输入 q 返回 Sage 屏幕。

14.7 读取和导入文件到 Sage

导入到 Sage 的文件必须以 *.py*‘ 结尾，例如 `foo.py`，并且包含合法的 Python 语法。前文置换群中的魔方群是一个简单示例。

另一种读取文件的方法是使用 `load` 或 `attach` 命令。创建一个名为 `example.sage` 的文件（位于 Sage 的主目录中），内容如下：

```
print("Hello World")
print(2^3)
```

使用 `load` 命令读取并执行 `example.sage` 文件：

```
sage: load("example.sage")
Hello World
8
```

```
>>> from sage.all import *
>>> load("example.sage")
Hello World
8
```

你也可以将 Sage 文件 attach 到正在运行的会话中：

```
sage: attach("example.sage")
Hello World
8
```

```
>>> from sage.all import *
>>> attach("example.sage")
Hello World
8
```

现在，如果你更改 example.sage 并在 Sage 中输入空行，那么 example.sage 的内容将自动重新加载到 Sage 中：

```
sage: !emacs example.sage&      #change 2^3 to 2^4
sage:                          #hit return
**************************************************
               Reloading 'example.sage'
**************************************************
Hello World
16
```

```
>>> from sage.all import *
>>> !emacs example.sage&      #change 2^3 to 2^4
>>>                          #hit return
**************************************************
               Reloading 'example.sage'
**************************************************
Hello World
16
```

14.8 Sage 命令的 Python 语言程序代码

假设你想知道 Sage 命令中用于计算置换群中心的 Python 程序是什么。可以使用 Sage 的帮助界面查找文件名：

```
sage: PermutationGroup.center?
Type:           instancemethod
Base Class:     <class 'instancemethod'>
String Form:    <unbound method PermutationGroup.center>
Namespace:      Interactive
File:           /home/wdj/sage/local/lib/python2.4/site-packages/sage/groups/permgroup.py
Definition:     PermutationGroup.center(self)
```

```
>>> from sage.all import *
>>> PermutationGroup.center?
Type:           instancemethod
Base Class:     <class 'instancemethod'>
String Form:    <unbound method PermutationGroup.center>
Namespace:      Interactive
```

(续下页)

(接上页)

```
File:           /home/wdj/sage/local/lib/python2.4/site-packages/sage/groups/permgroup.py
Definition:     PermutationGroup.center(self)
```

现在你知道该命令位于 permgroup.py 文件中，并且知道该 Python 模块的目录。你可以使用编辑器来阅读源代码。

14.9 Sage 中的“特殊函数”

Sage 有许多特殊函数（请参见参考手册 http://doc.sagemath.org/html/en/reference/functions/ ），并且大多数可以进行符号操作。如果尚未实现，则其他符号包可能具有此功能。

通过 Maxima，可以进行一些符号操作：

```
sage: maxima.eval("f:bessel_y (v, w)")
'bessel_y(v,w)'
sage: maxima.eval("diff(f,w)")
'(bessel_y(v-1,w)-bessel_y(v+1,w))/2'
sage: maxima.eval("diff (jacobi_sn (u, m), u)")
'jacobi_cn(u,m)*jacobi_dn(u,m)'
sage: jsn  =  lambda x: jacobi("sn",x,1)
sage: P  = plot(jsn,0,1, plot_points=20); Q  = plot(lambda x:bessel_Y(  1, x),  1/2,1)
sage: show(P)
sage: show(Q)
```

```
>>>  from  sage.all  import  *
>>> maxima.eval("f:bessel_y (v, w)")
'bessel_y(v,w)'
>>> maxima.eval("diff(f,w)")
'(bessel_y(v-1,w)-bessel_y(v+1,w))/2'
>>> maxima.eval("diff (jacobi_sn (u, m), u)")
'jacobi_cn(u,m)*jacobi_dn(u,m)'
>>> jsn  =  lambda x: jacobi("sn",x,Integer(1))
>>> P  = plot(jsn,Integer(0),Integer(1), plot_points=Integer(20)); Q  = plot(lambda x:bessel_Y(␣
↪Integer(1), x), Integer(1)/Integer(2),Integer(1))
>>> show(P)
>>> show(Q)
```

除了 maxima 外，pari 和 octave 也有特殊函数（实际上，Sage 封装了一些 pari 的特殊函数）。

下面是使用 Sage 接口（位于 sage/interfaces/octave.py）和 octave (https://www.gnu.org/software/octave/doc/latest) 的示例。

```
sage: octave("atanh(1.1)")    ## optional - octave
(1.52226,1.5708)
```

```
>>>  from  sage.all  import  *
>>> octave("atanh(1.1)")    ## optional - octave
(1.52226,1.5708)
```

下面是使用 Sage 接口调用 pari 特殊函数的示例。

```
sage: pari('2+I').besselk(3)
0.0455907718407551 + 0.0289192946582081*I
sage: pari('2').besselk(3)
0.0615104584717420
```

```
>>> from sage.all import *
>>> pari('2+I').besselk(Integer(3))
0.0455907718407551 + 0.0289192946582081*I
>>> pari('2').besselk(Integer(3))
0.0615104584717420
```

14.10 Sage 是什么？

Sage 是一个用来进行数论、代数和几何计算的框架，最初设计用于椭圆曲线和模形式的计算。其长期目标是使其更广泛地应用于代数、几何和数论。它是开源的，并根据 GPL 条款免费提供。可以从参考手册中的章节标题大致了解 Sage 涵盖的主题。

14.10.1 Sage 的历史

Sage 由 William Stein 于 2004 年秋在哈佛大学创立，0.1 版于 2005 年 1 月发布。该版本包括 Pari，但不包括 GAP 或 Singular。0.2 版于 3 月发布，0.3 版于 4 月发布，0.4 版于 7 月发布。在此期间，Sage 增加了对 Cremona 数据库、多元多项式和大型有限域的支持。此外，还编写了更多的文档。0.5 beta 版于 8 月发布，0.6 beta 版于 9 月发布，0.7 版于同月晚些时候发布。在此期间，增加了对向量空间、环、模符号和 Windows 用户的支持。自 2005 年 10 月发布 0.8 版本以来，Sage 包含了 GAP 和 Singular 的完整发行版，尽管某些 GAP 数据库需要单独添加。添加 Singular 并不容易，因为从源代码编译 Singular 非常困难。0.9 版于 11 月发布。该版本经历了 34 次发布！自 0.9.34 版（肯定在 0.10.0 版之前）以来，Maxima 和 clisp 被包含在 Sage 中。0.10.0 版于 2006 年 1 月 12 日发布。Sage 1.0 版于 2006 年 2 月初发布。截至 2008 年 2 月，最新版本为 2.10.2。

许多人贡献了重要的代码和其他专业技术，例如协助在各种操作系统上进行编译。一般来说，代码作者会在其 Python 文档的 AUTHOR 部分以及 Sage 网站的鸣谢部分予以体现。

CHAPTER 15

文档贡献

除了 William Stein 外，Gary Zablackis 也对本文档做出了贡献。

[CF] {CF} A. Campillo and J. I. Farran, *Symbolic Hamburger-Noether expressions of plane curves and applications to AG codes*, Math. Comp., vol 71(2002)1759-1780. http://www.ams.org/mcom/2002-71-240/S0025-5718-01-01390-4/home.html

[CO] {CO} H. Cohen, J. Oesterlé, *Dimensions des espaces de formes modulaires*, p. 69-78 in Modular functions in one variable VI. Lecture Notes in Math. 627, Springer-Verlag, NewYork, 1977.

[GAP] {GAP4} The GAP Group, *GAP - Groups, Algorithms, and Programming, Version 4.4*; 2005, https://www.gap-system.org

[G] {G} Solomon Golomb, *Shift register sequences*, Aegean Park Press, Laguna Hills, Ca, 1967

[Sing] {GPS05} G.-M. Greuel, G. Pfister, and H. Schönemann. Singular 3.0. A Computer Algebra System for Polynomial Computations. Centre for Computer Algebra, University of Kaiserslautern (2005). https://www.singular.uni-kl.de

[Pari] {PARI2} PARI/GP, version 2.1.5, Bordeaux, 2004, https://pari.math.u-bordeaux.fr/

[M] {M} James L. Massey, *Shift-Register Synthesis and BCH Decoding*, IEEE Trans. on Information Theory, vol. 15(1), pp. 122-127, Jan 1969.

[SAGE] {SJ} William Stein, David Joyner, SAGE: System for Algebra and Geometry Experimentation, Comm. Computer Algebra 39(2005)61-64. (SIGSAM Bull. June 2005) https://sagemath.org/ https://github.com/sagemath/sage/issues/

CHAPTER 16

索引与表格

- genindex
- search

索引

A

algebraic-geometric
    codes, 114
attach into Sage, 122

B

background, running Sage in, 119
Brauer
    character, 67

C

calculus
    critical points, 4
    differentiation, 3
    integration, 6
center
    groups, 25
character
    Brauer, 67
    modular representation, 67
characteristic polynomial
    matrix, 51
check matrix
    codes, 56
ciphers
    connection polynomial, 59
class_number
    number field, 98
codes
    algebraic-geometric, 114
    check matrix, 56
    dual, 56
    generator matrix, 56
    Golay, 56
conjugacy classes
    group, 23
connection polynomial
    ciphers, 59
cosets of Gamma_0, 89
critical points
    calculus, 4
cyclotomic
    number field, 100

D

database
    number field, 97
differentiation
    calculus, 3
discrete logs, 85
discriminant
    number field, 97
dual
    codes, 56

E

eigenvalues, 47
eigenvectors, 47
elliptic curve
    modular form, 95
    point counting, 103
elliptic curves, 92, 94
evaluation
    polynomial, 81

F

factorization
    polynomial, 78
Frobenius normal form, 47

G

gcd
    polynomial, 79
generator matrix
    codes, 56
Golay
    codes, 56
group

conjugacy classes, 23
normal subgroups, 24
permutation, 21
Rubik's cube, 21
groups
center, 25

H

Hecke operators, 90
help in Sage, 120
Hermite normal form, 47
history
Sage, 124

I

importing into Sage, 121
integral basis
number field, 100
integration
calculus, 6

L

LaTeX output, 119
linear equations
solve, 52
load into Sage, 121

M

matrix
characteristic polynomial, 51
ring, 71
modular form
elliptic curve, 95
modular forms, 88
modular representation
character, 67
modular symbols, 90

N

normal subgroups
group, 24
number field
class_number, 98
cyclotomic, 100
database, 97
discriminant, 97
integral basis, 100

P

p-adics, 73
permutation
group, 21
plot
a function, 18
a parametric curve, 18
curve using surf, 16
point counting
elliptic curve, 103
polynomial
evaluation, 81
factorization, 78
gcd, 79
powers, 77
quotient ring, 74
ring, 72
roots, 81
symbolic manipulation, 82
power series, 5
powers
polynomial, 77
Python and Sage, 122

Q

quadratic residues, 87
quotient ring
polynomial, 74

R

rational canonical form, 47
referencing
Sage, 119
Riemann-Roch space, 112, 115
ring
matrix, 71
polynomial, 72
roots
polynomial, 81
Rubik's cube
group, 21

S

Sage
history, 124
referencing, 119
Smith normal form, 47
solve
linear equations, 52
special functions in Sage, 123
symbolic manipulation
polynomial, 82

T

Taylor series, 5